# TEMA 5

# EXPLOTACIÓN DE PARALELISMO

# TEMA 5

# EXPLOTACIÓN DE PARALELISMO

## ARQUITECTURA DE COMPUTADORES

*Dpto. Arquitectura de Computadores*

*Universidad de Málaga*

# CONTENIDOS

- Introducción
- Paralelismo de instrucciones
- Paralelismo de datos: Vectorial, GPU
- Paralelismo multi-hilo

Transparencias basadas en:

- MªÁngeles González. Estructuras de Computadores. UMA.
- Oscar Plata. Arquitecturas Paralelas. UM A

# INTRODUCCIÓN

TEMA 5: EXPLOTACIÓN DE PARALELISMO

# PARALLELISM EVERYWHERE

*Software* | *Hardware*

- Parallel Requests
  - Assigned to computer
  - e.g. search “Garcia”
- Parallel Threads
  - Assigned to core
  - e.g. lookup, ads
- Parallel Instructions
  - > 1 instruction @ one time
  - e.g. 5 pipelined instructions
- Parallel Data
  - > 1 data item @ one time
  - e.g. add of 4 pairs of words
- Hardware descriptions
  - All gates functioning in parallel at same time

*Leverage Parallelism & Achieve High Performance*

Warehouse Scale Computer

Smart Phone

# PROCESSORS

- Many kinds of processors:


CPU

GPU


FPGA

- What differentiates these processors?

# PROCESSOR EXECUTION

- Each processor is designed for different kinds of programs
- **CPUs**
  - Programs for general purpose computation: i.e., single / few threads
- **GPUs**
  - Programs with lots of independent work: i.e., many threads
- **Other processors**
  - Deep Neural Networks, Digital Signal Processing, …

**Thus, processors exploit different kinds of parallelism**

# CURRENT TRENDS IN ARCHITECTURE

- Generalized exploitation of parallelism to improve performance and reduce power consumption
- Classes of **application parallelism**
  - Data-level parallelism: DLP
  - Task-level parallelism: TaLP
    - Special case: Request-level parallelism: RLP
- Classes of **architectural parallelism**
  - Instruction-level parallelism: ILP
  - Vector architectures / Graphic Processor Units (GPUs): DLP
  - Thread-level parallelism: TLP
  - Process-level parallelism: PLP
- All of them exploit some type of parallelism, but at the cost of requiring some (manual/automatic) restructuring of the application

# PARALLEL PROCESSING

- Where is the parallelism? It depends on the application domain
- **CPUs**
  - Instruction-level parallelism (ILP)
  - Data-level parallelism (DLP)
  - Thread-level parallelism (TLP)
- **GPUs**
  - Data-level parallelism (DLP)
  - Thread-level parallelism (TLP)

# PARALLEL PROCESSING

- Whose job is it to find parallelism?
  - ILP: hardware dynamically schedules instructions
  - DLP: software makes parallelism explicit (typically, compiler)
  - TLP: software makes parallelism explicit (typically, programmer)
- **CPUs**
  - Expensive, complex hardware ➡ Few cores (tens)
  - (Relatively) easy to write fast software
- **GPUs**
  - Simple, cheap hardware ➡ Many cores (thousands)
  - (Often) hard to write fast software

# PARALELISMO DE INSTRUCCIONES (ILP)

## INSTRUCTION-LEVEL PARALLELISM

TEMA 5: EXPLOTACIÓN DE PARALELISMO

# PARALLELISM EVERYWHERE

*Software* | *Hardware*

- Parallel Requests
  - Assigned to computer
  - e.g. search “Garcia”
- Parallel Threads
  - Assigned to core
  - e.g. lookup, ads
- Parallel Instructions
  - > 1 instruction @ one time
  - e.g. 5 pipelined instructions
- Parallel Data
  - > 1 data item @ one time
  - e.g. add of 4 pairs of words
- Hardware descriptions
  - All gates functioning in parallel at same time

*Leverage Parallelism & Achieve High Performance*

We are here

Warehouse Scale Computer

Smart Phone

# HOW CPUs EXPLOIT PARALLELISM

- Example program: polynomial evaluation
  - Compute polinomial: $value = coef_0 + coef_1 * x + coef_2 * x^2 + coef_3 * x^3 + \cdots$

### C program

```
int poly(int *coef, int terms, int x)
{
  int power = 1;
  int value = 0;
  for (int j = 0; j < terms; j++) {
    value += coef[j] * power;
    power *= x;
  }
  return value;
}
```

```
r0: value
r1: &coef[terms]
r2: x
r3: &coef[0]
r4: power
r5: coef[j]
```

### ARM code program

```
poly:
  cmp    r1, #0
  ble    .L4
  push   {r4, r5}
  mov    r3, r0
  add    r1, r0, r1, lsl #2
  movs   r4, #1
  movs   r0, #0
.L3:
  ldr    r5, [r3], #4
  cmp    r1, r3
  mla    r0, r4, r5, r0
  mul    r4, r2, r4
  bne    .L3
  pop    {r4, r5}
  bx     lr
.L4:
  movs   r0, #0
  bx     lr
```

# HOW CPUs EXPLOIT PARALLELISM

- Example program: polynomial evaluation
  - Compute polinomial: $value = coef_0 + coef_1 * x + coef_2 * x^2 + coef_3 * x^3 + \cdots$

### C program

```
int poly(int *coef, int terms, int x)
{
  int power = 1;
  int value = 0;
  for (int j = 0; j < terms; j++) {
    value += coef[j] * power;
    power *= x;
  }
  return value;
}
```

```
r0: value
r1: &coef[terms]
r2: x
r3: &coef[0]
r4: power
r5: coef[j]
```

### ARM code program

```
poly:
  cmp    r1, #0
  ble    .L4
  push   {r4, r5}
  mov    r3, r0
  add    r1, r0, r1, lsl #2
  movs   r4, #1
  movs   r0, #0
.L3:
  ldr    r5, [r3], #4
  cmp    r1, r3
  mla    r0, r4, r5, r0
  mul    r4, r2, r4
  bne    .L3
  pop    {r4, r5}
  bx     lr
.L4:
  movs   r0, #0
  bx     lr
```

# HOW CPUs EXPLOIT PARALLELISM

- Example program: polynomial evaluation
  - Compute polinomial: $value = coef_0 + coef_1 * x + coef_2 * x^2 + coef_3 * x^3 + \cdots$

**C program**

```
for (int j = 0; j < terms; j++) {
    value += coef[j] * power;
    power *= x;
  }
```

```
.L3:
  ldr     r5, [r3], #4      // r5 <- coef[r3]; r3 += 4
  cmp     r1, r3            // compare: r3 < &coef[terms] ?
  mla     r0, r4, r5, r0    // value += r5 * power (mul + add)
  mul     r4, r2, r4        // power *= x
  bne     .L3               // repeat?
```

**ARM code program**

```
r0: value
r1: &coef[terms]
r2: x
r3: &coef[0]
r4: power
r5: coef[j]
```

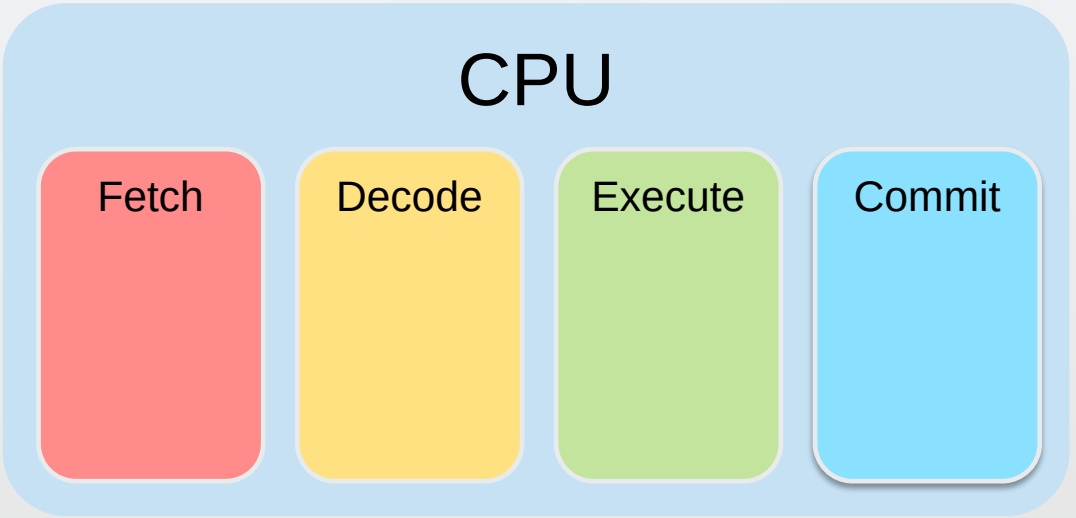
# SIMPLE CPU MODEL: NO PARALLELISM

- Execute instructions in program order
- Divide instruction execution into stages, e.g.:
  - FETCH: Get next instruction from memory
  - DECODE: Figure out what to do & read inputs
  - EXECUTE: Perform the necessary operations
  - COMMIT (Write-back): Write the results back to registers / memory

# SIMPLE CPU MODEL: NO PARALLELISM

- Evaluating polynomial on the simple CPU model

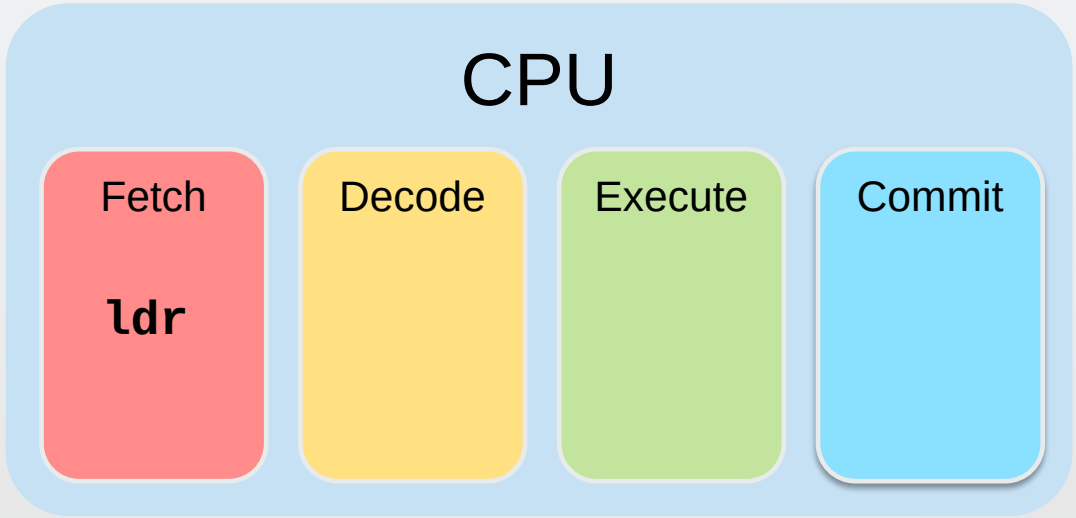
```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```



# SIMPLE CPU MODEL: NO PARALLELISM

- Evaluating polynomial on the simple CPU model

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

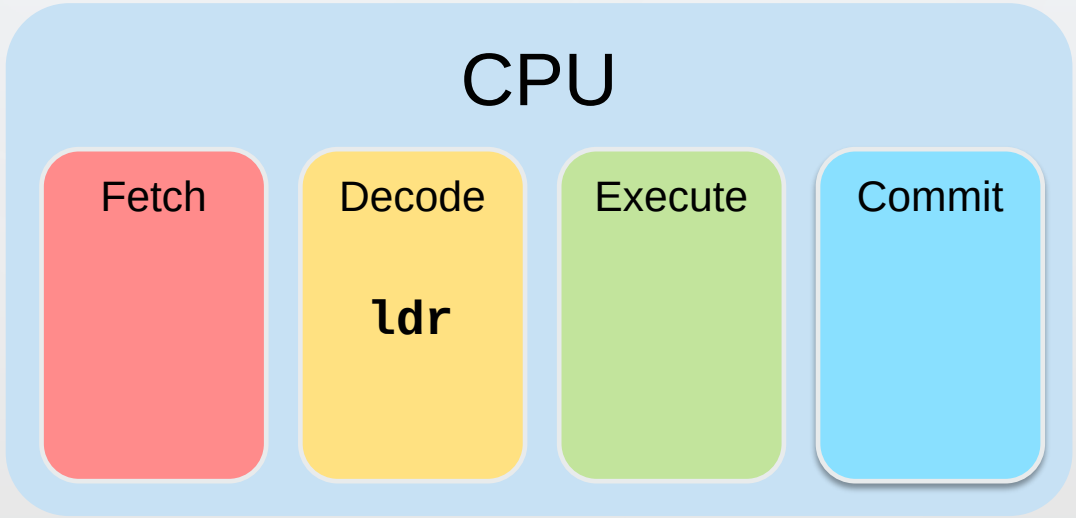



**1. Read "ldr r5, [r3] #4" from memory**

# SIMPLE CPU MODEL: NO PARALLELISM

- Evaluating polynomial on the simple CPU model

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

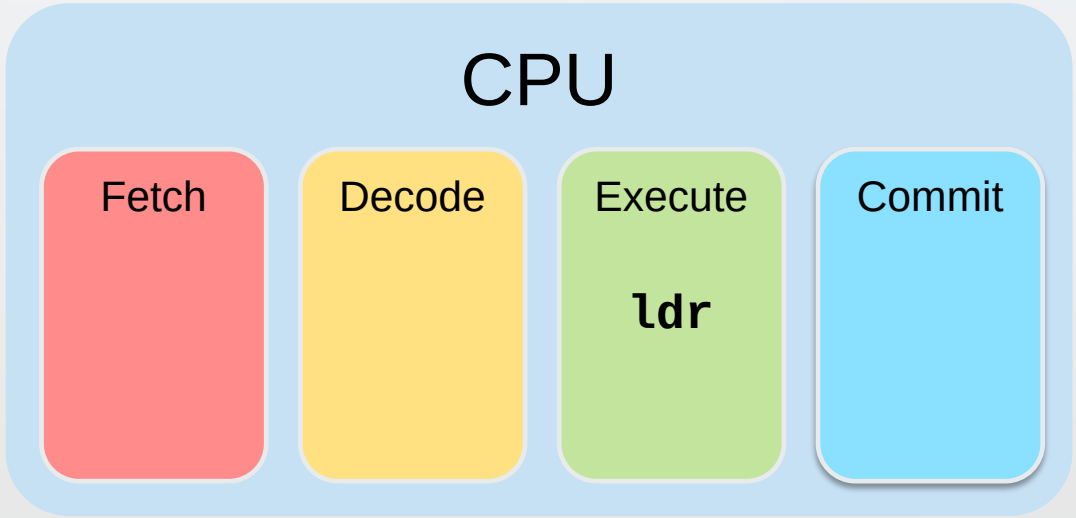



**2. Decode “ldr r5, [r3] #4” and read input regs**

# SIMPLE CPU MODEL: NO PARALLELISM

- Evaluating polynomial on the simple CPU model

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

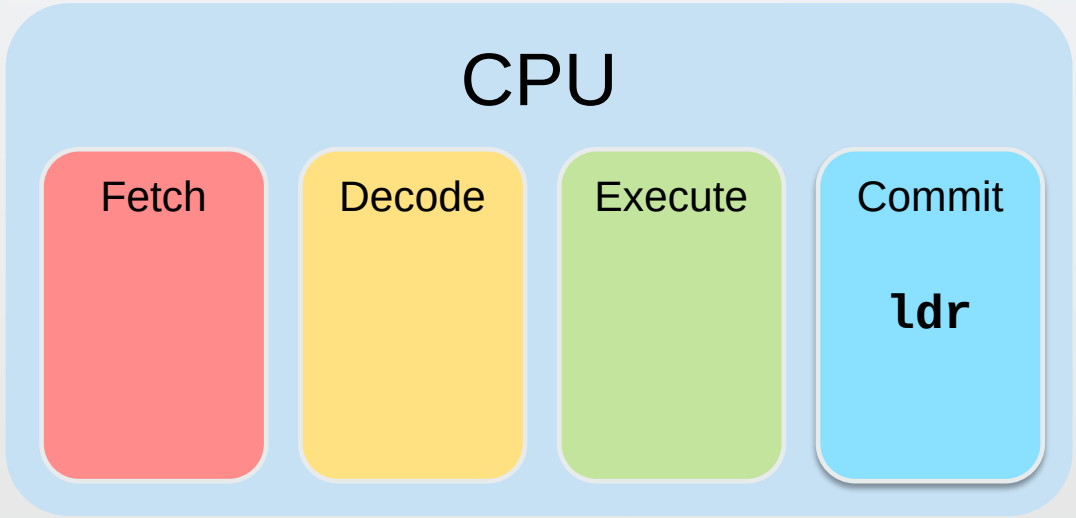



**3. Load memory at r3 and compute r3 + 4**

# SIMPLE CPU MODEL: NO PARALLELISM

- Evaluating polynomial on the simple CPU model

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

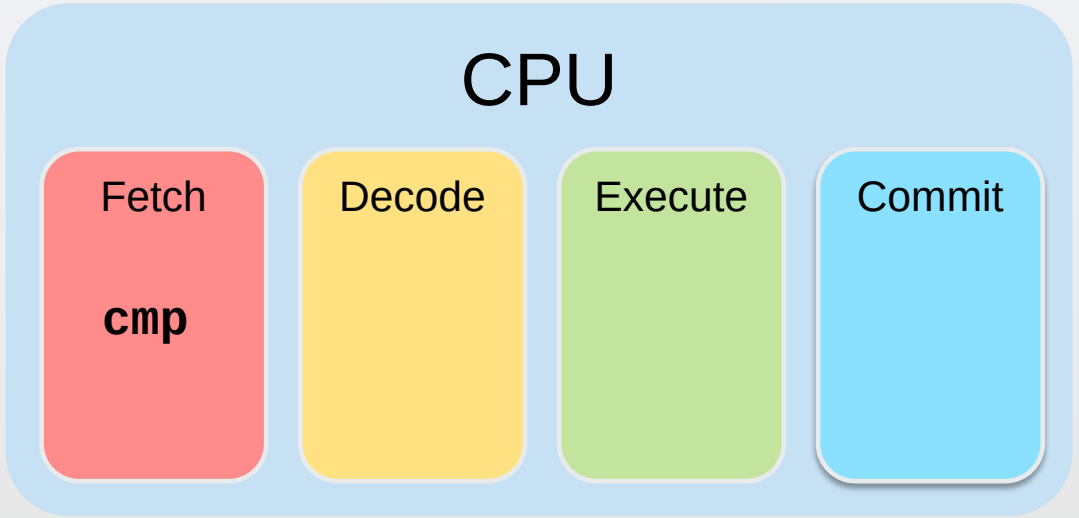



**4. Write values into regs r5 and r3**

# SIMPLE CPU MODEL: NO PARALLELISM

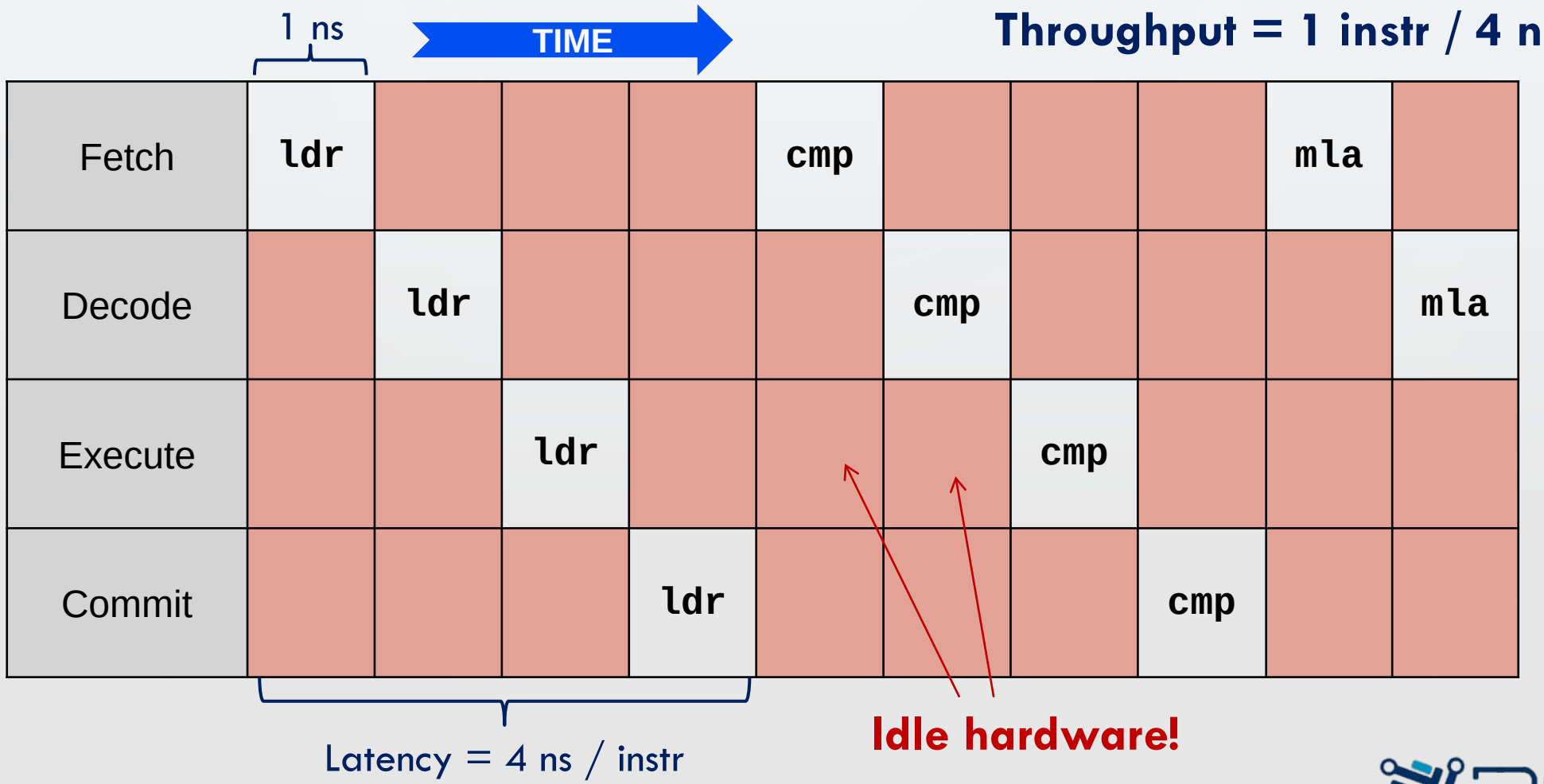
- Evaluating polynomial on the simple CPU model

```
.L3:
    ldr     r5, [r3], #4
    cmp     r1, r3
    mla     r0, r4, r5, r0
    mul     r4, r2, r4
    bne     .L3
```



# SIMPLE CPU MODEL: NO PARALLELISM

- Evaluating polynomial on the simple CPU model

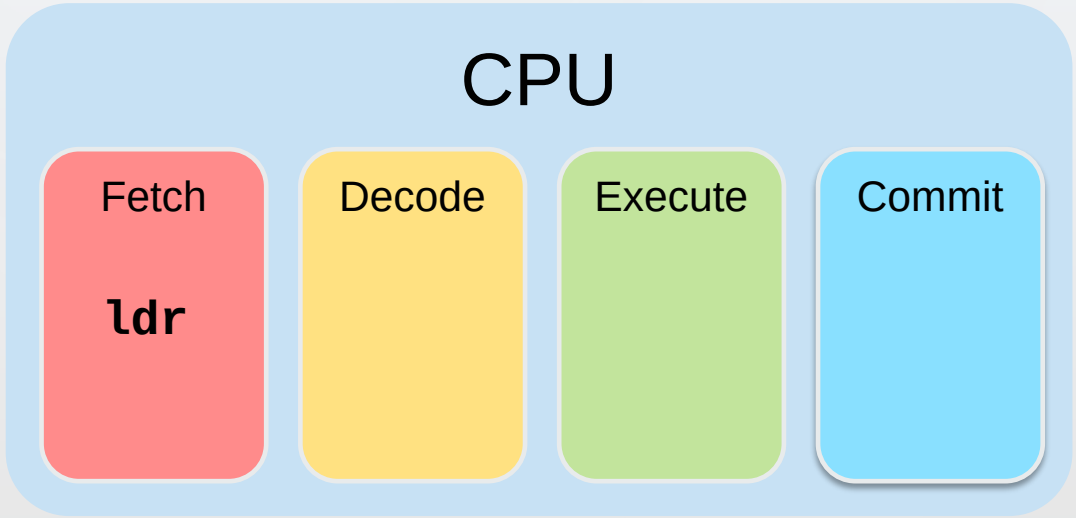


# CPU MODEL: PIPELINIG

- Evaluating polynomial on the CPU model
  - **Pipelining:** start on the next instruction immediately

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

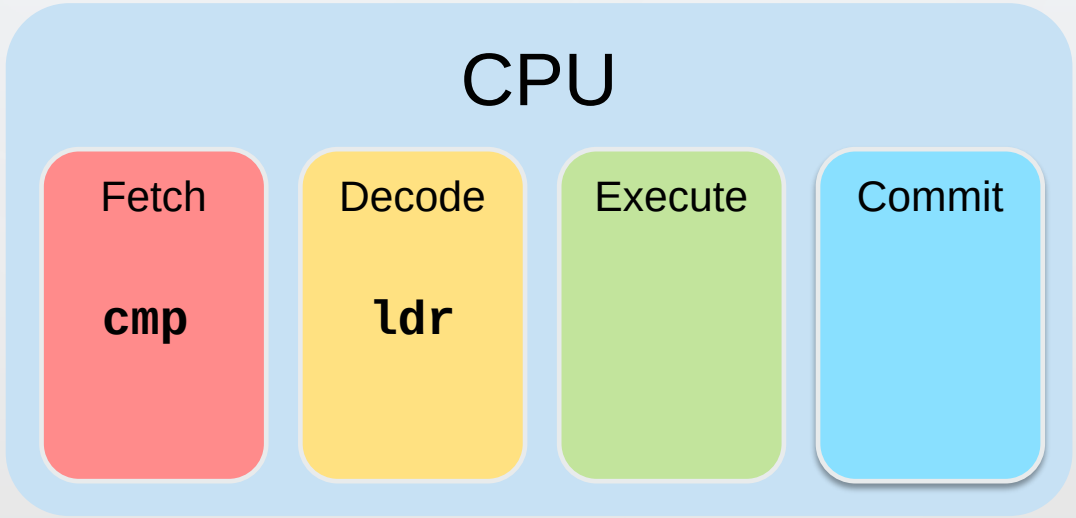


# CPU MODEL: PIPELINIG

- Evaluating polynomial on the CPU model
  - **Pipelining:** start on the next instruction immediately

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

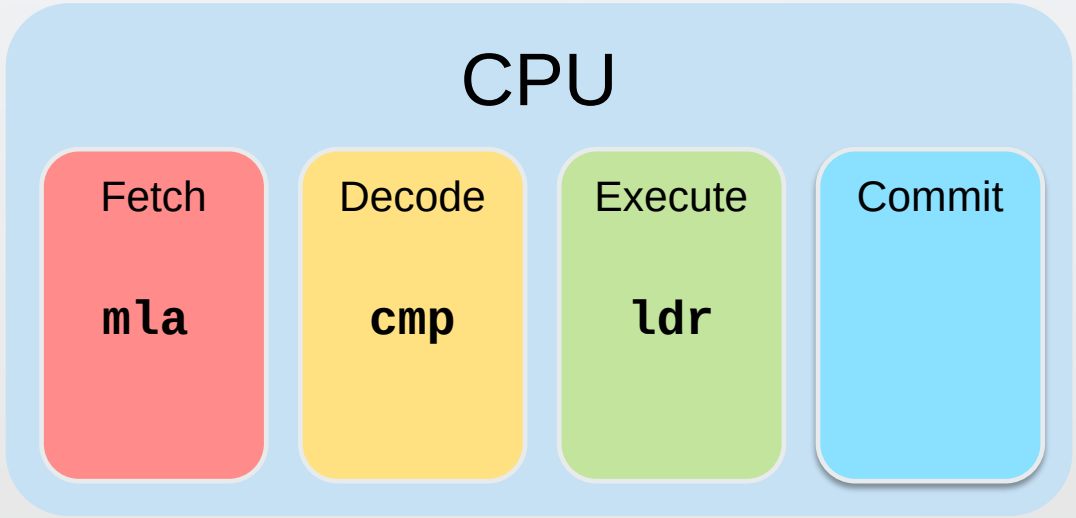


# CPU MODEL: PIPELINIG

- Evaluating polynomial on the CPU model
  - **Pipelining:** start on the next instruction immediately

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

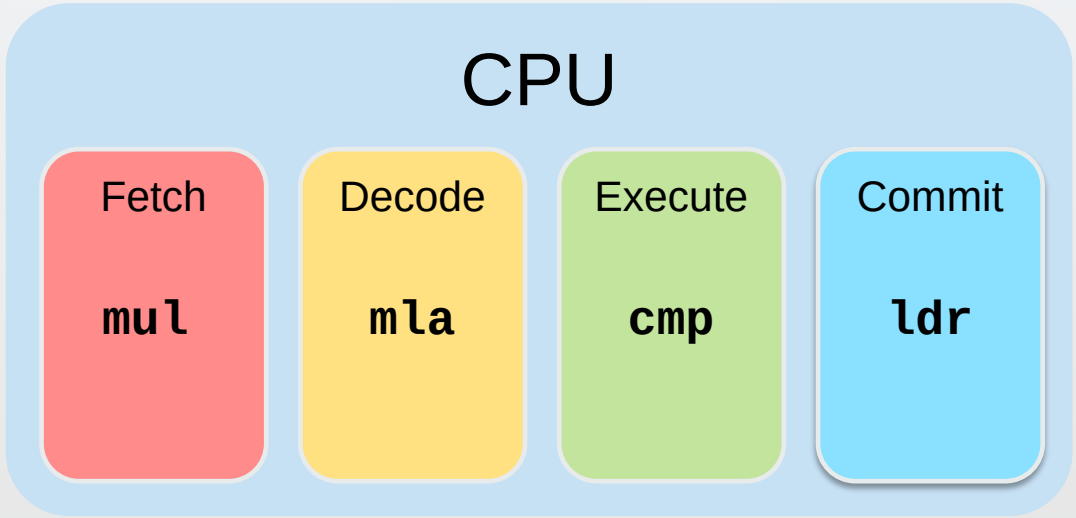


# CPU MODEL: PIPELINIG

- Evaluating polynomial on the CPU model
  - **Pipelining:** start on the next instruction immediately

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

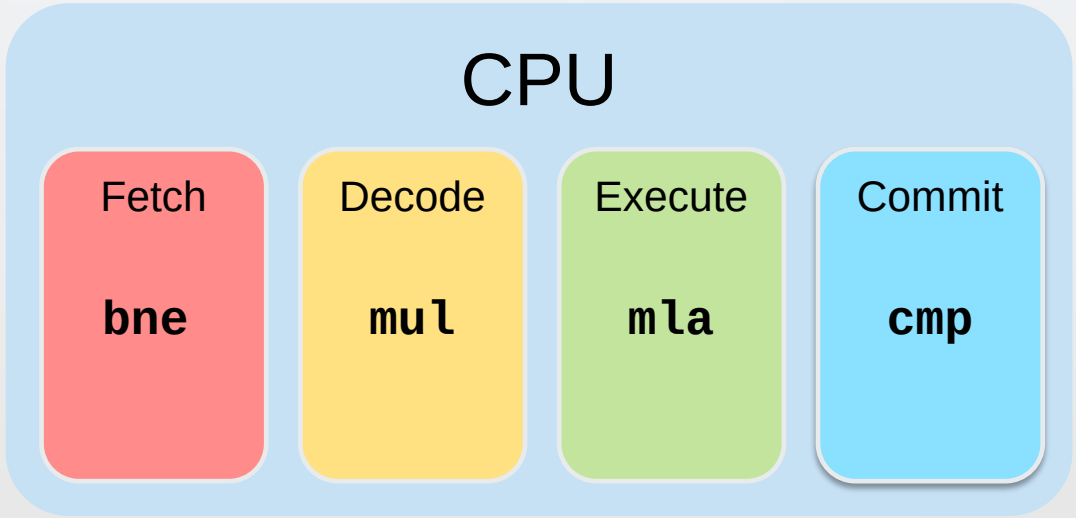


# CPU MODEL: PIPELINIG

- Evaluating polynomial on the CPU model
  - **Pipelining:** start on the next instruction immediately

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

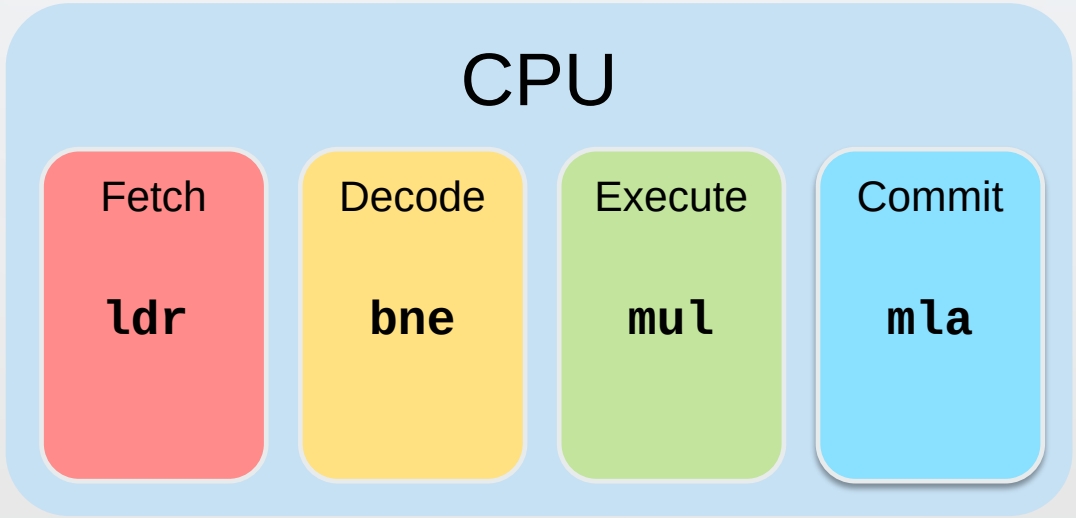


# CPU MODEL: PIPELINIG

- Evaluating polynomial on the CPU model
  - **Pipelining:** start on the next instruction immediately

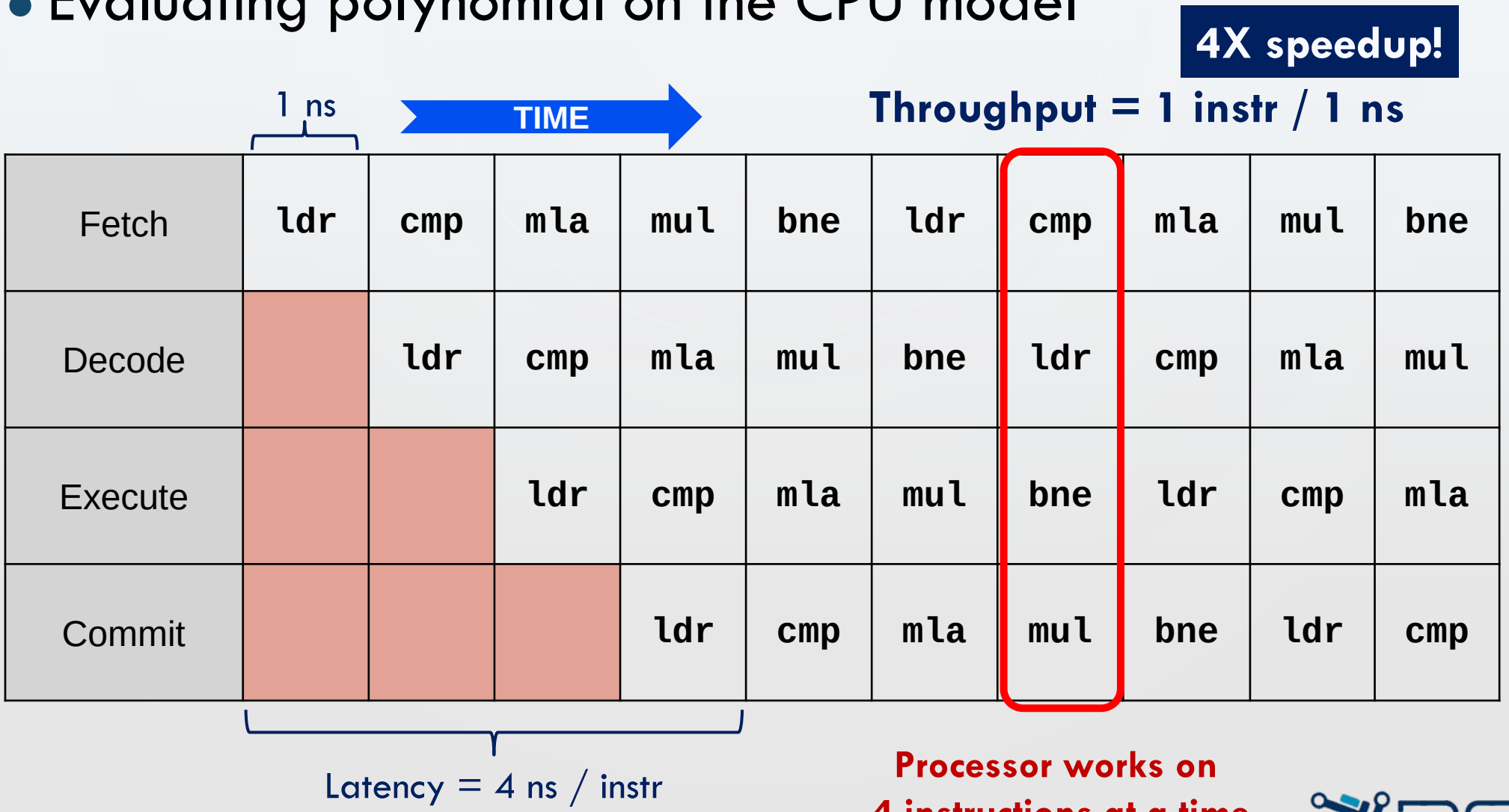
```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```



# EXECUTION TIMING: PIPELINING

- Evaluating polynomial on the CPU model

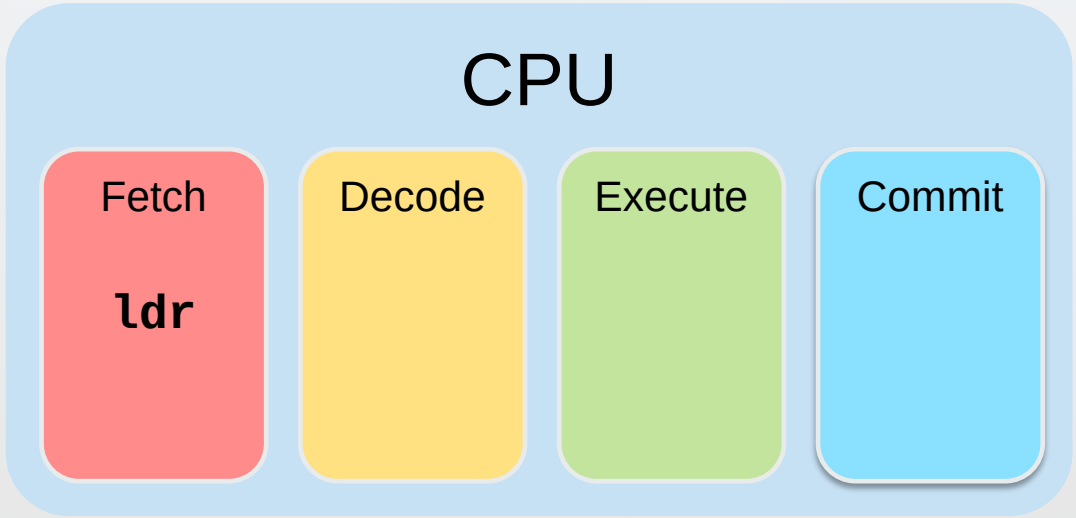


# LIMITATIONS OF PIPELINIG

- Pipelining requires **independent** work
  - **Data hazards** limit parallelism: introduce **stalls** in the execution

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

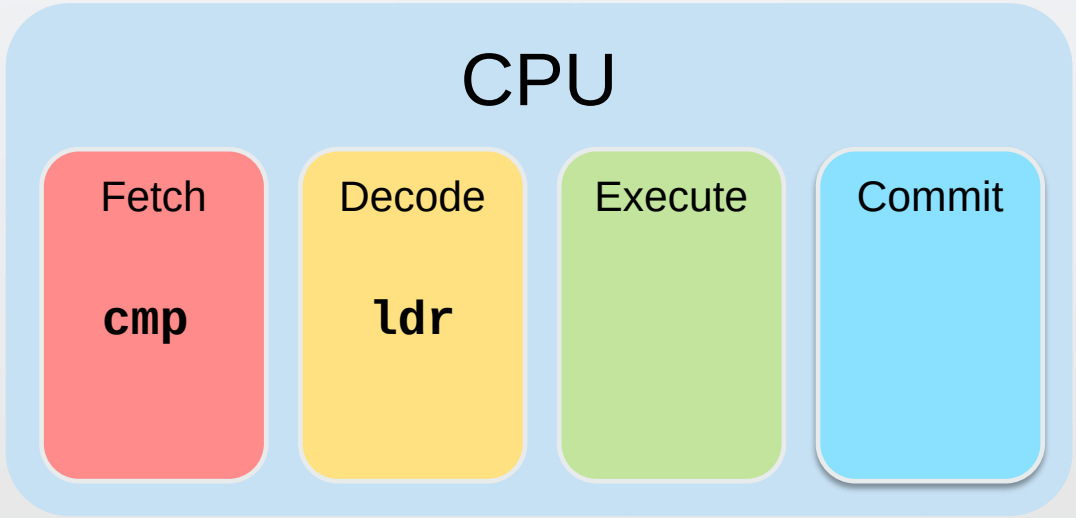


# LIMITATIONS OF PIPELINIG

- Pipelining requires **independent** work
  - **Data hazards** limit parallelism: introduce **stalls** in the execution

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

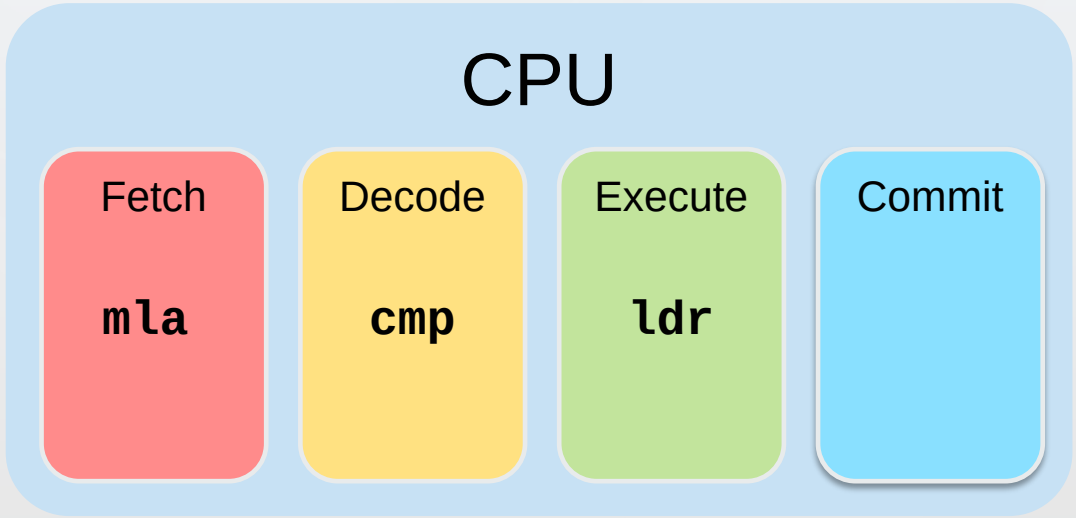


# LIMITATIONS OF PIPELINIG

- Pipelining requires **independent** work
  - **Data hazards** limit parallelism: introduce **stalls** in the execution

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

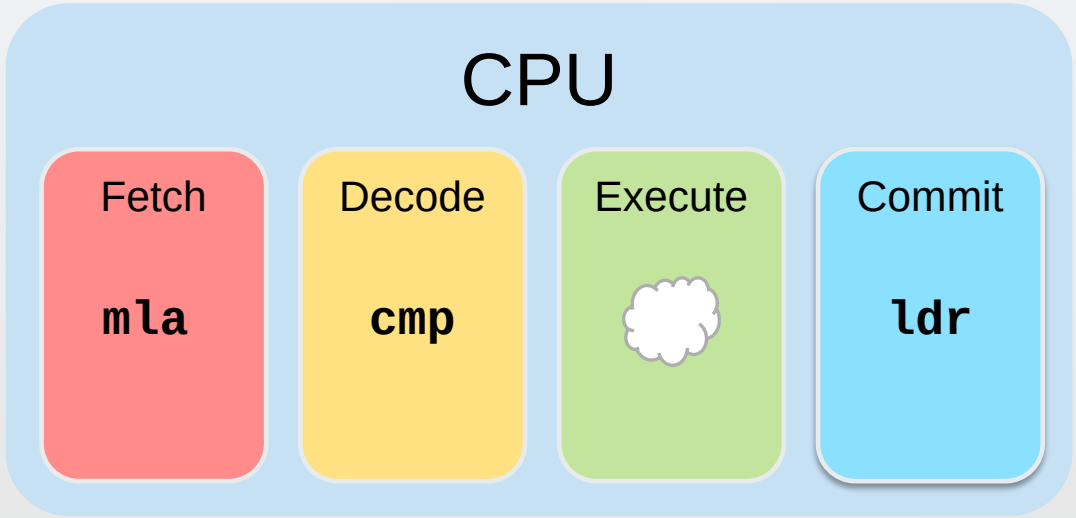


# LIMITATIONS OF PIPELINIG

- Pipelining requires **independent** work
  - **Data hazards** limit parallelism: introduce **stalls** in the execution

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```



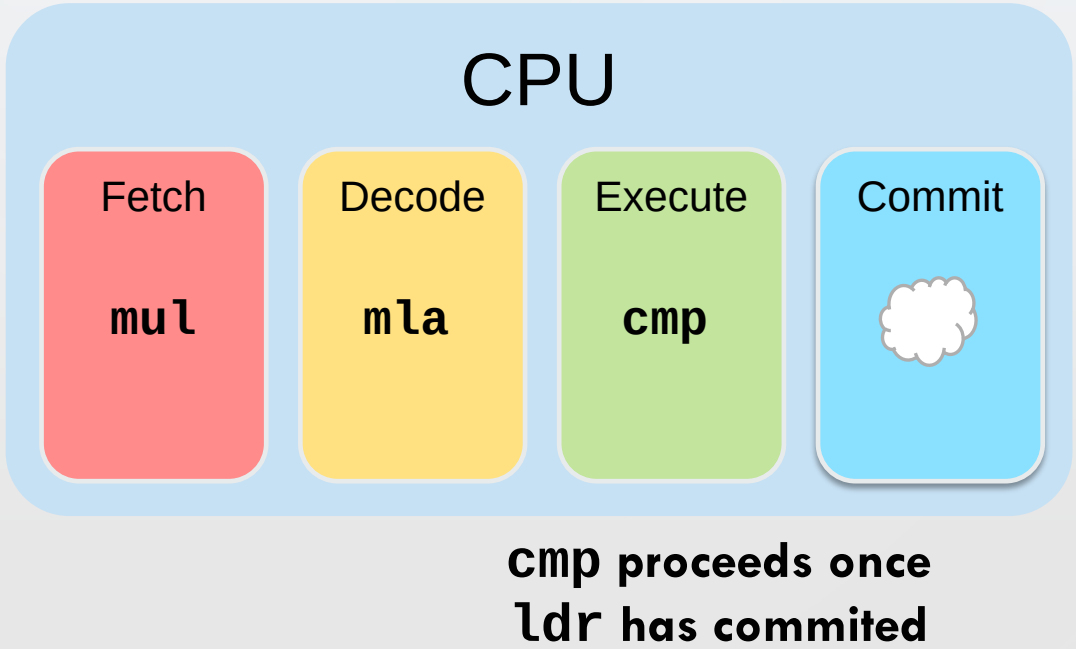

**Inject a "bubble" (NOP) into the pipeline**

# LIMITATIONS OF PIPELINIG

- Pipelining requires **independent** work
  - Data **hazards** limit parallelism: introduce **stalls** in the execution

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

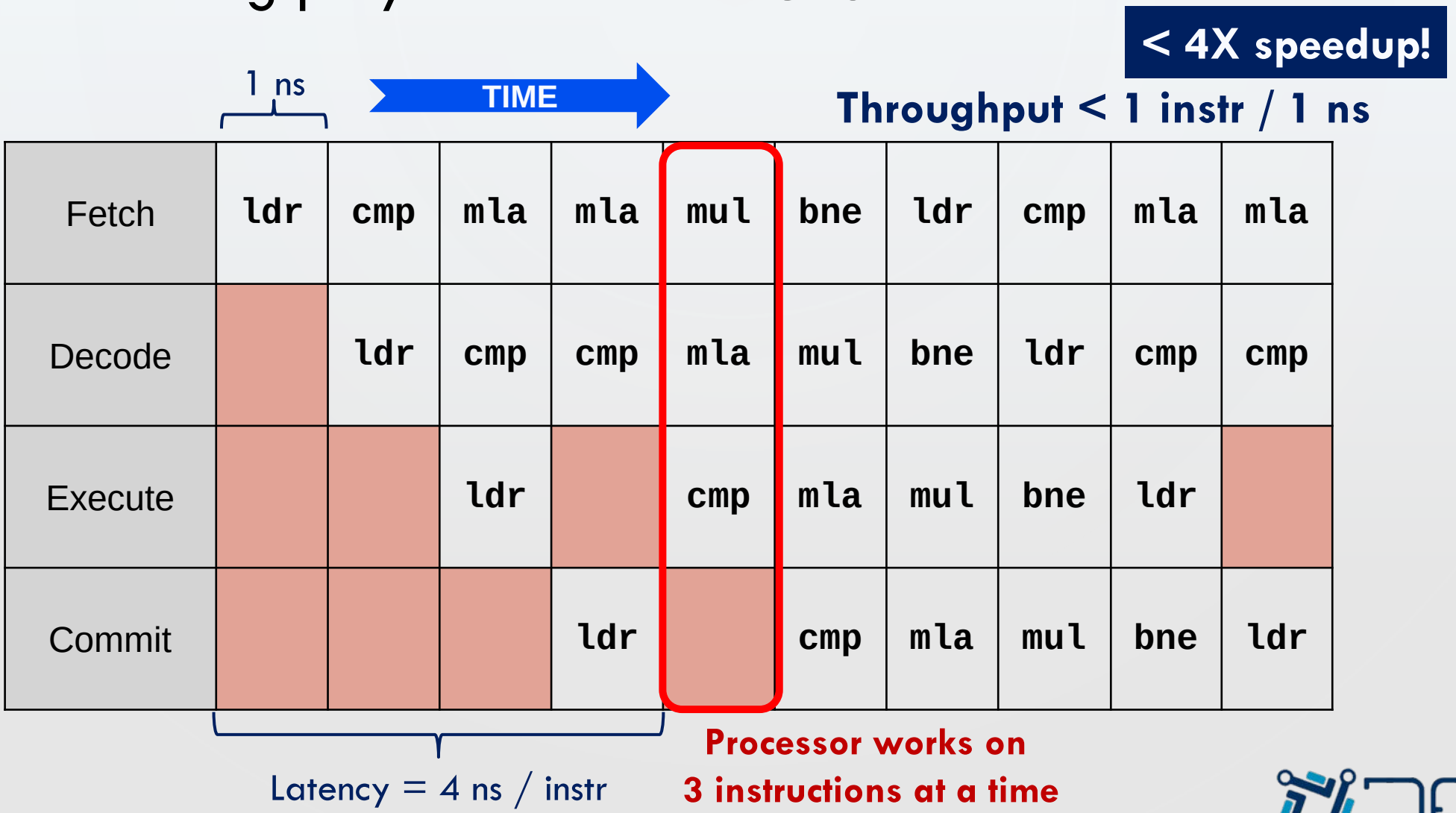
```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```




**`cmp` proceeds once `ldr` has commited**

# PIPELINING: STALLS DEGRADES PERFORMANCE

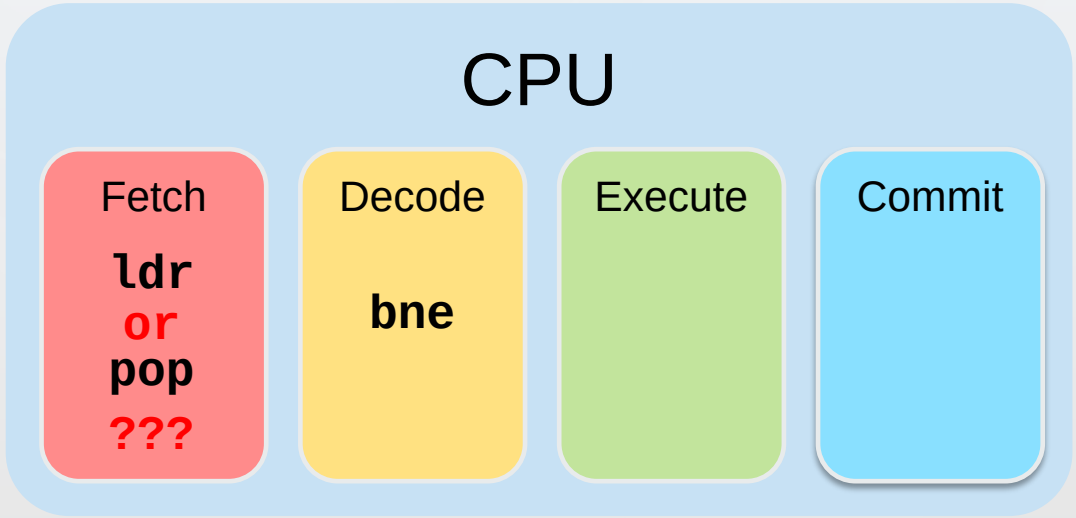
- Evaluating polynomial on the CPU model



# LIMITATIONS OF PIPELINIG

- Pipelining requires **independent** work
  - Control hazards limit parallelism: introduce stalls in the execution

```
.L3:
    ldr     r5, [r3], #4
    cmp     r1, r3
?   mla     r0, r4, r5, r0
    mul     r4, r2, r4
    bne     .L3
?
    pop     {r4, r5}
    bx      lr
```

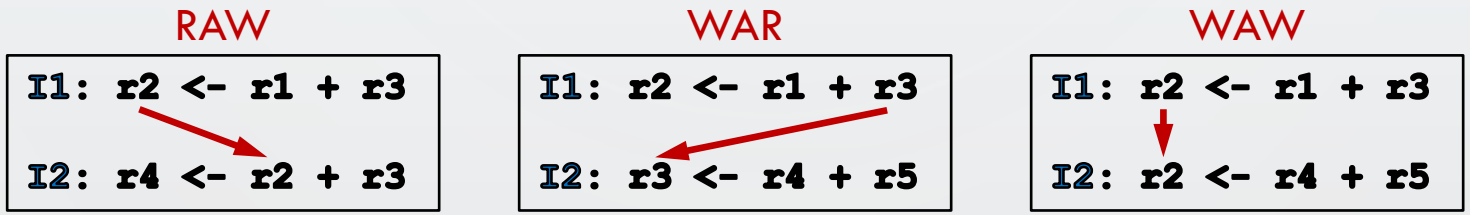


# PIPELINING

- Dealing with **data hazards**
  - Forwarding data
- Dealing with **control hazards**
  - Speculation on the next instruction to execute after a branch
  - Pipeline must be flushed when speculation fails
- Summary
  - Pipelining is a simple, effective way to improve throughput
  - Pipelining has limits
    - Hard to keep pipeline busy because of hazards
    - Forwarding is expensive in deep pipelines
    - Pipeline flushes are expensive in deep pipelines
  - **Pipelining is ubiquitous, but tops out at ~15 stages**

# INCREASING PARALLELISM VIA DATAFLOW

- **Data hazards** when using pipelining
  - It causes stalls in the execution, limiting parallelism
  - Besides, many data hazards come from **false data dependencies** due to the **sequential program order**




- **Dataflow** instruction execution into stages, e.g.:
  - It tracks how instructions actually depend on each other
  - Only **true data dependences** limit parallelism
- Dataflow increases parallelism by eliminating unnecessary dependences

# INCREASING PARALLELISM VIA DATAFLOW

- Evaluating polynomial using **dataflow** execution

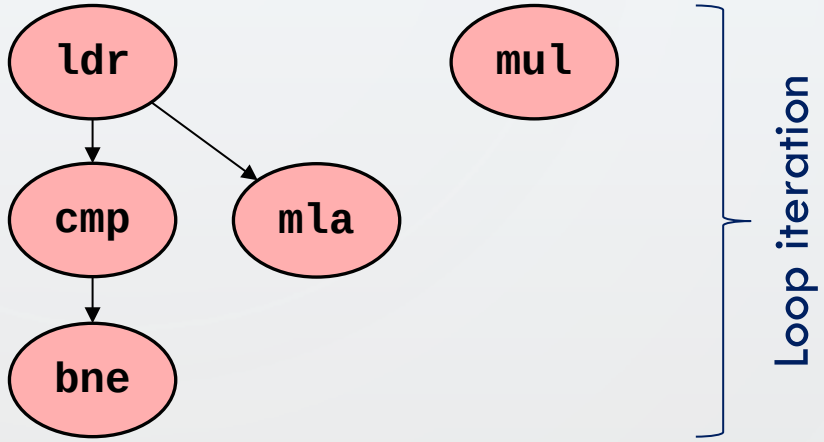
**Only true data dependences considered**

Iter. n

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

Iter. n+1

```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```



# INCREASING PARALLELISM VIA DATAFLOW

- Evaluating polynomial using **dataflow** execution

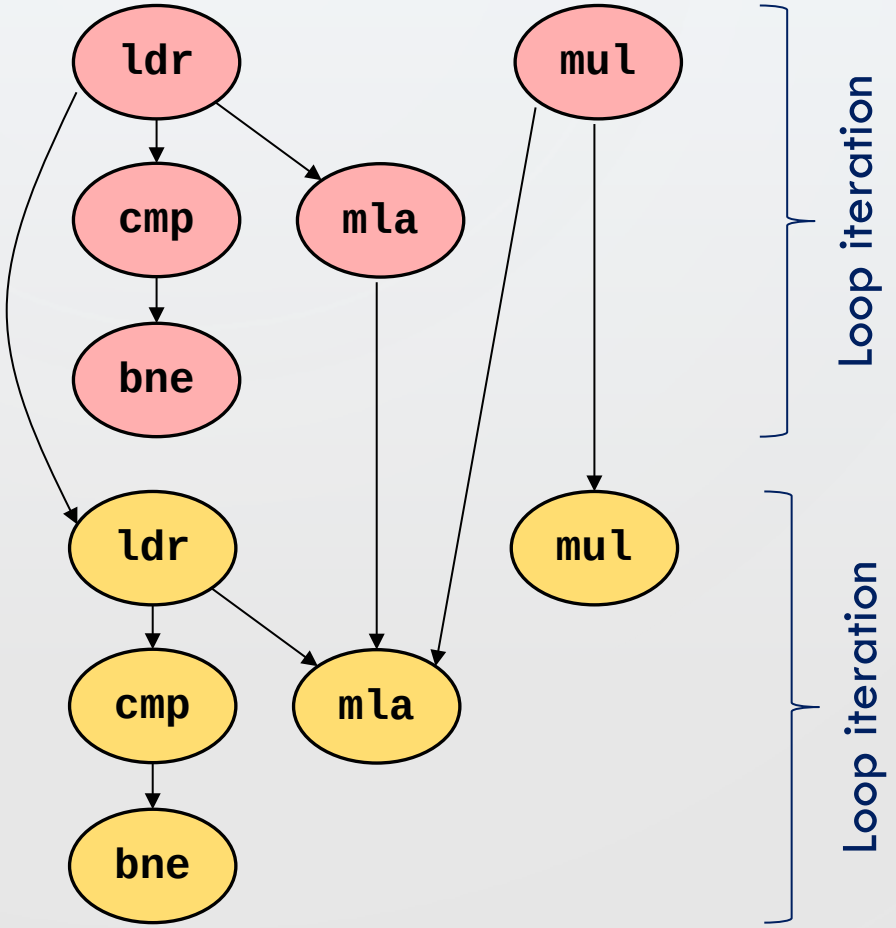
**Only true data dependences considered**

Iter. n

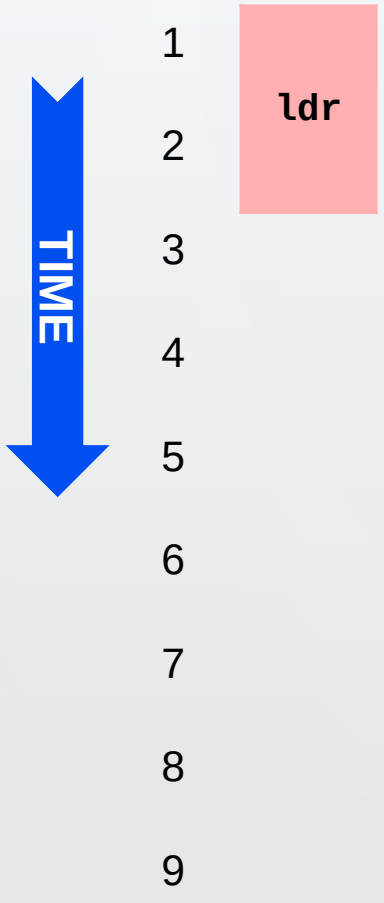
```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```

Iter. n+1

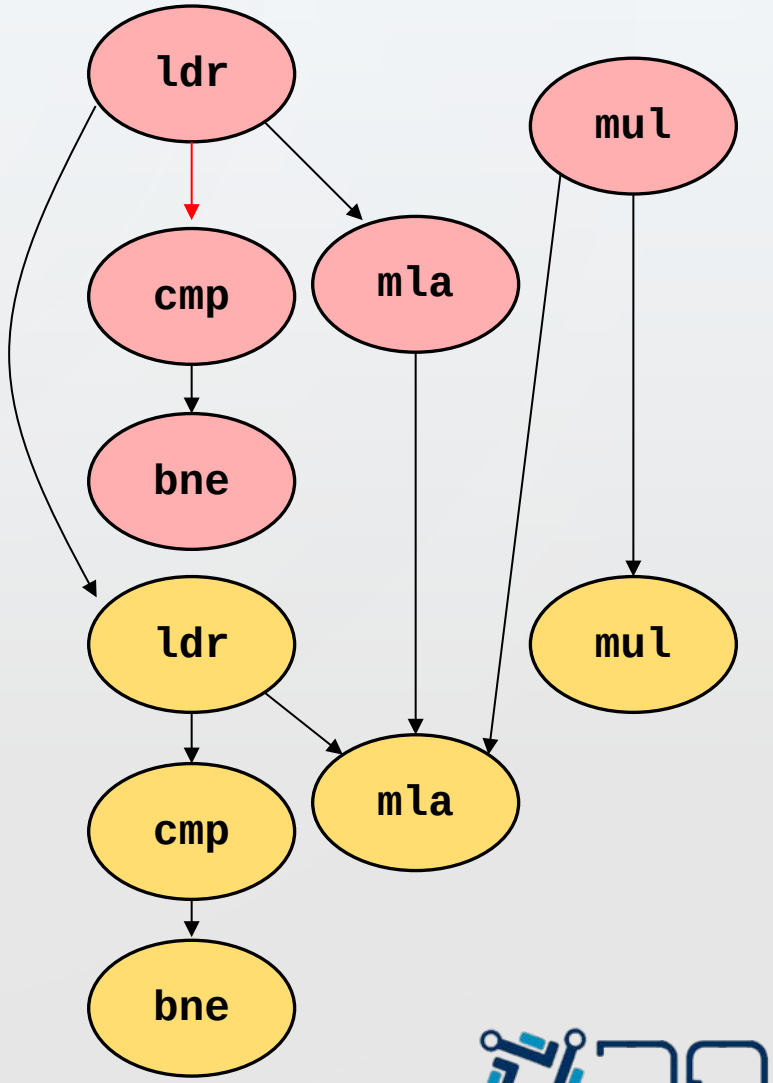
```
.L3:
  ldr     r5, [r3], #4
  cmp     r1, r3
  mla     r0, r4, r5, r0
  mul     r4, r2, r4
  bne     .L3
```



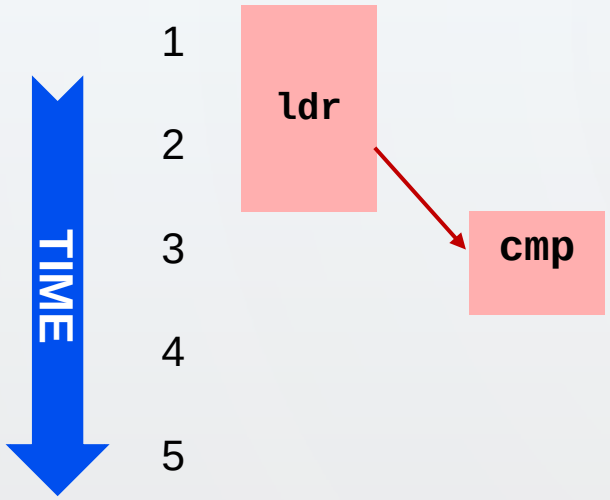
# DATAFLOW EXECUTION



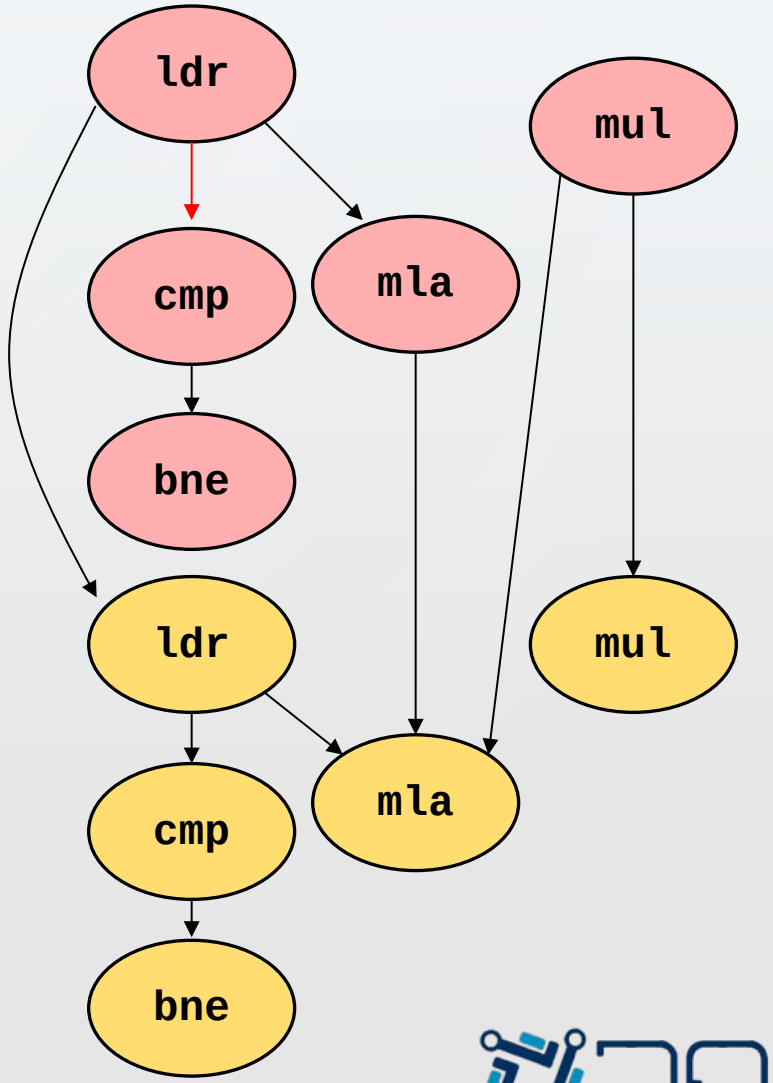

**ldr**, **mul**: 2 cycles
**cmp**, **bne**: 1 cycle
**mla**: 3 cycles



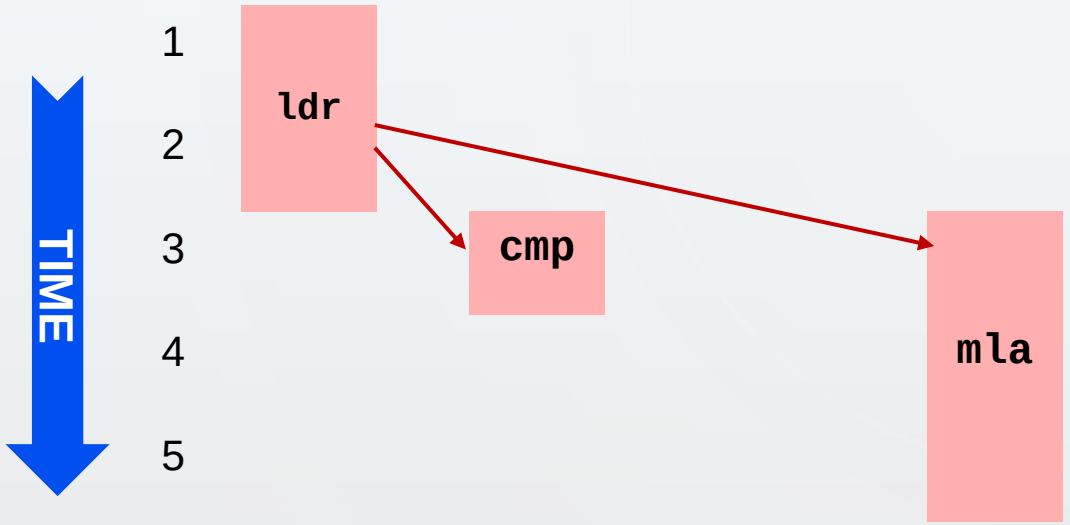
# DATAFLOW EXECUTION



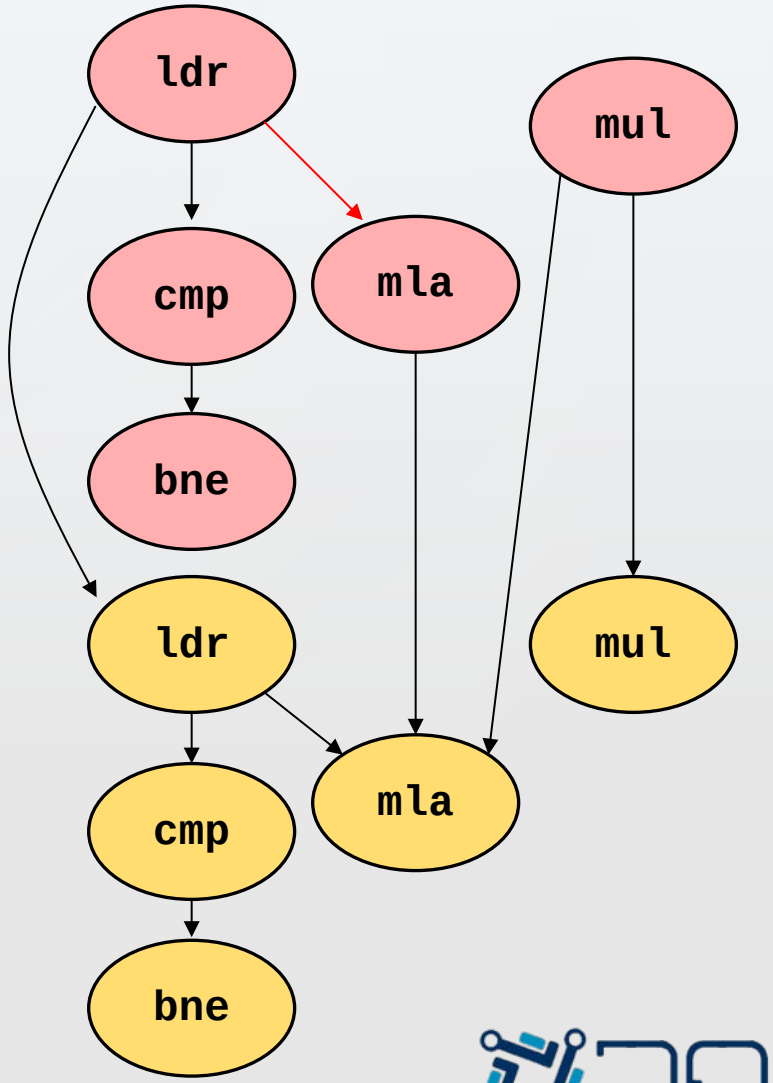

**ldr**, **mul**: 2 cycles
**cmp**, **bne**: 1 cycle
**mla**: 3 cycles



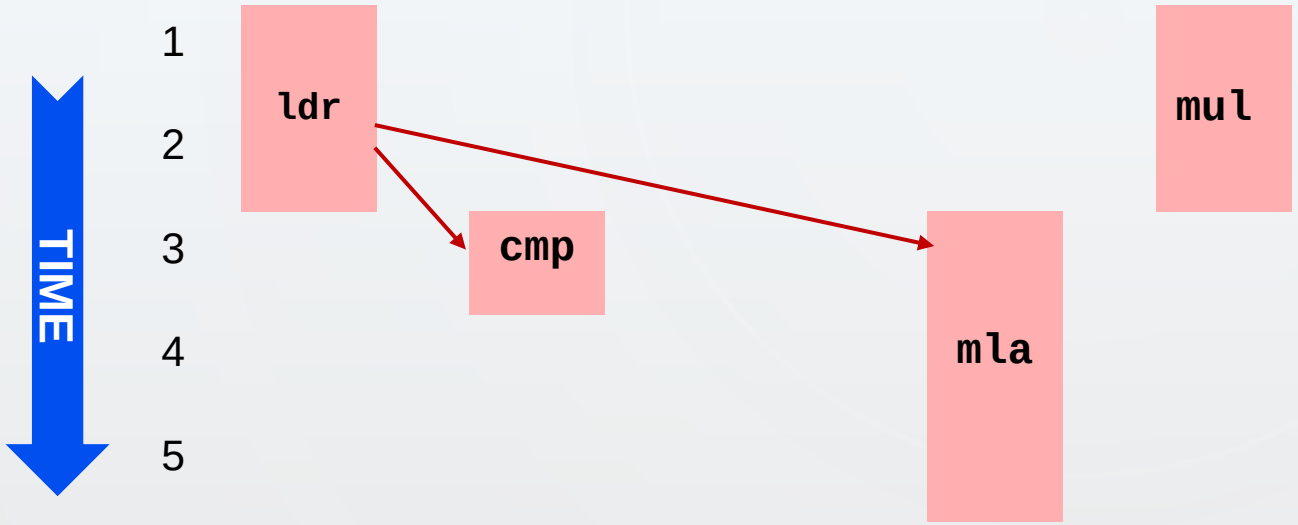
# DATAFLOW EXECUTION

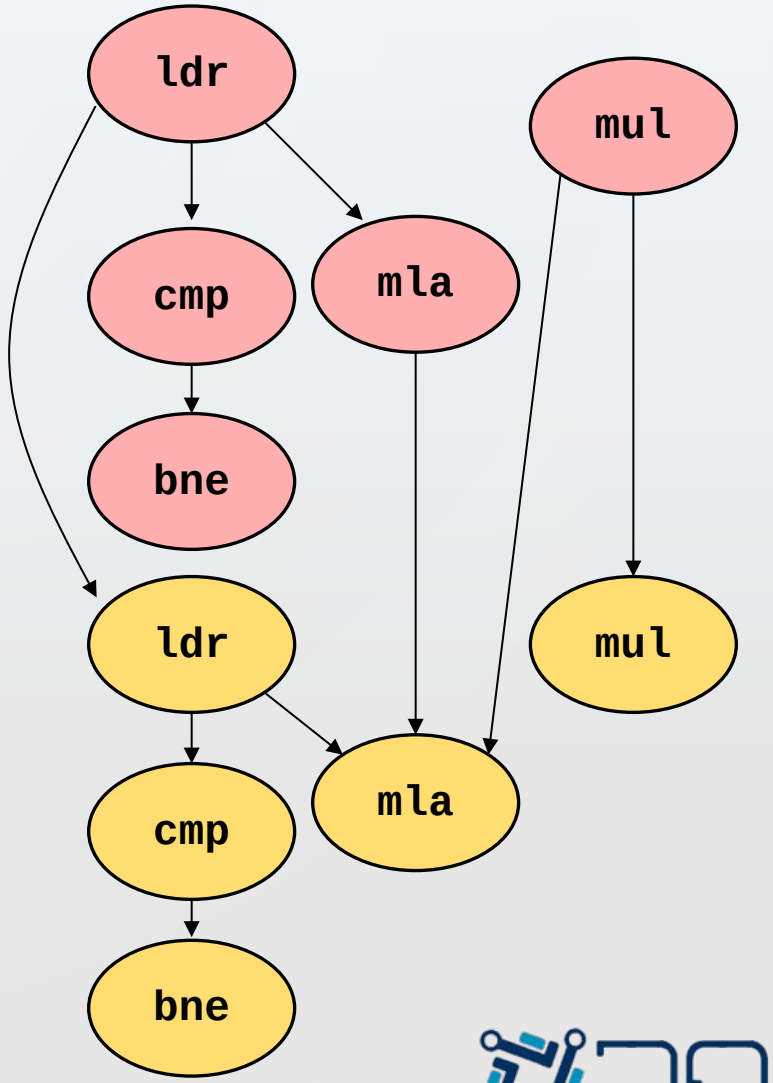



**ldr**, **mul**: 2 cycles
**cmp**, **bne**: 1 cycle
**mla**: 3 cycles



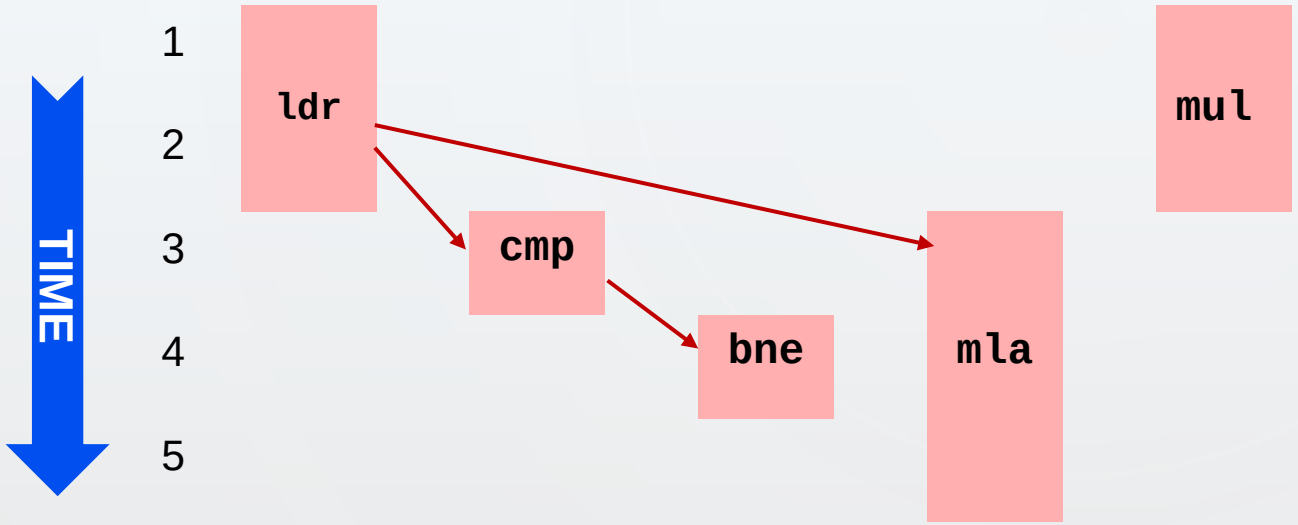
# DATAFLOW EXECUTION

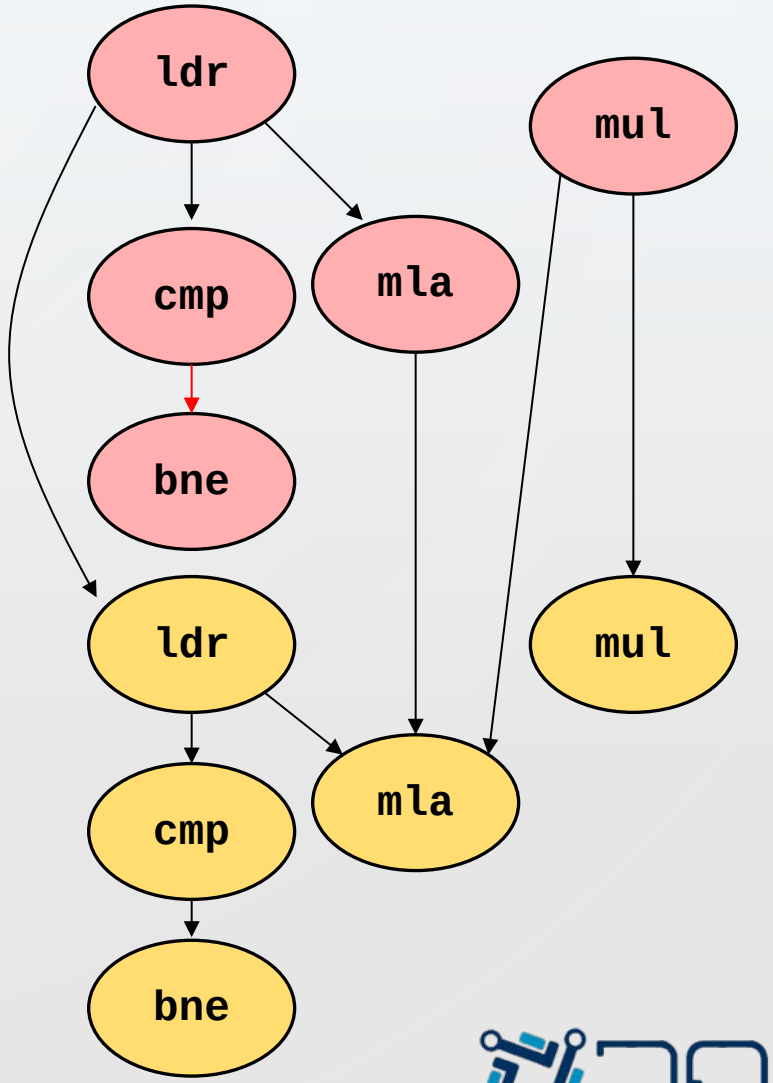



**ldr**, **mul**: 2 cycles
**cmp**, **bne**: 1 cycle
**mla**: 3 cycles

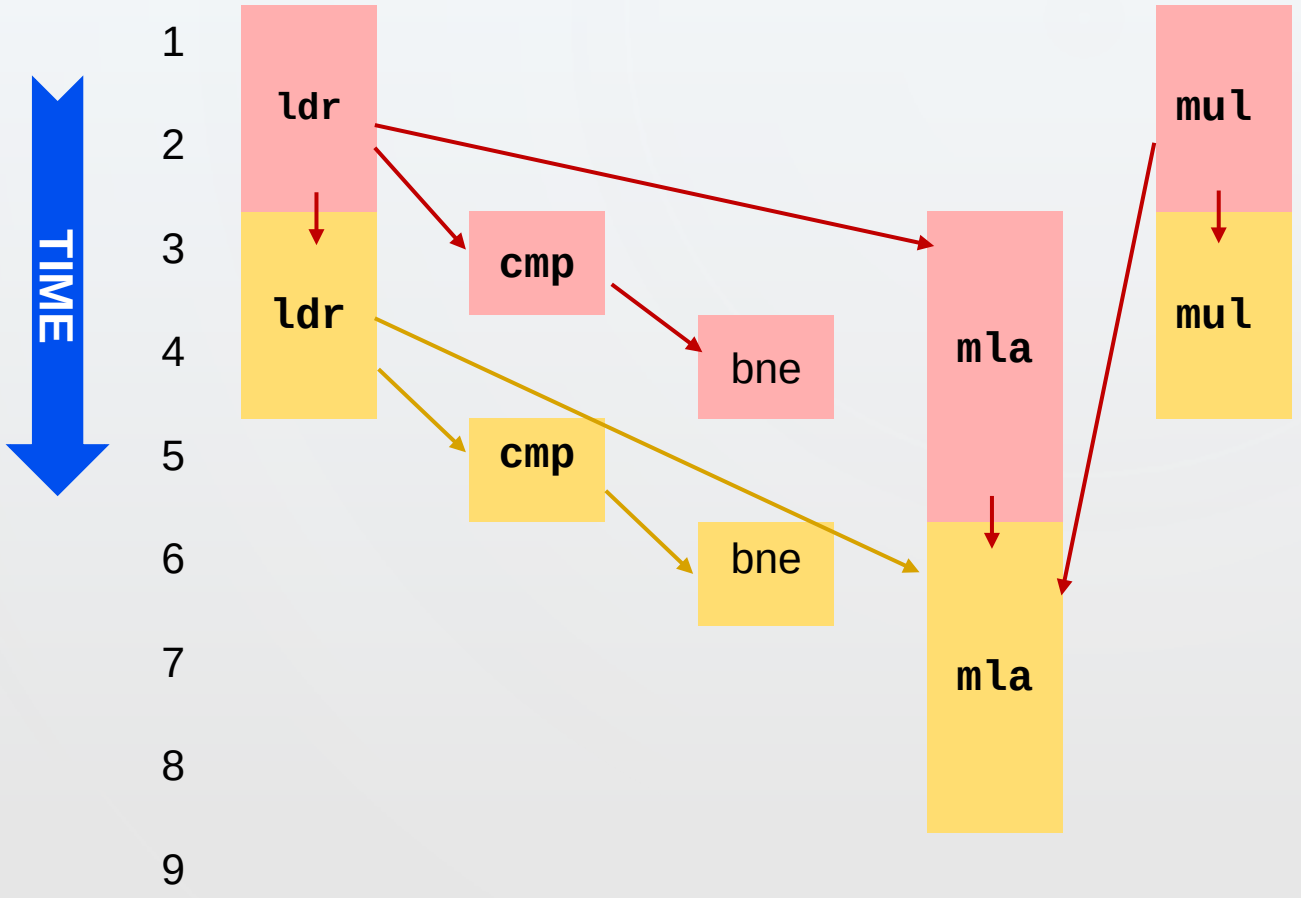
# DATAFLOW EXECUTION



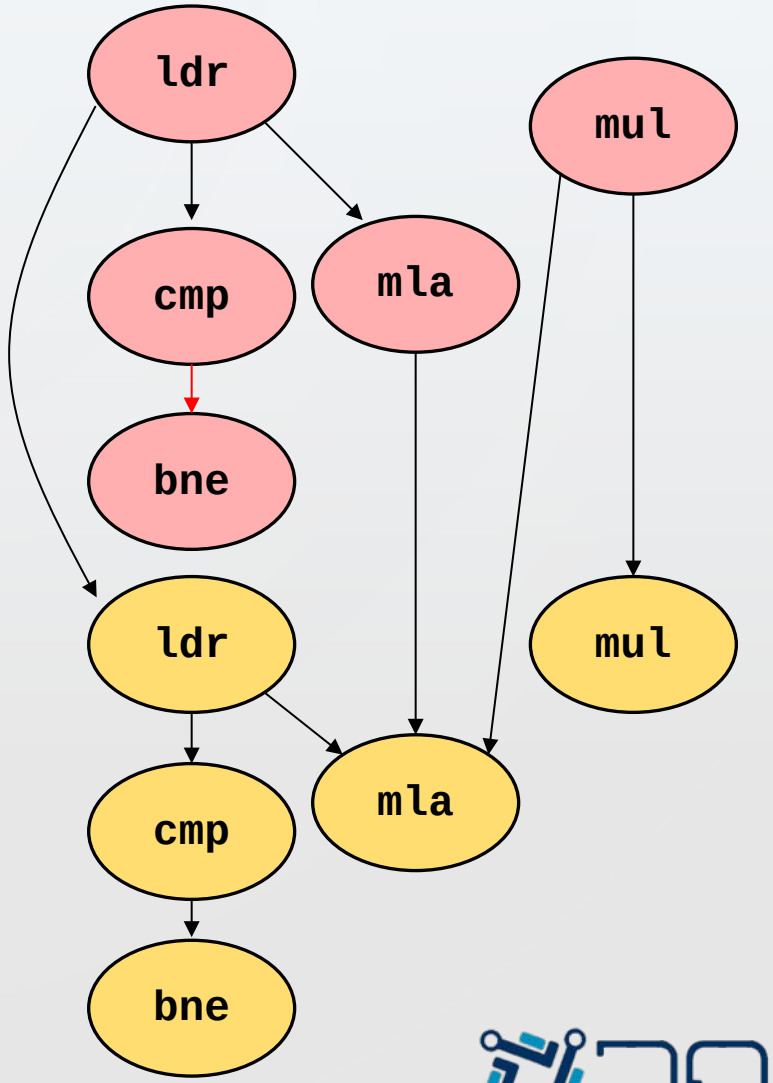




**ldr**, **mul**: 2 cycles
**cmp**, **bne**: 1 cycle
**mla**: 3 cycles

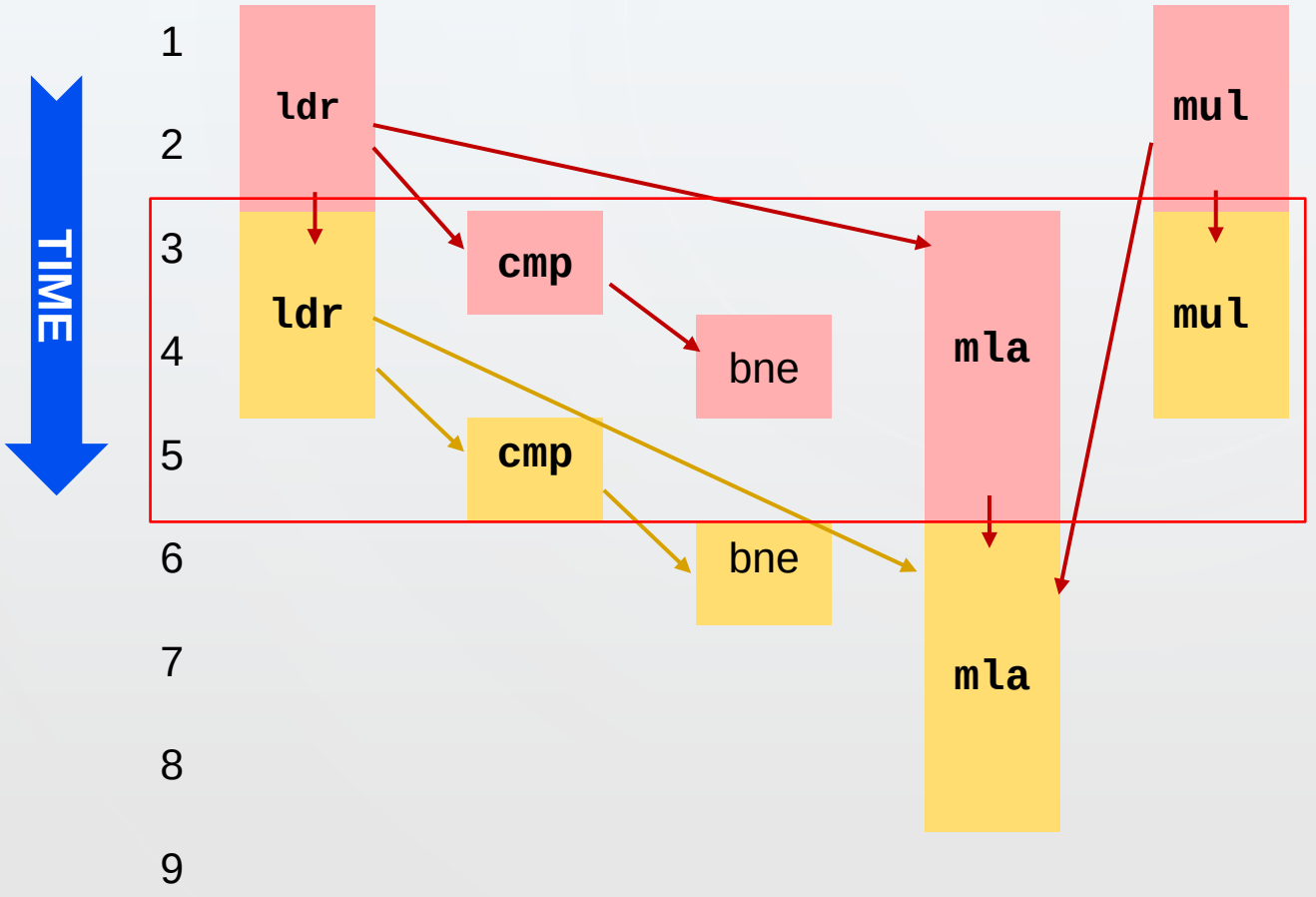
# DATAFLOW EXECUTION

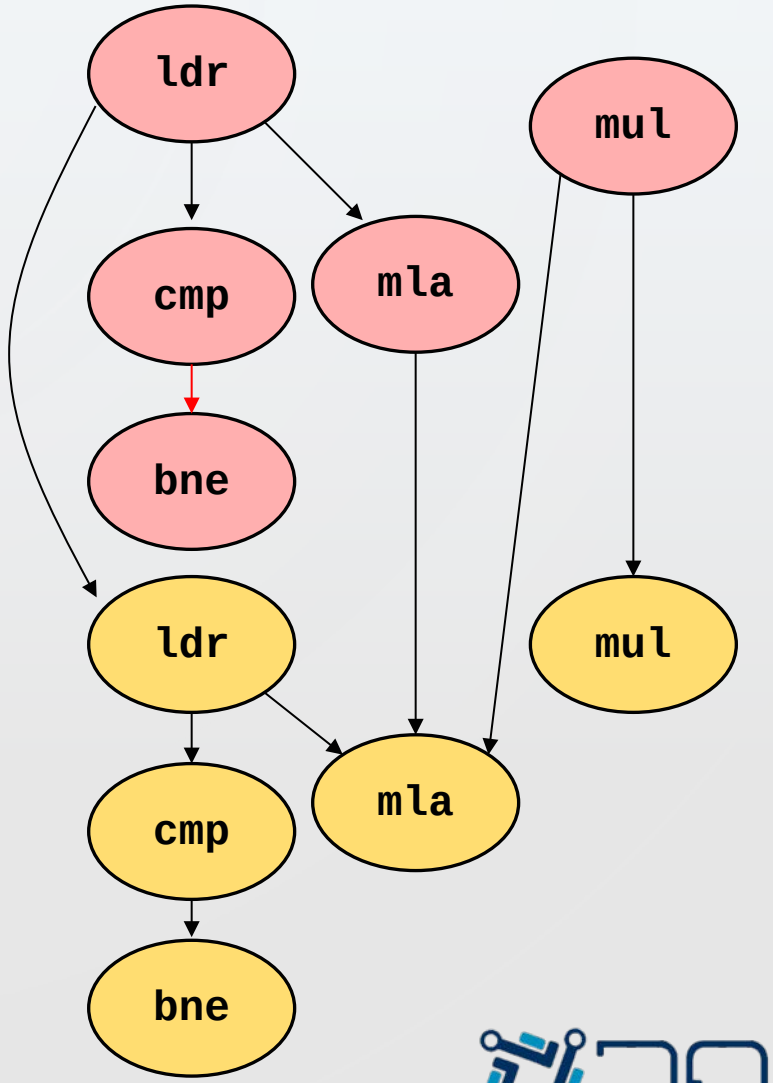



**ldr**, **mul**: 2 cycles
**cmp**, **bne**: 1 cycle
**mla**: 3 cycles



# DATAFLOW EXECUTION



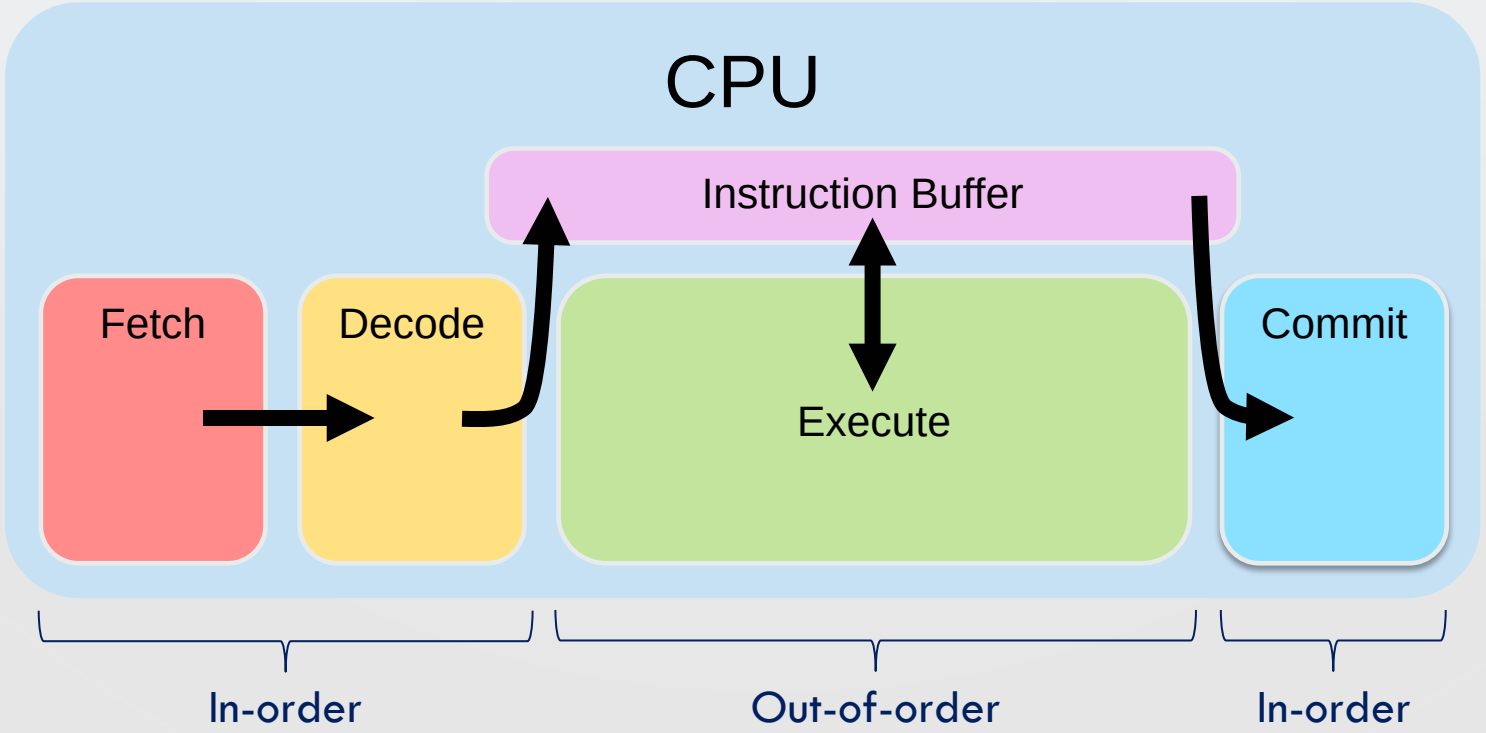

**ldr**, **mul**: 2 cycles
**cmp**, **bne**: 1 cycle
**mla**: 3 cycles

# DATAFLOW EXECUTION: PERFORMANCE

- Throughput of the dataflow execution
  - Execution of a loop iteration determined by the `mla` instruction
  - **Throughput**: 3 cycles / loop iteration
  - **IPC** (Instruction per cycle): 5 / 3 = 1.67
- Drawback: Dataflow breaks sequential execution
- **Out-of-Order** (OoO) execution
  - Restricted dataflow model, implemented in CPUs
  - Execute programs in dataflow order, but give the *illusion* of sequential execution

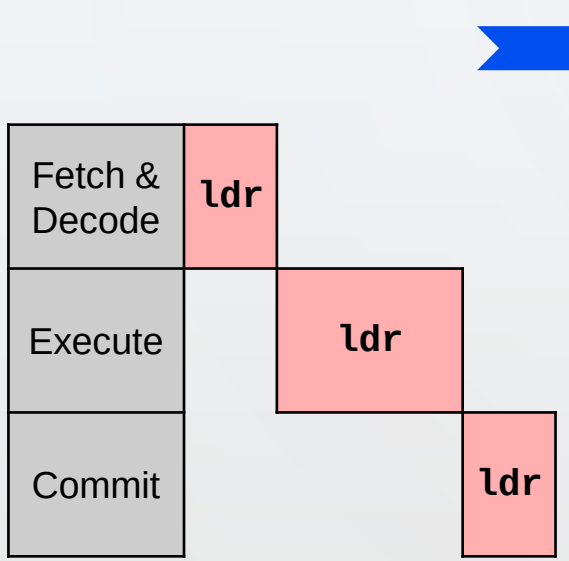
# OoO CPU MODEL

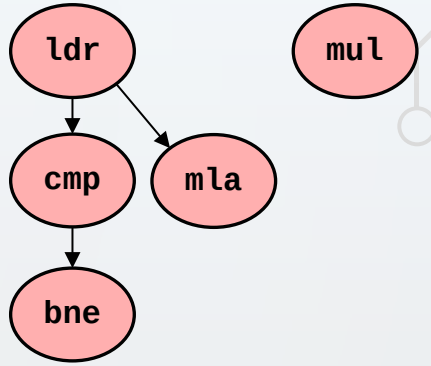
- Extension of the simple model to support OoO execution
  - OoO is hidden behind in-order **front-end** and **commit**
  - Instructions only enter & leave instruction buffer in program order



# OoO CPU MODEL: EXECUTION

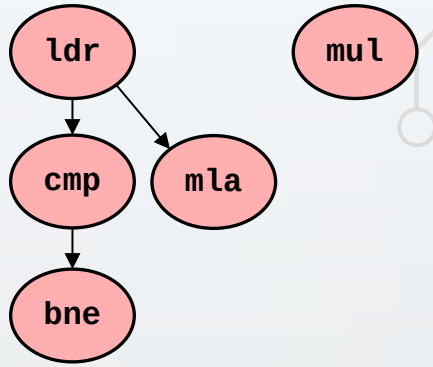
- Evaluating polynomial on the OoO CPU model

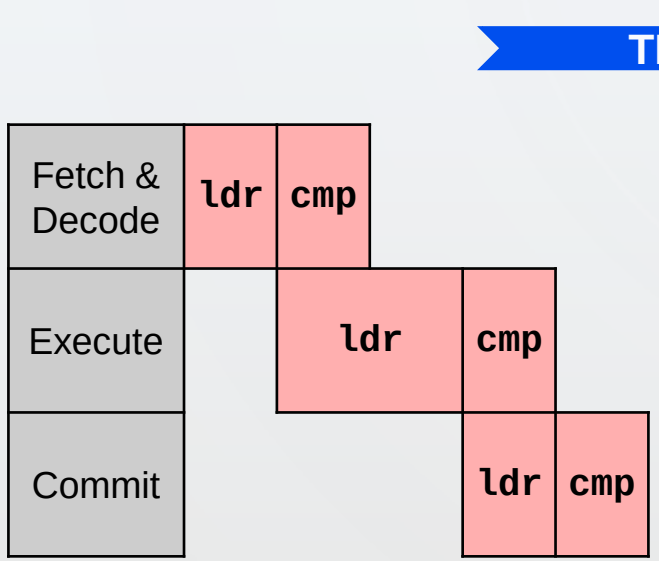


# OoO CPU MODEL: EXECUTION

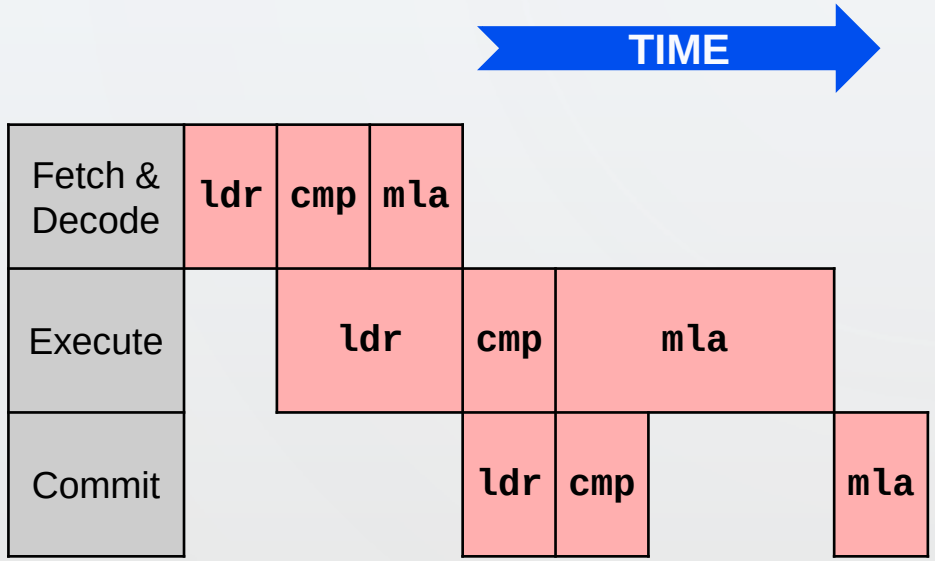
- Evaluating polynomial on the OoO CPU model

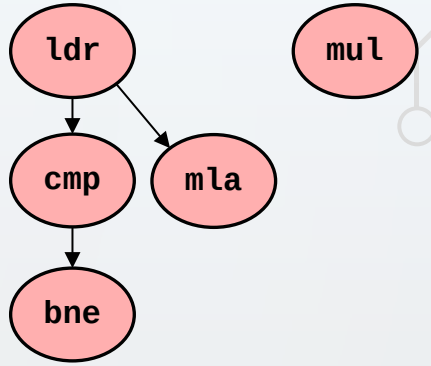


# OoO CPU MODEL: EXECUTION

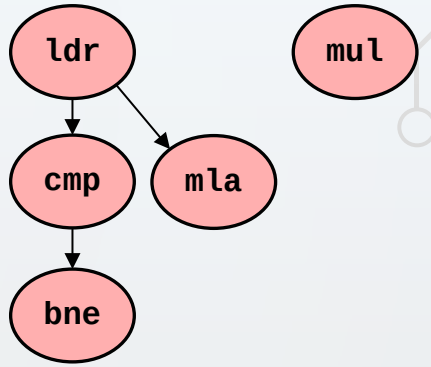
- Evaluating polynomial on the OoO CPU model

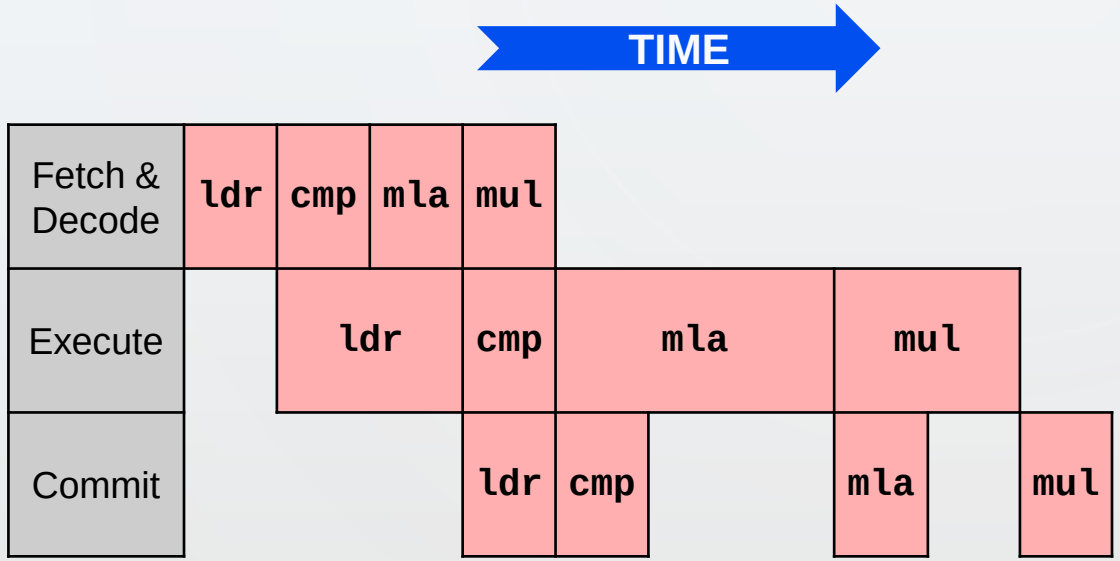


# OoO CPU MODEL: EXECUTION

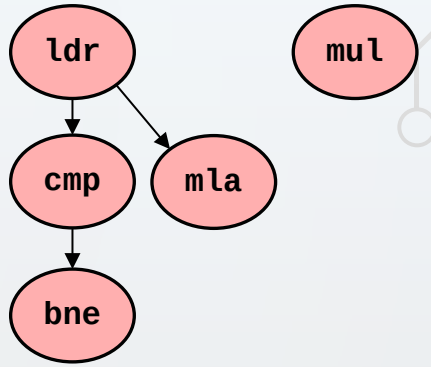
- Evaluating polynomial on the OoO CPU model

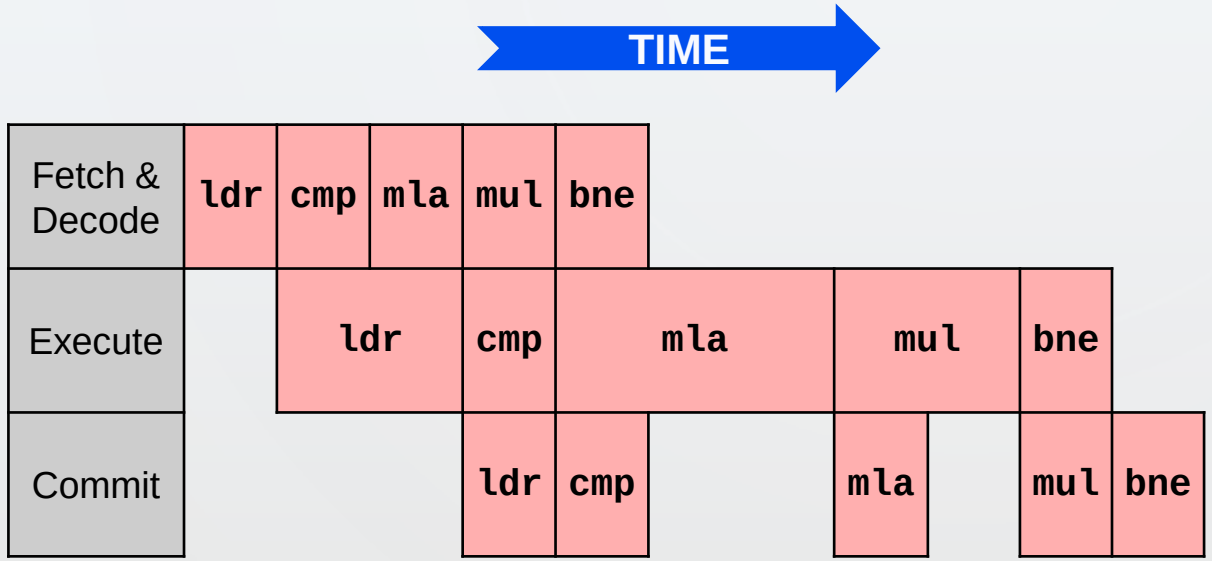


# OoO CPU MODEL: EXECUTION

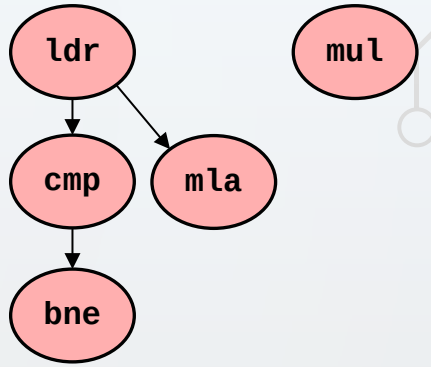
- Evaluating polynomial on the OoO CPU model

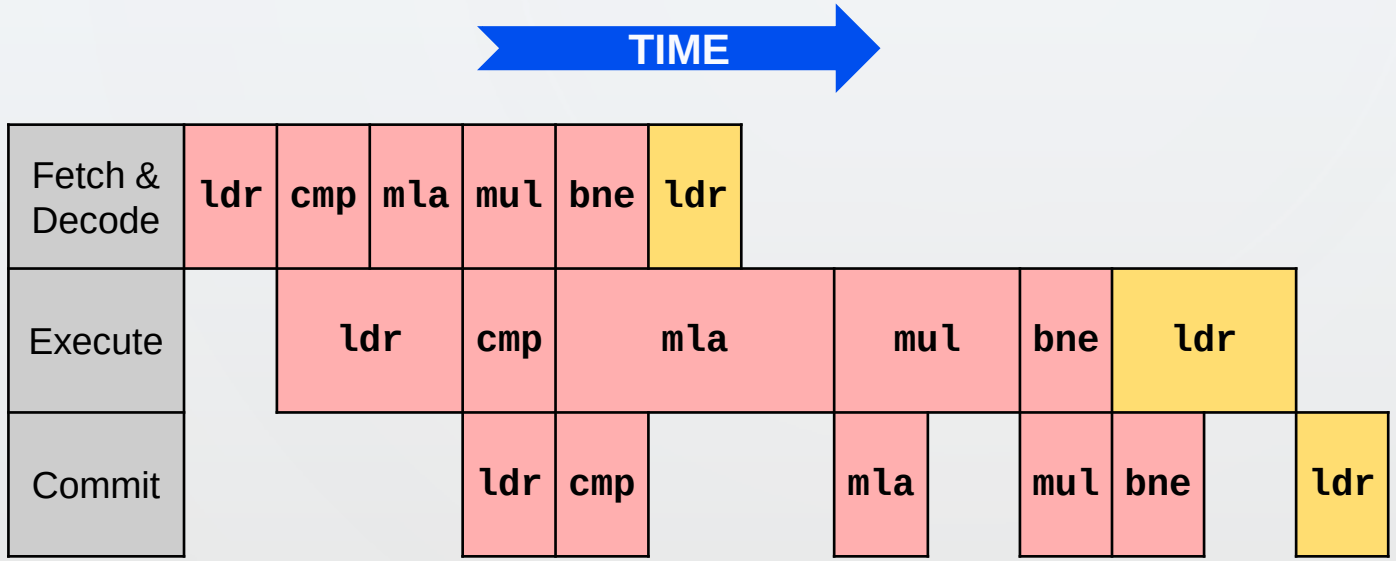


# OoO CPU MODEL: EXECUTION

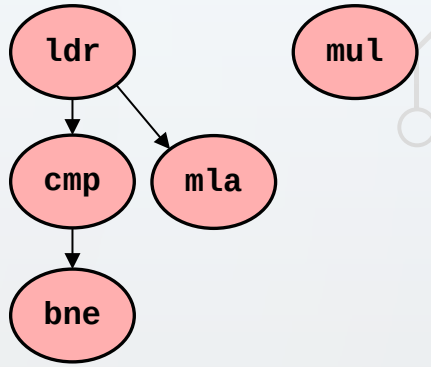
- Evaluating polynomial on the OoO CPU model

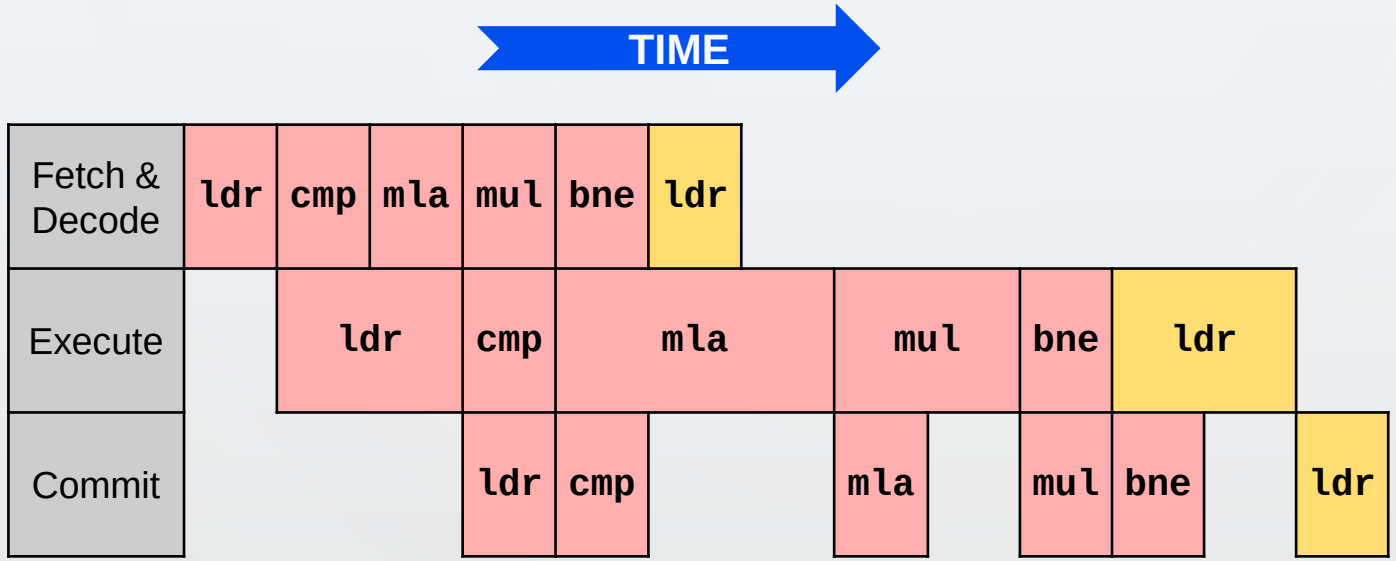


# OoO CPU MODEL: EXECUTION

- Evaluating polynomial on the OoO CPU model

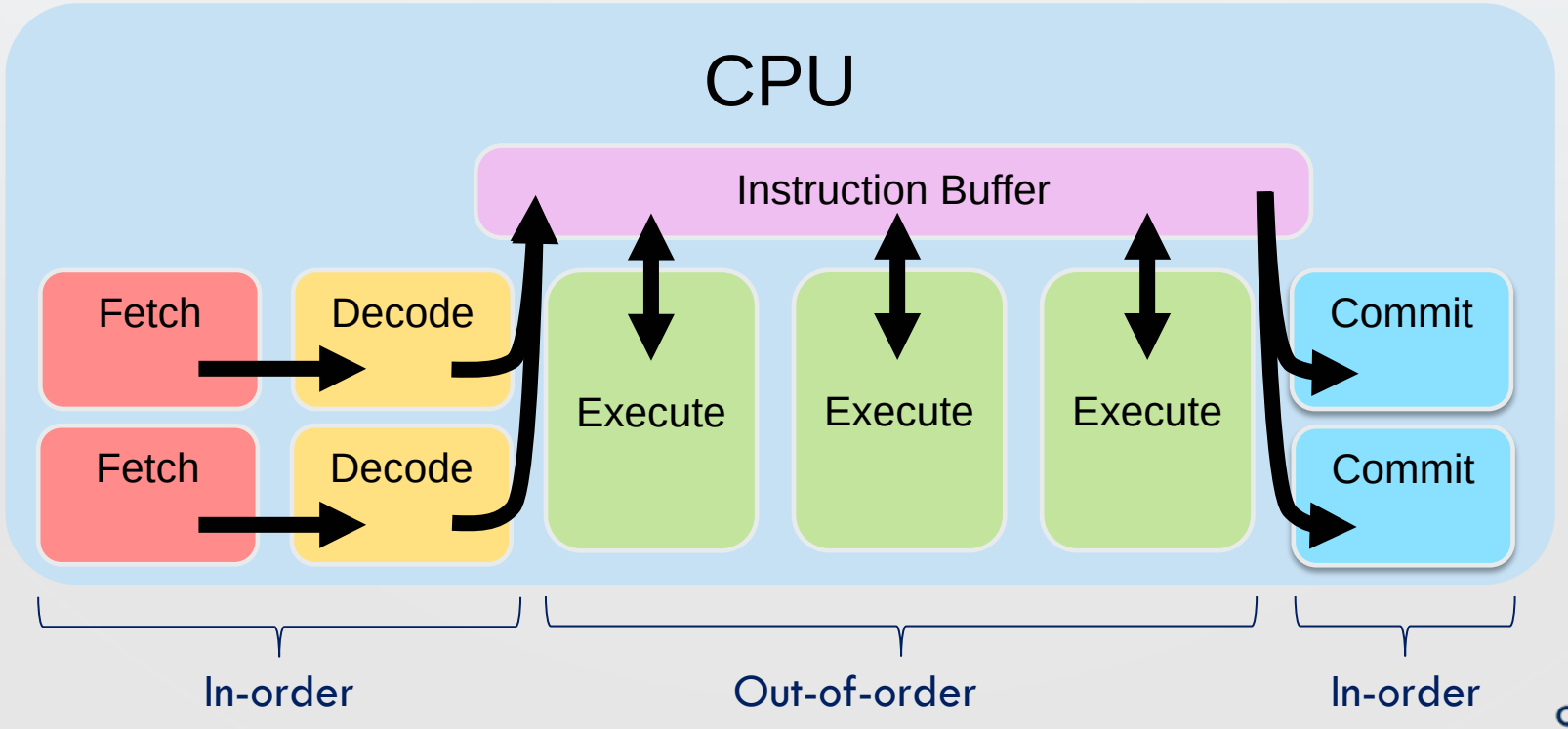



- This is not really OoO! All instructions are executed in order
- What is wrong?
- We have a small number of instructions in the buffer and only one execution unit

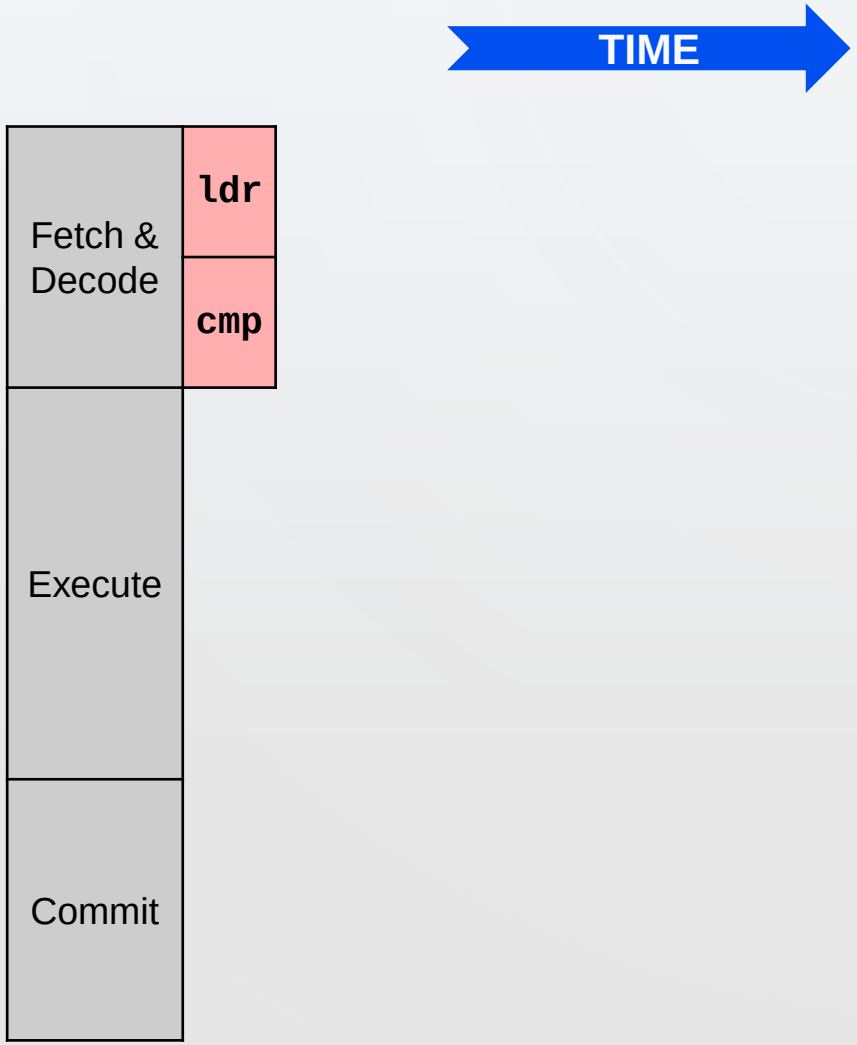
# SUPERSCALAR OoO CPU MODEL

- Extension of the OoO model to add more instructions in the buffer and more execution units
  - Exploit **ILP** (Instruction Level Parallelism)



# SUPERSCALAR OoO CPU MODEL: EXECUTION

- Evaluating polynomial on the superscalar OoO CPU model

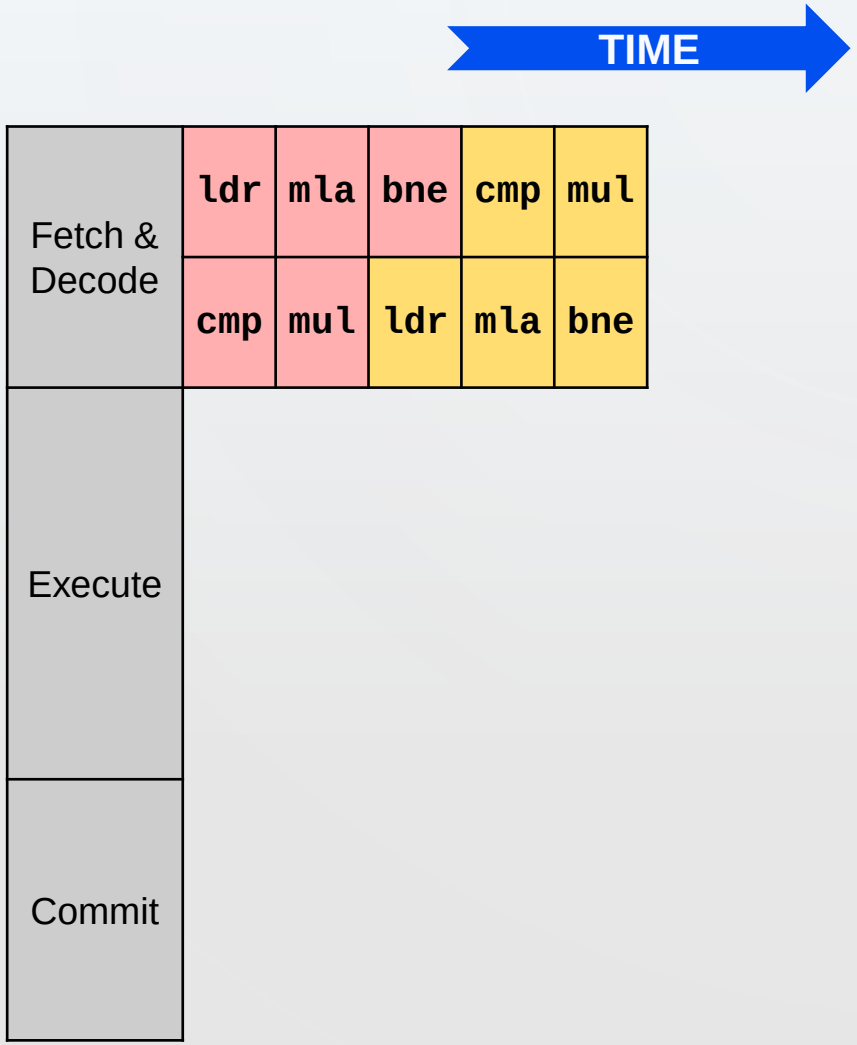



```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3

ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```

# SUPERSCALAR OoO CPU MODEL: EXECUTION

- Evaluating polynomial on the superscalar OoO CPU model

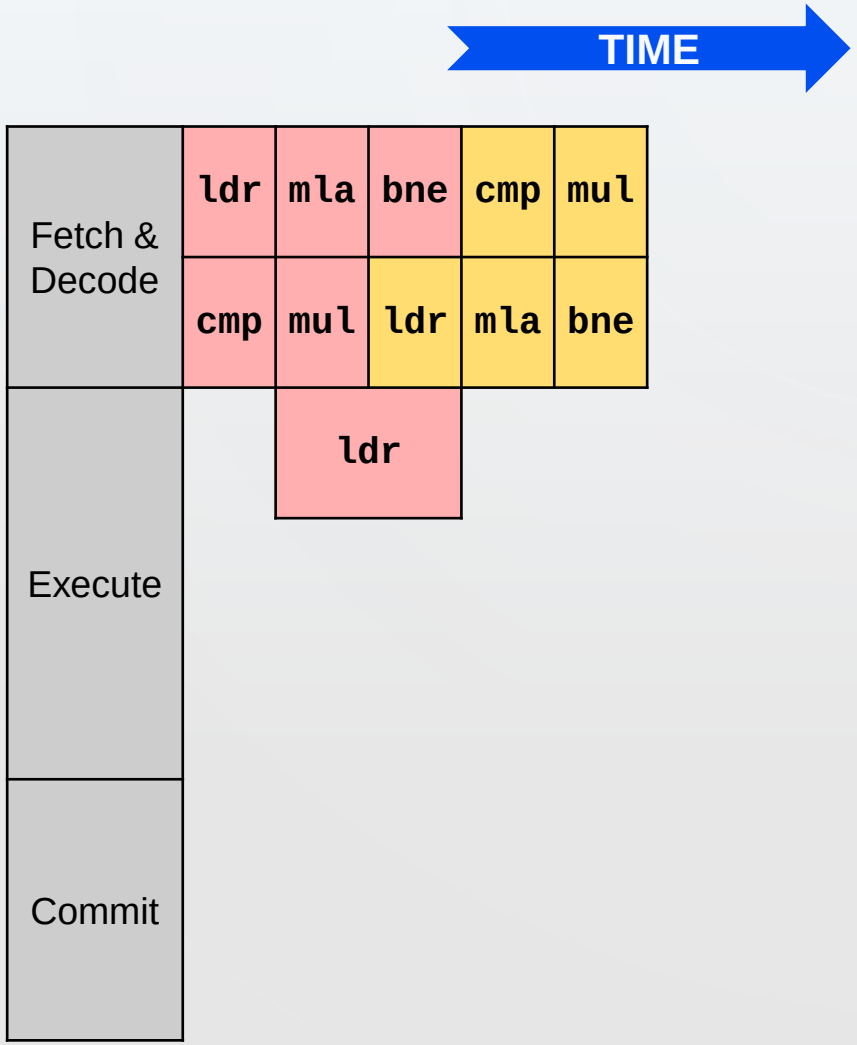



```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3

ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```

# SUPERSCALAR OoO CPU MODEL: EXECUTION

- Evaluating polynomial on the superscalar OoO CPU model

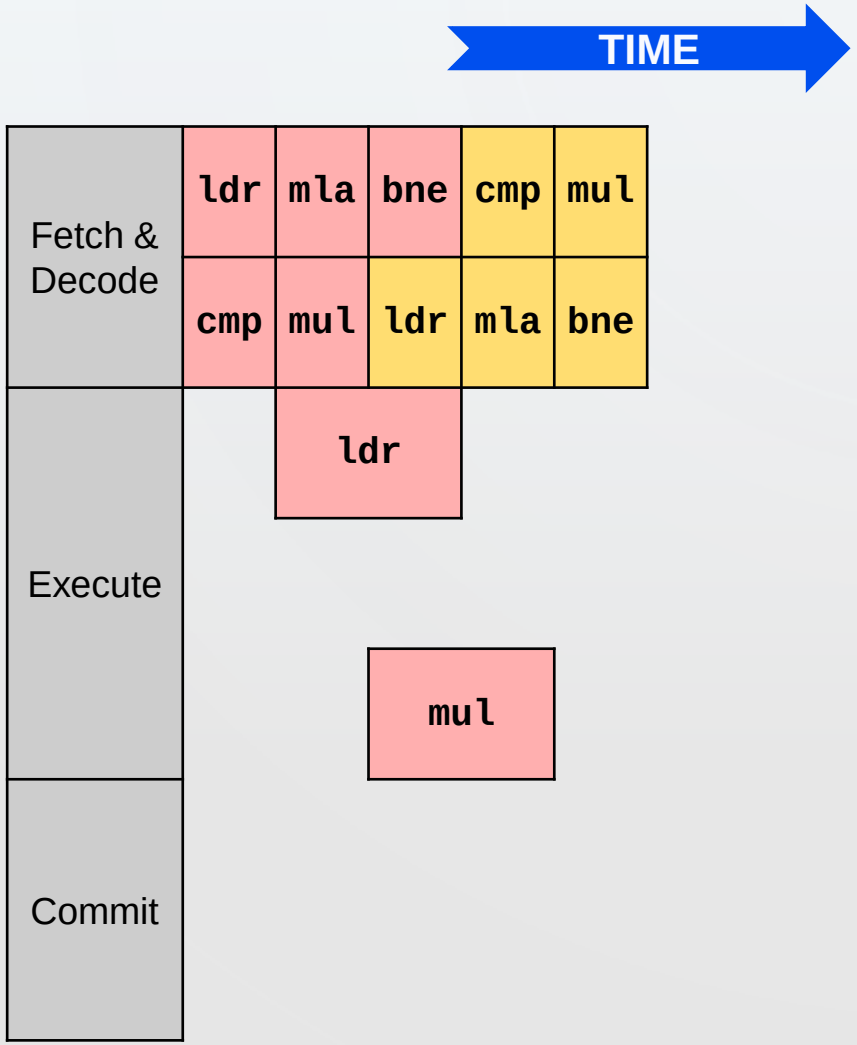



```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3

ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```

# SUPERSCALAR OoO CPU MODEL: EXECUTION

- Evaluating polynomial on the superscalar OoO CPU model

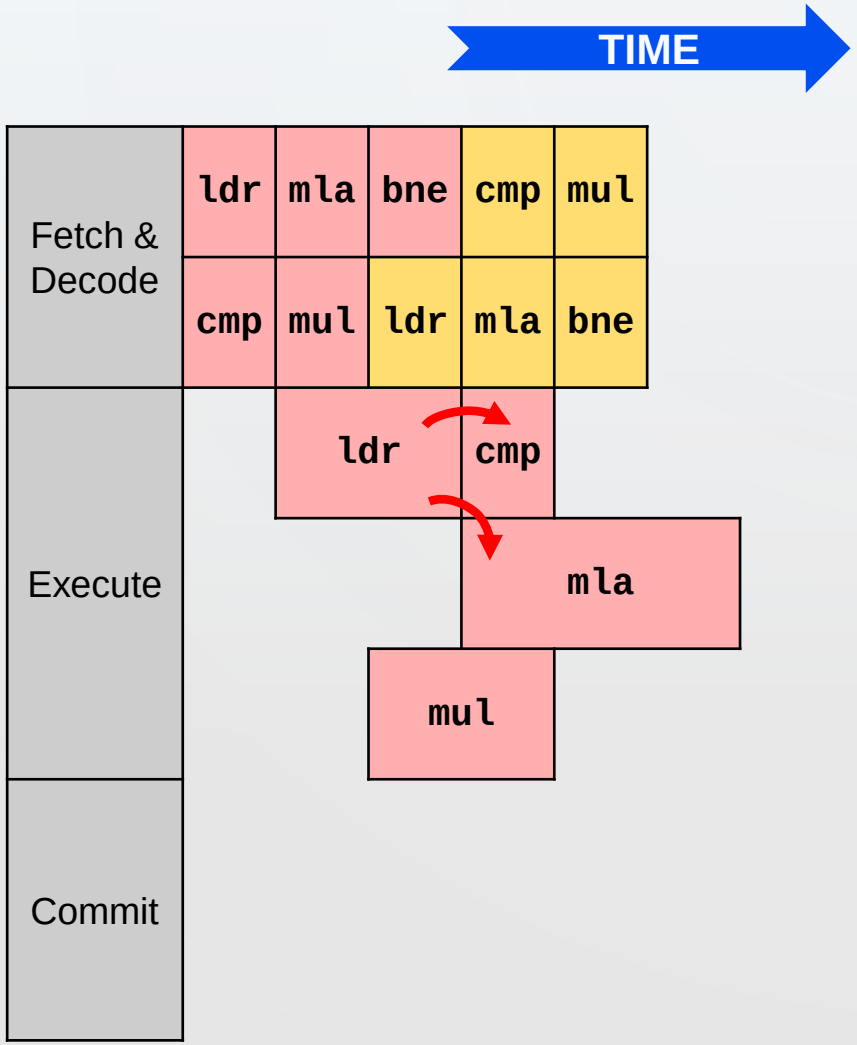



```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3

ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```

# SUPERSCALAR OoO CPU MODEL: EXECUTION

- Evaluating polynomial on the superscalar OoO CPU model

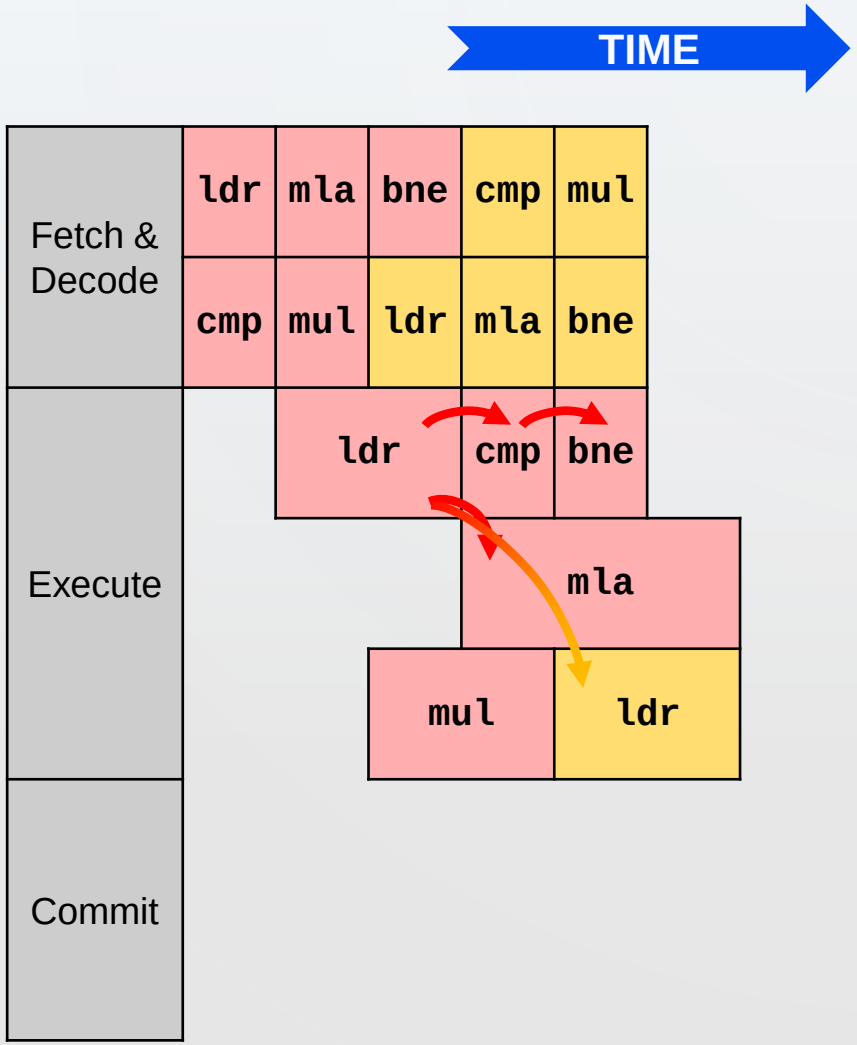



```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3

ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```

# SUPERSCALAR OoO CPU MODEL: EXECUTION

- Evaluating polynomial on the superscalar OoO CPU model

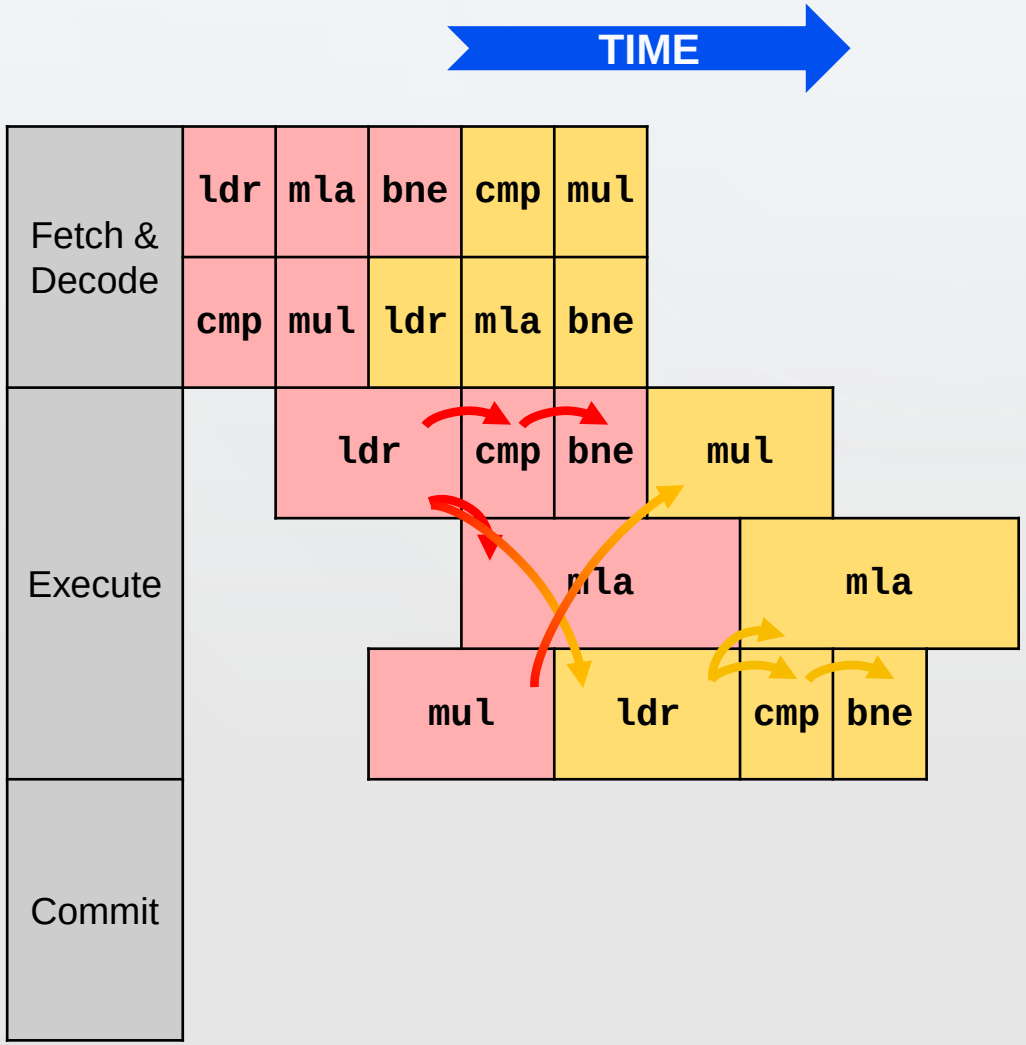


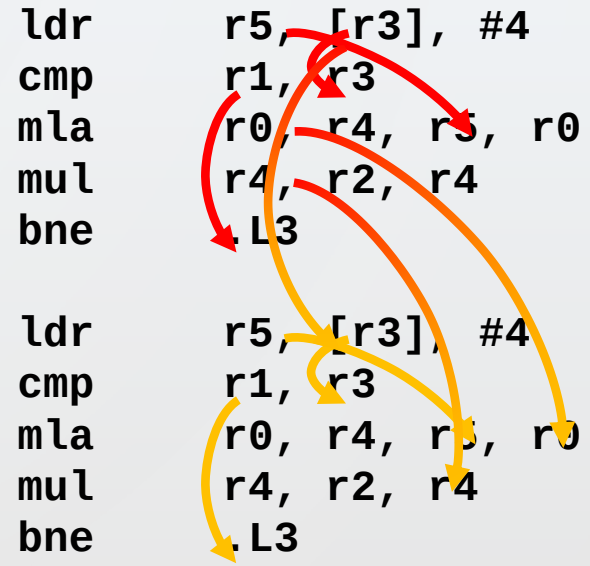

```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3

ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```

# SUPERSCALAR OoO CPU MODEL: EXECUTION

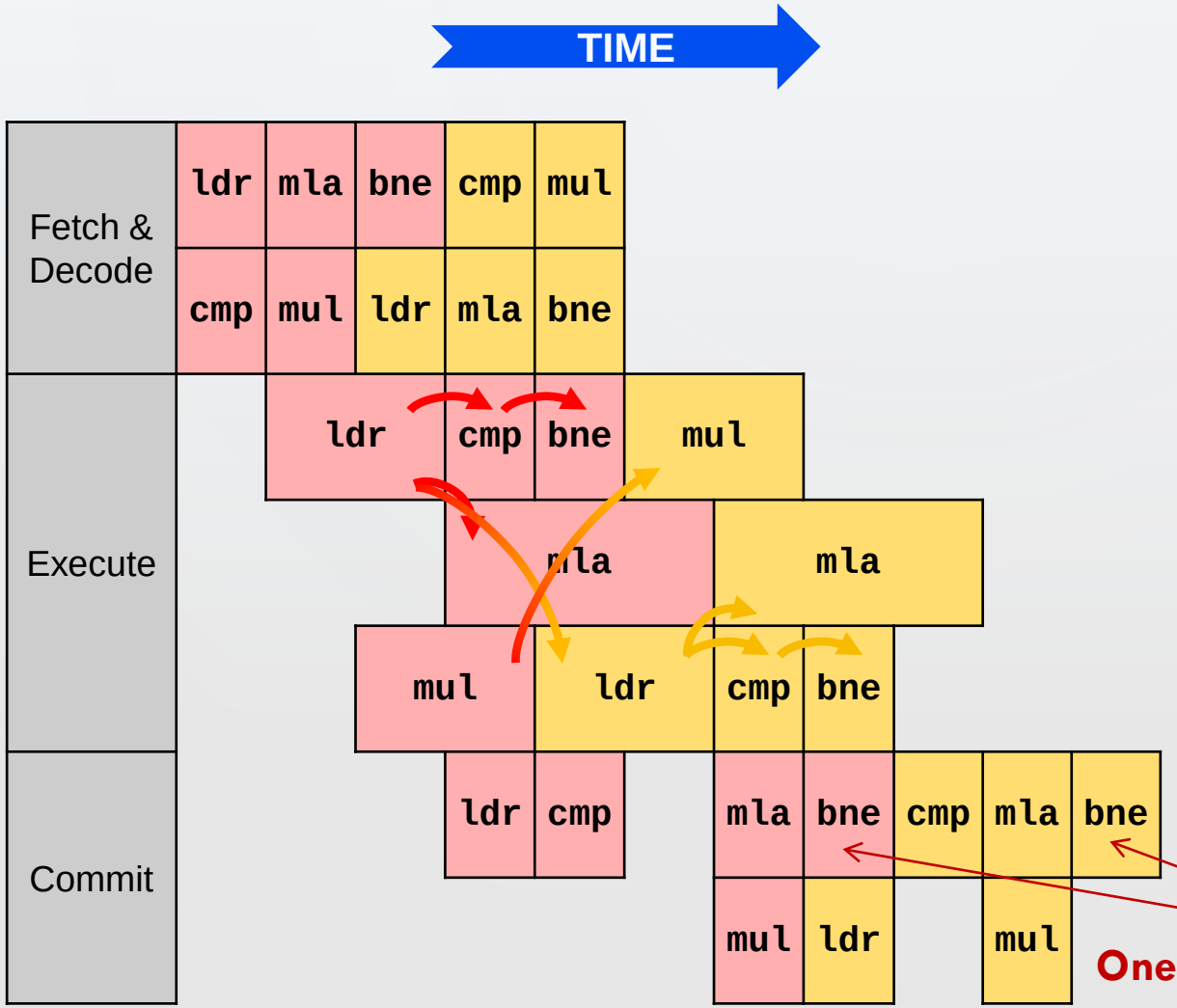
- Evaluating polynomial on the superscalar OoO CPU model



# SUPERSCALAR OoO CPU MODEL: EXECUTION

- Evaluating polynomial on the superscalar OoO CPU model



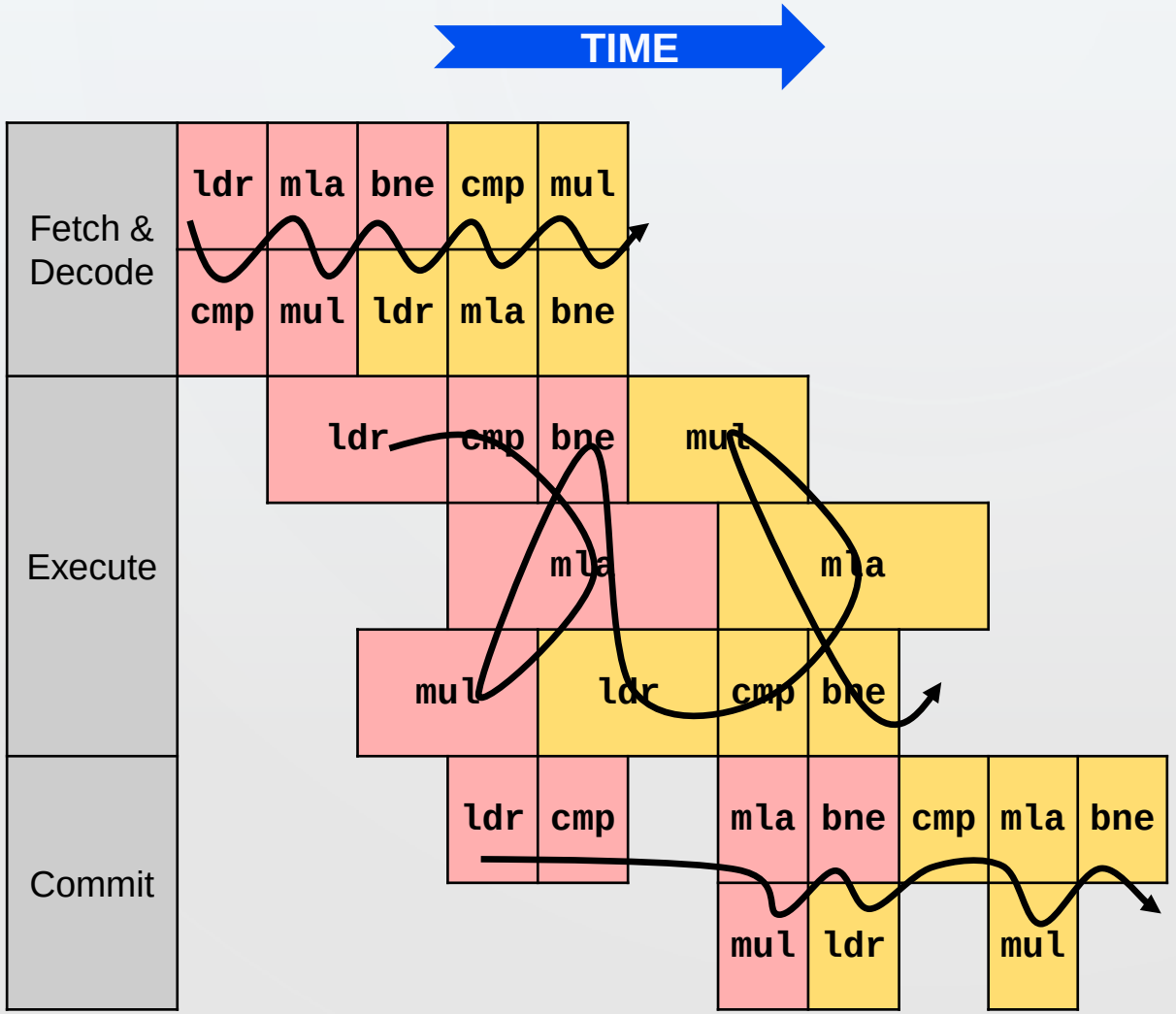

```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3

ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```

One loop iteration / 3 cycles!

# SUPERSCALAR OoO CPU MODEL: EXECUTION

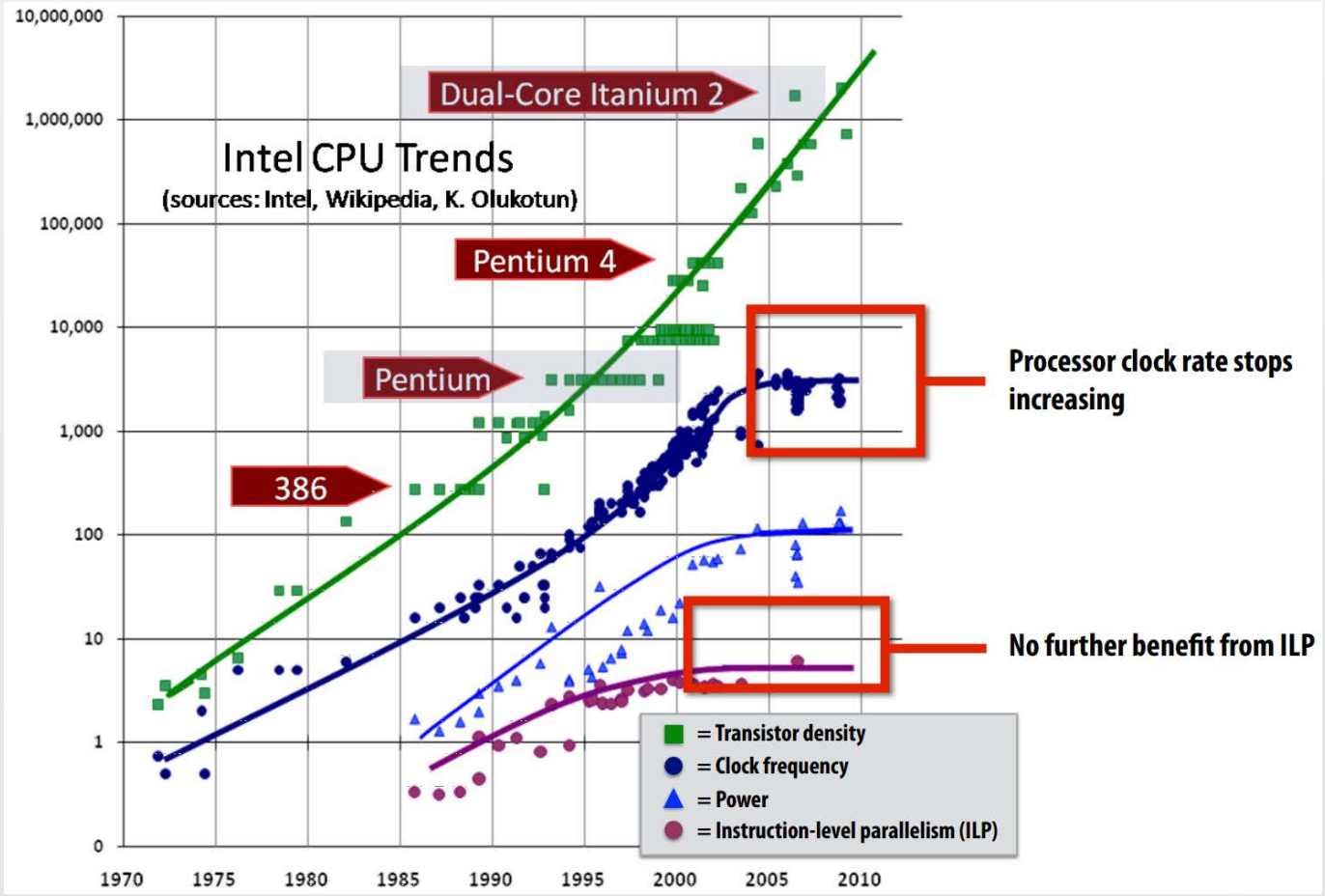
- Evaluating polynomial on the superscalar OoO CPU model




- Observe:
  - Front-end and commit are in-order
  - Execute are out-of-order

# SUPERSCALAR OoO CPU MODEL: ILP

- Pipelining and ILP are the basis of the huge CPU performance improvements from mid 1980s to late 1990s
- But, pipelining and ILP are hard to scale!



# LIMITATIONS OF PIPELINING AND ILP

- 4-wide superscalar × 20-stage pipeline = 80 instruct. in flight
- High-performance OoO buffers *hundreds* of instructions
- **Programs have limited ILP**
  - Even with perfect scheduling, >8-wide issue does not help
- Pipelines cannot go more than 15 or so stages deep
  - Branch misprediction penalty grows
  - Frequency limited by power
- Dynamic scheduling overheads are great
- Out-of-order scheduling is expensive

# LIMITATIONS OF ILP: SOLUTIONS

- ILP works great! … But is complex & hard to scale
- Besides, all instruction scheduling work is done by the CPU with the help of the compiler
  - **The programmer does not worry about exploiting ILP**

Solutions to limitations of ILP:

## SIMD (Vector) Processing

- Amortizing cost/complexity of managing an instruction stream across many ALUs
  - It allows to decrease both instruction scheduling and decoding logic

## Multithreaded Processing

- Add hardware support to manage multiple threads of execution
  - It allows to mix instructions from different streams, probably independent

# LIMITATIONS OF ILP: SOLUTIONS

- ILP works great! … But is complex & hard to scale
- Besides, all instruction scheduling work is done by the CPU with the help of the compiler
  - **The programmer does not worry about exploiting ILP**

Solutions to limitations of ILP:

**SIMD (Vector) Processing**

- Amortizing cost/complexity of managing an instruction stream across many ALUs
  - It allows to decrease both instruction scheduling and decoding logic

**Multithreaded Processing**

- Add hardware support to manage multiple threads of execution
  - It allows to mix instructions from different streams, probably independent

# PARALELISMO DE DATOS - VECTORIAL (SIMD)

## DATA-LEVEL PARALLELISM IN VECTOR, SIMD, AND GPU ARCHITECTURES

TEMA 5: EXPLOTACIÓN DE PARALELISMO

# PARALLELISM EVERYWHERE

*Software*

- Parallel Requests
  - Assigned to computer
  - e.g. search “Garcia”
- Parallel Threads
  - Assigned to core
  - e.g. lookup, ads
- Parallel Instructions
  - > 1 instruction @ one time
  - e.g. 5 pipelined instructions
- Parallel Data — SIMD
  - > 1 data item @ one time
  - e.g. add of 4 pairs of words
- Hardware descriptions
  - All gates functioning in parallel at same time

*Leverage Parallelism & Achieve High Performance*

*Hardware*

Warehouse Scale Computer

Smart Phone

# AN ALTERNATE CLASSIFICATION

- Instructions and data stream

| | | Data Streams | |
|---|---|---|---|
| | | Single | Multiple |
| **Instruction Streams** | Single | **SISD**:<br>Intel Pentium 4 | **SIMD**: SSE, AVX instructions of x86<br>NEON (ARM) |
| | Multiple | **MISD**:<br>No examples today | **MIMD**:<br>Intel Xeon e5345 |

# SIMD (VECTOR) PARALLELISM

- SIMD architectures can exploit significant data-level parallelism for:
  - Matrix-oriented scientific computing
  - Media-oriented image and sound processors
- SIMD is more energy efficient than MIMD
  - Only needs to fetch one instruction per data operation
  - Makes SIMD attractive for personal mobile devices
- SIMD allows programmer to continue to think sequentially

# SIMD (VECTOR) PARALLELISM

- Vector architectures
- SIMD extensions
- Graphics Processor Units (GPUs)

# SIMD (VECTOR) PROCESSING

- SIMD processing: Single Instruction, Multiple Data
  - A SIMD instruction specifies a single operation on a set of values (vectors)
- Example program
  - Assumes 4-value vectors

```
for (int i = 0; i <= N; i++) {
    C[i] = A[i] + B[i];
}
```

```
.L1:
  ld   r4, [r1]
  ld   r5, [r2]
  add  r4, r4, r5
  st   [r3], r4
  add  r0, r0, #4
  cmp  r0, #N
  ble  .L1
```

```
r0: #0
r1: &A[i]
r2: &B[i]
r3: &C[i]
```

Scalar code

```
for (int i = 0; i <= N; i+=4) {
    C[i:i+3] = A[i:i+3] + B[i:i+3];
}
```

```
.L1:
  vld   vr4, [r1]
  vld   vr5, [r2]
  vadd  vr4, vr4, vr5
  vst   [r3], vr4
  add   r0, r0, #16
  cmp   r0, #N
  ble   .L1
```

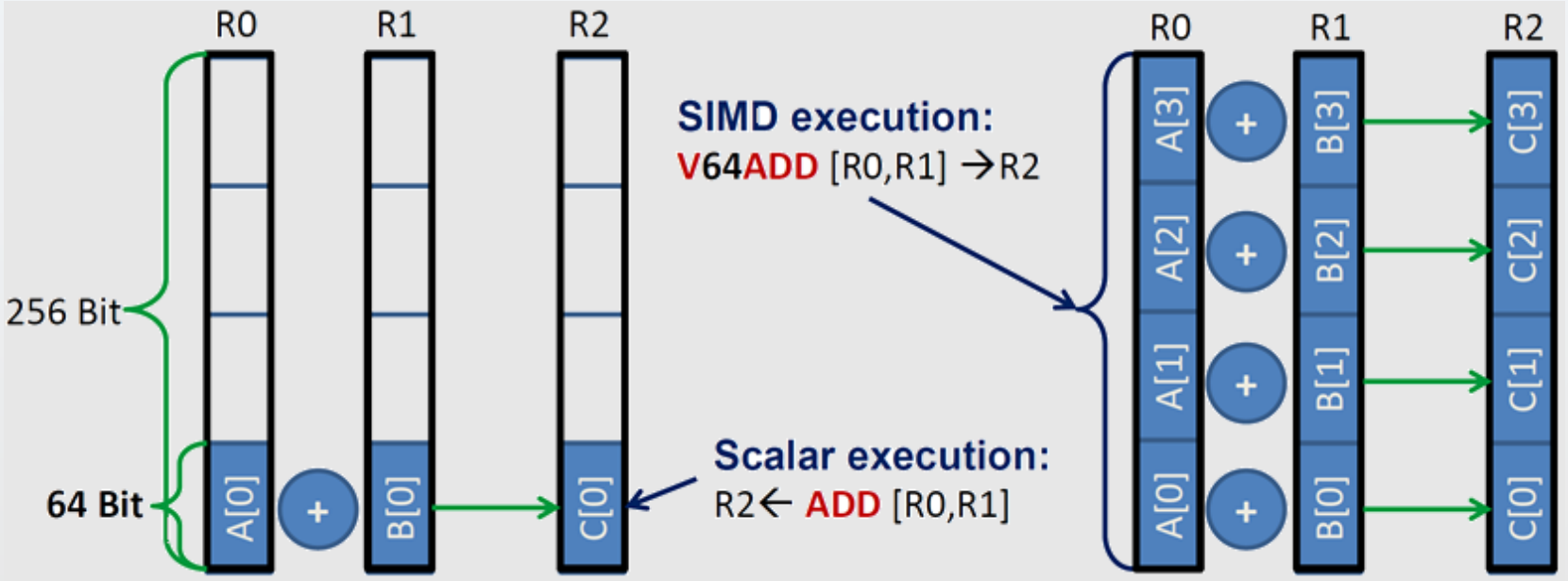
SIMD code

SIMD instruction

SIMD register (holds 4 values)

# SIMD (VECTOR) EXTENSIONS

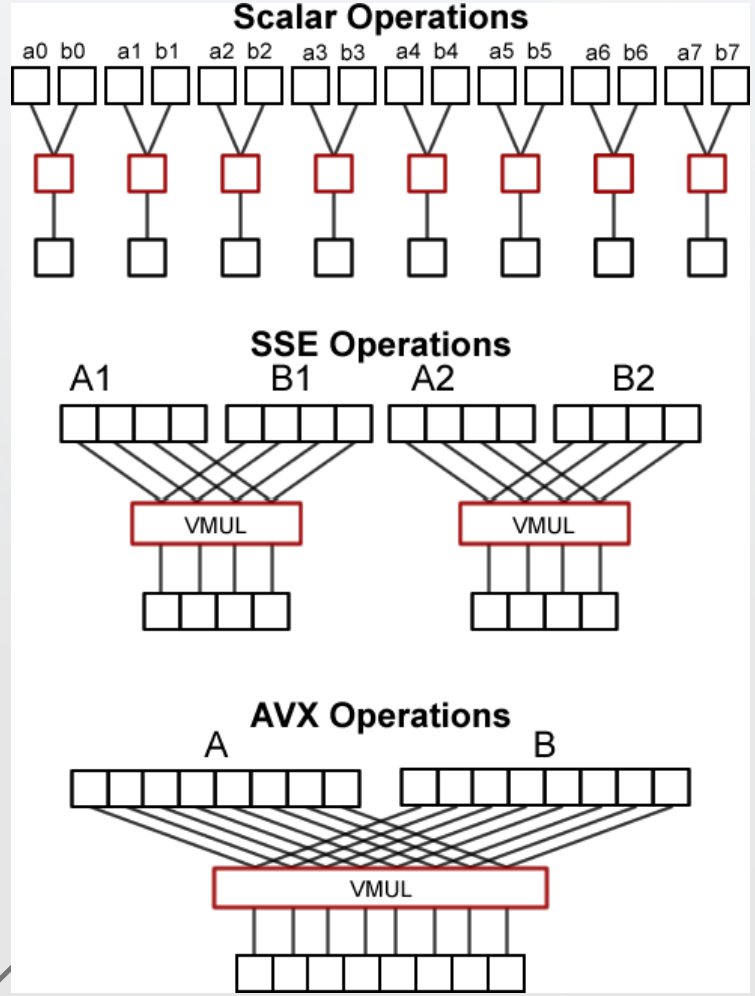
- Example
  - Adding two registers holding double precision floating point operands

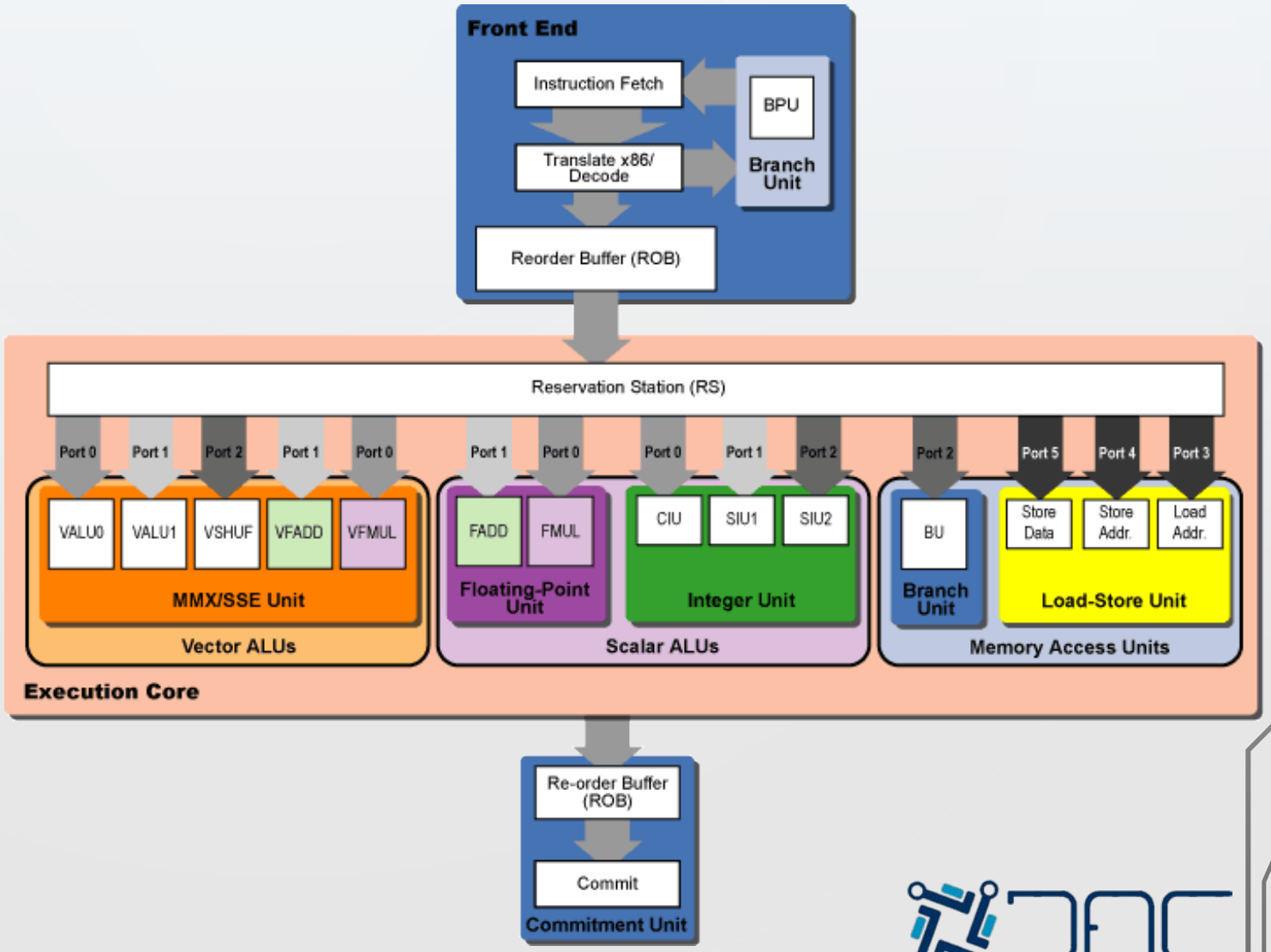



    - **Scalar execution:** Operands are single values
    - **SIMD execution:** Operands are vectors of multiple values (four in the example)
  - A single SIMD instruction is equivalent to four scalar instructions (in this example)

# SCALAR/VECTOR ARCHITECTURES

- Vector / SIMD



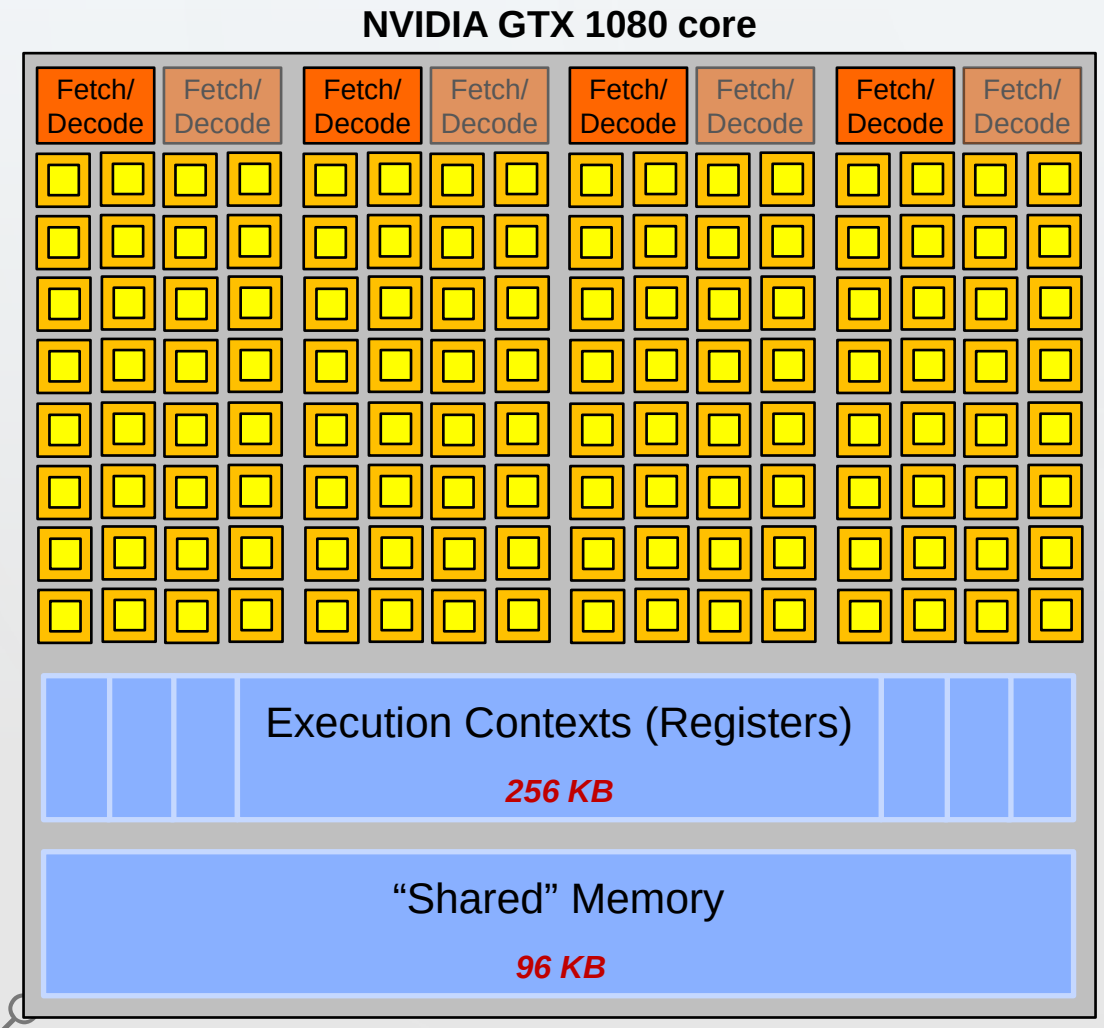
# SIMD PROCESSING TECHNOLOGIES: CPUs

- SSE (Streaming SIMD Extensions)  Architecture Intel x86
  - SSE (1999): 32 x 128-bit vector registers (4 x 32-bit)
  - SSE2 (2001): possibilitiy of 2 x 64-bit, 4 x 32-bit, 8 x 16-bit, 16 x 8-bit
  - SSE3, SSSE3, SSE4: new instructions
- AVX (Advanced Vector Extensions) Architecture Intel x86
  - AVX (2008): 16 x 256-bit vector registers
  - AVX2 (Haswell, 2013): new instructions
  - AVX-512 (Knights Landing, 2015): 32 x 512-bit vector registers
- ARM NEON, PowerPC AltiVec…
- Vector instructions generated by the compiler
- **Explicit SIMD**
  - Parallelism may be explicitly given by programmer using intrinsics or parallel language semantics
  - SIMD parallelization performed at compile time using vector instructions

# SIMD PROCESSING TECHNOLOGIES: GPUs

- **Implicit SIMD**
  - Compiler generates a scalar binary
  - N instances of the program are **always run** together (**synchronized**)
  - In other words, the interface to the hardware itself is data-parallel
  - **Hardware** (not compiler) is responsible for simultaneously executing the same instruction from multiple instances on different data on SIMD ALUs
- SIMD width of modern GPUs is 8 to 32 floats
  - Divergence (conditional execution) can be a big issue

# GPUS: THROUGHPUT-ORIENTED PROCESSORS




SIMD function unit
Control shared across 32 units

- Instructions operates on 32 pieces of data at a time (called “warps”)
  - ALUs process two data pieces per clock cycle
- Warp ≡ thread issuing 32-wide vector instructions
- Different instructions from up to 4 warps can be executed simultaneously (SMT)
- Up to 64 warps are simultaneously interleaved

# LIMITATIONS OF ILP: SOLUTIONS

- ILP works great! … But is complex & hard to scale
- Besides, all instruction scheduling work is done by the CPU with the help of the compiler
  - **The programmer does not worry about exploiting ILP**

Solutions to limitations of ILP:

**SIMD (Vector) Processing**

- Amortizing cost/complexity of managing an instruction stream across many ALUs
  - It allows to decrease both instruction scheduling and decoding logic

**Multithreaded Processing**

- Add hardware support to manage multiple threads of execution
  - It allows to mix instructions from different streams, probably independent

# PARALELISMO MULTI-HILO

## THREAD-LEVEL PARALLELISM

TEMA 5: EXPLOTACIÓN DE PARALELISMO

# PARALLELISM EVERYWHERE

*Software*

- Parallel Requests
  - Assigned to computer
  - e.g. search “Garcia”
- Parallel Threads We are here
  - Assigned to core
  - e.g. lookup, ads
- Parallel Instructions
  - > 1 instruction @ one time
  - e.g. 5 pipelined instructions
- Parallel Data
  - > 1 data item @ one time
  - e.g. add of 4 pairs of words
- Hardware descriptions
  - All gates functioning in parallel at same time

*Leverage Parallelism & Achieve High Performance*

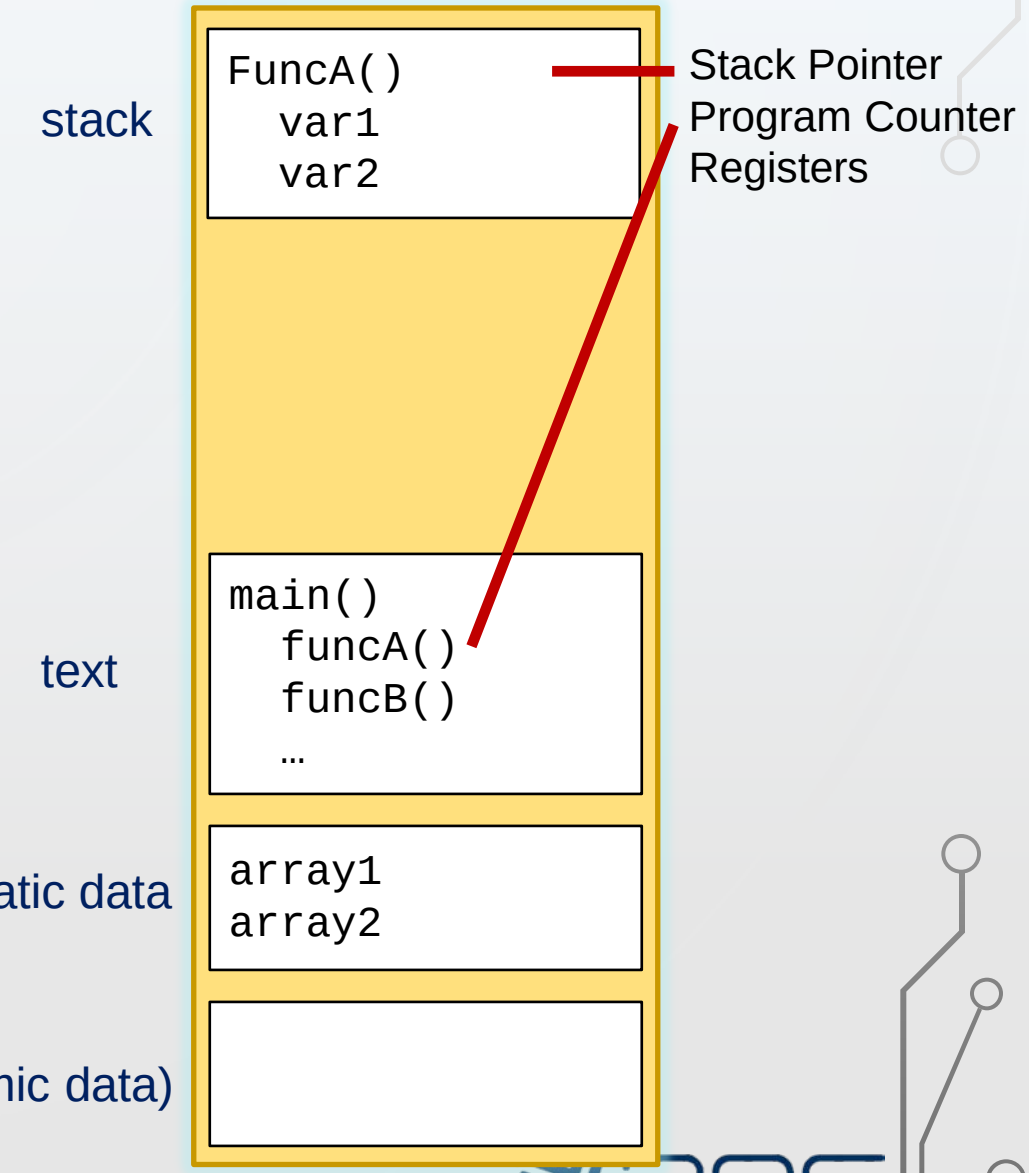
*Hardware*

Warehouse Scale Computer

Smart Phone

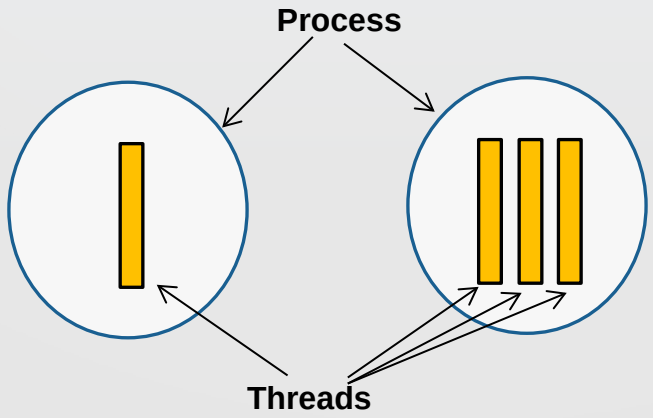
# PROCESS CONCEPT

- Program:
  - description of how to perform an activity (algorithm)
  - instructions and static data values
  - static file (image)
- Process:
  - a program in execution; process execution must progress in sequential fashion
  - a snapshot of a program in execution
  - an instance of a program running in a computer



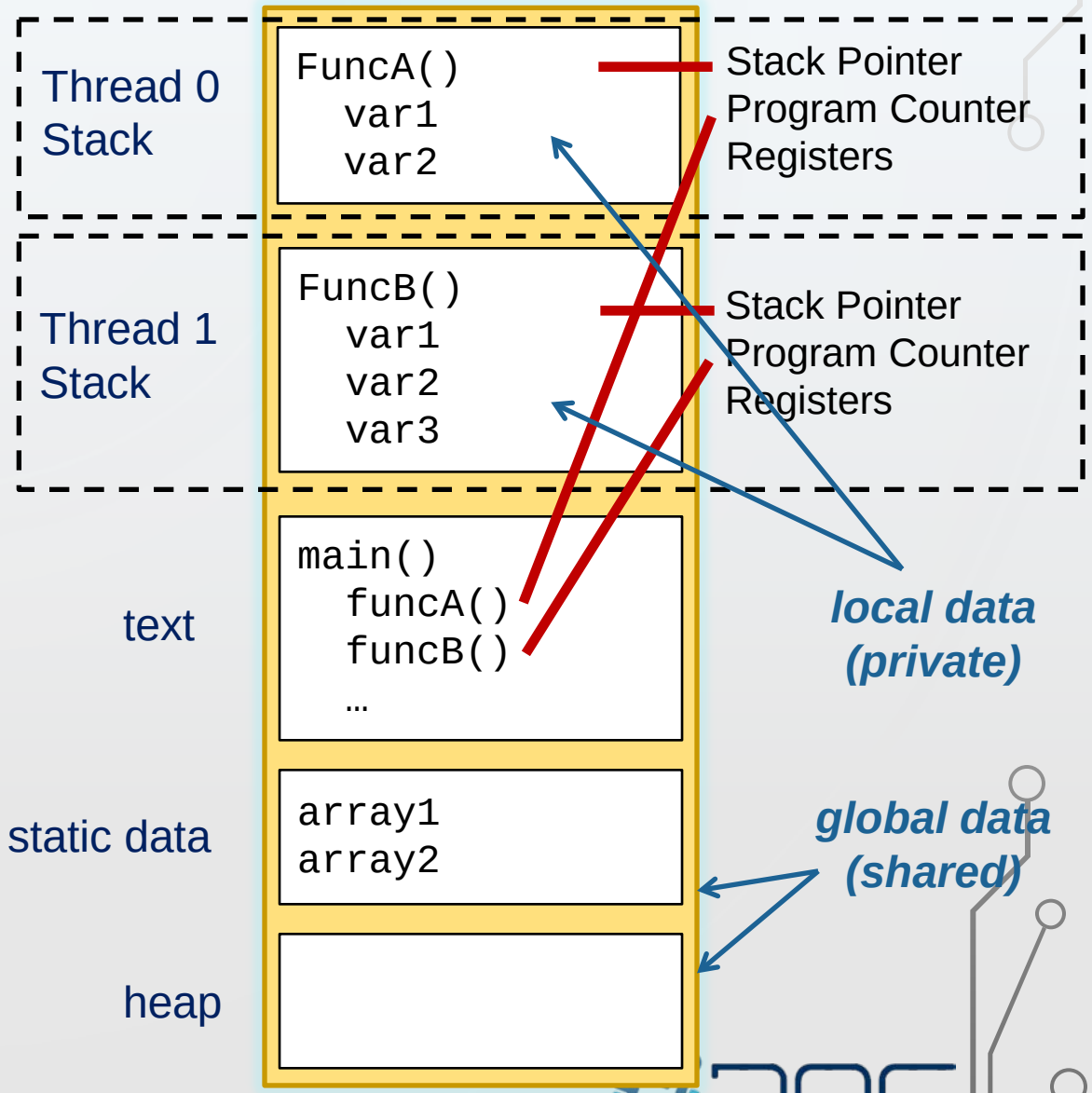
# THREAD CONCEPT

- Idea: Split the process into two entities
  - Thread: only the unit of  Scheduling/Execution (control flow)
  - Process: the unit of Resource ownership + thread
- More than one control flow (**thread**) "live" in a same address space
  - It is allowed several executions with the same program (code and data) and OS resources

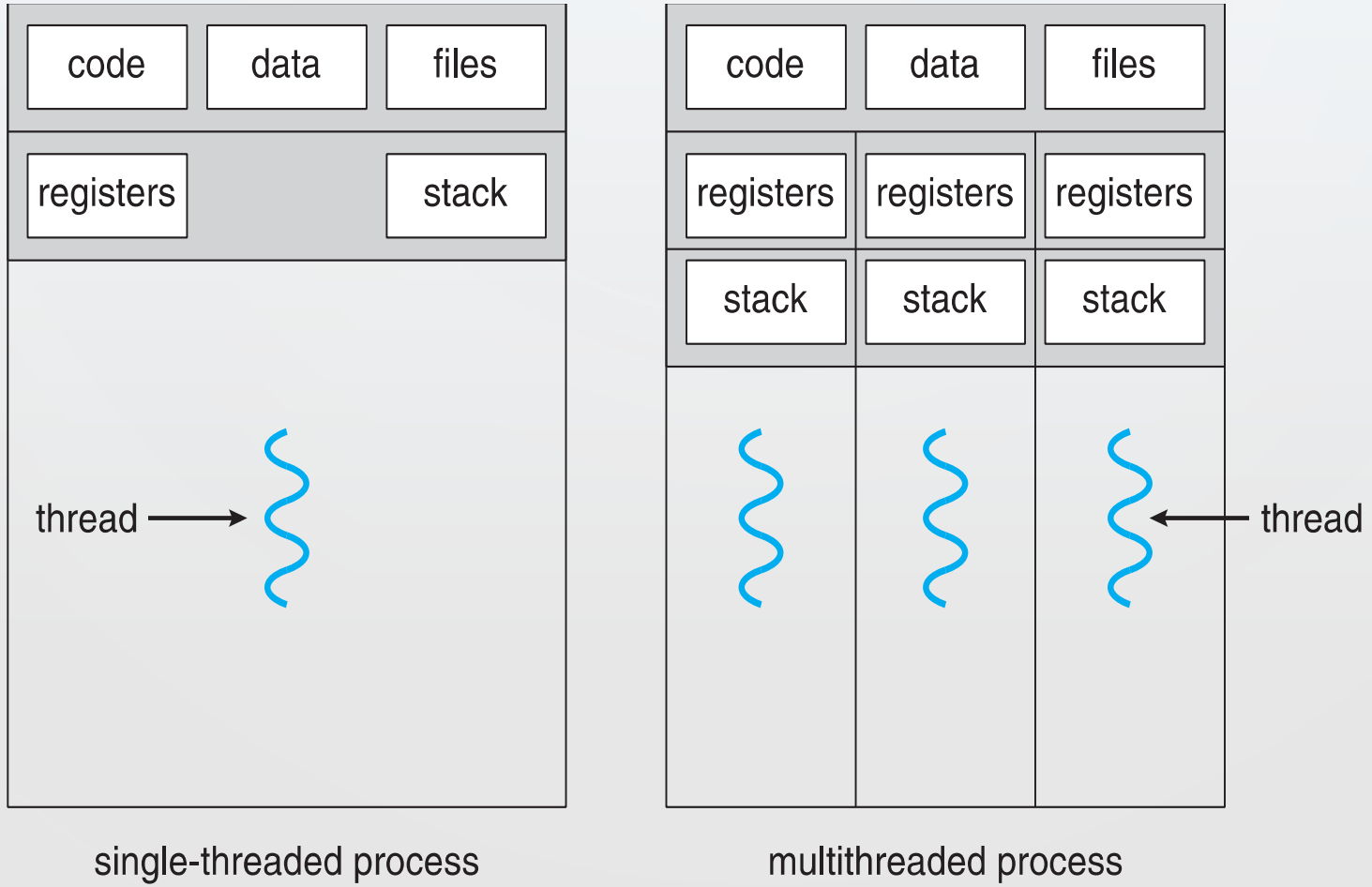


# THREAD DEFINITION

- A *thread* is the entity within a process that can be scheduled for execution
- A thread is sometimes called a **lightweight process**
- Each process is started with a single thread, but can create additional threads from any of its threads
  - **A process that has multiples threads can do more than one task at a time**



# SINGLE AND MULTITHREADED PROCESSES




single-threaded process

multithreaded process

# MULTITHREADED PROCESSING

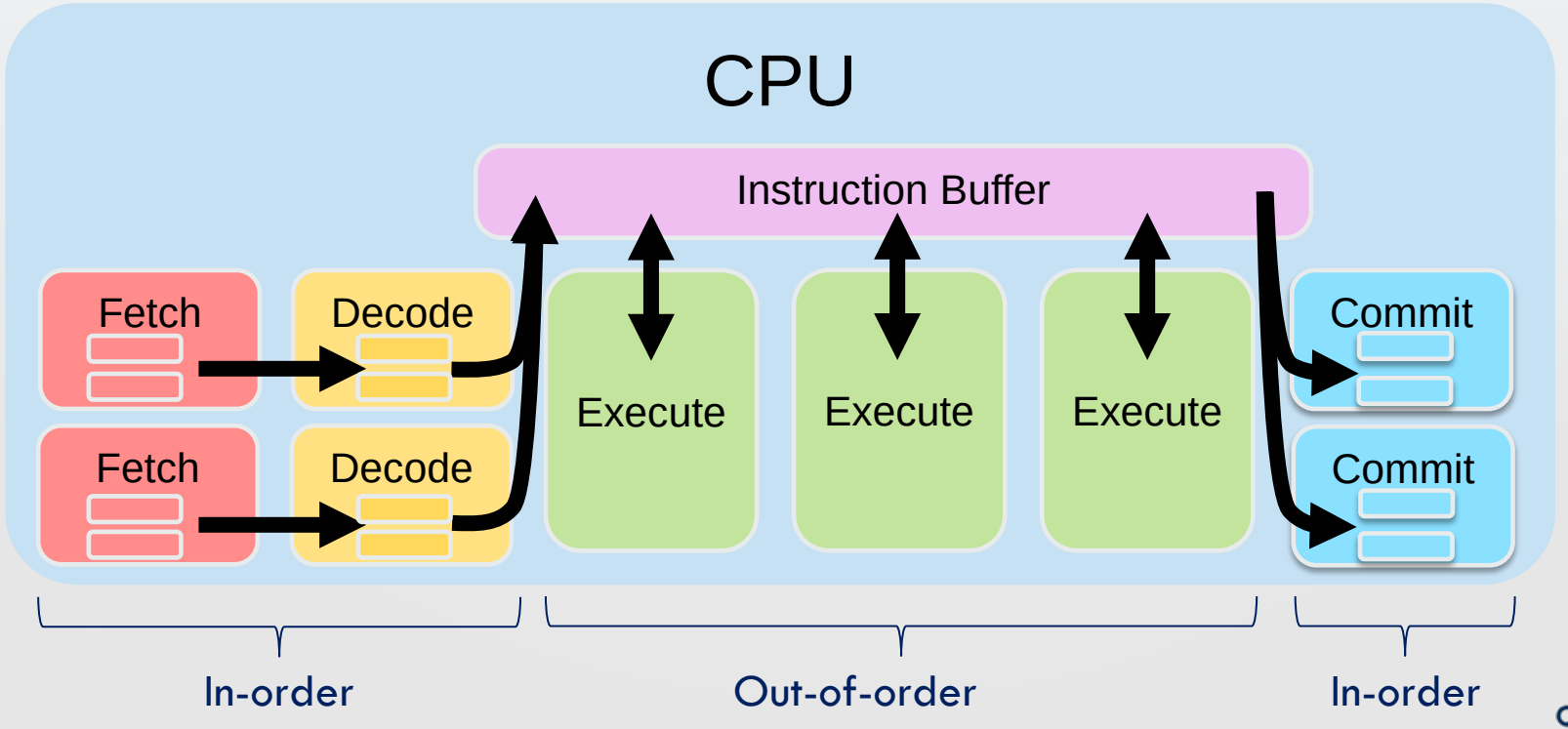
- Thread-Level parallelism
  - Uses MIMD model
- Several types of threads:
  - **Software threads**: threads under the control of the operating system
  - In contrast, **hardware threads** are hardware-supported instruction streams
- Threads can be found:
  - Full OS process
    - Software thread (OS thread)
  - Programmer-defined thread
  - Compiler-generated threads
  - Hardware threads

# MULTITHREADED PROCESSING

- Advantage of multithreading
  - It permits to mix inside the CPU, instructions from different threads, which are probably independent (no data/control hazards)
- However, compiler and hardware (CPU) are very limited to generate threads
- Application threads are usually defined by the **programmer**
- Multithreaded software is hard to develop!

# MULTITHREADED SUPERSCALAR OoO CPU MODEL

- Extension of the superescalar OoO model to add hardware support to the concurrent execution of multiple threads
  - Hardware must track instructions from different threads



# CLASSIFICATION OF HW-SUPPORTED MULTITHREADED PROCESSING

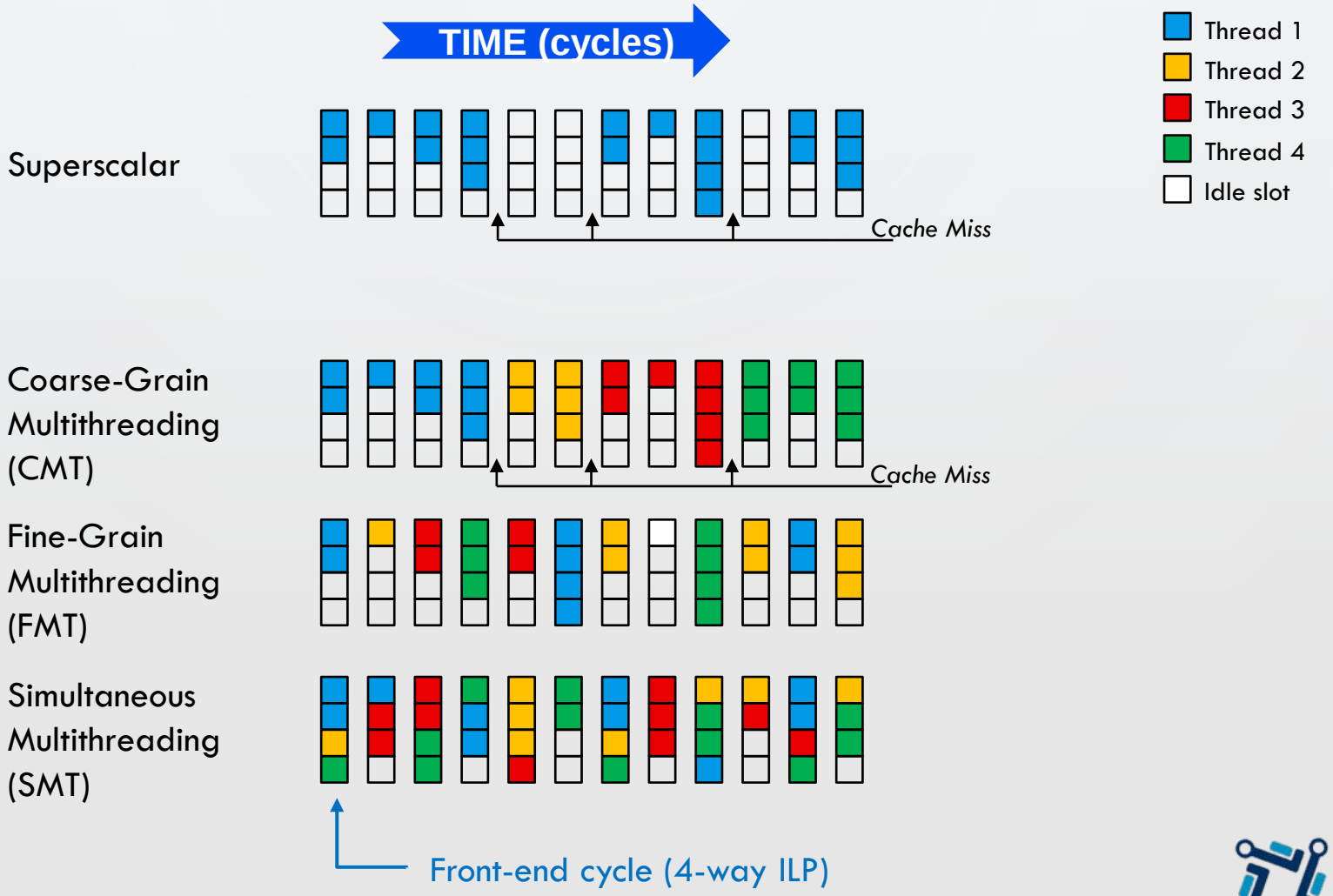
- Depending on how the instructions are scheduled:
- Instructions are issued from only a single thread in a given cycle
  - **Blocked Multithreading** or **Coarse-Grained Multithreading (CMT)**
    - The instructions of a thread are executed successively until an event occurs that may cause latency
    - This event induces a context switch
  - **Interleaved Multithreading** or **Fine-Grained Multithreading (FMT)**
    - An instruction of another thread is fetched and fed into the execution pipeline at each processor cycle

# CLASSIFICATION OF HW-SUPPORTED MULTITHREADED PROCESSING

- Depending on how the instructions are scheduled:
- Instructions are issued from multiple threads in a given cycle
  - **Simultaneous Multithreading (SMT)**
    - Instructions are simultaneously issued from multiple threads to the execution units

# MULTITHREADED PROCESSING

- Comparison



# MULTITHREADED PROCESSING: SMT

- SMT allows the best use of CPU resources compared to CMT and FMT
  - In fact, SMT transforms TLP into ILP and allow to execute multiple threads on an ILP processor with small extra hardware
  - However, that causes great complexities in instruction scheduling
  - As a result, SMT was generally implemented supporting 2 or 4 hardware threads
    - Example: Intel hyperthreading supports 2 hardware threads

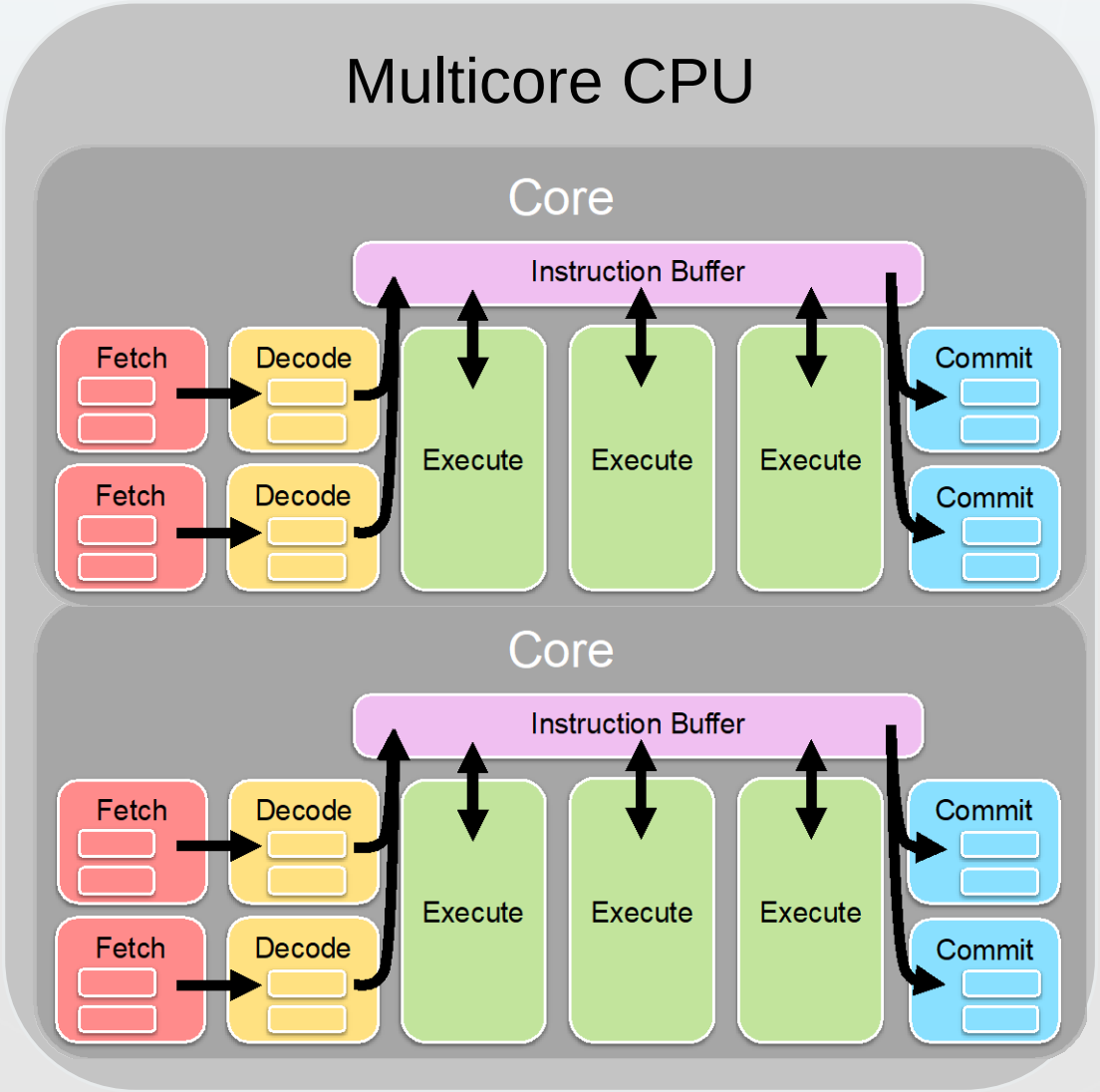
Solution to limitations of SMT:

## Multicore Processing

- Amortizing cost/complexity of instruction scheduling across two or more simpler control/execution units (cores)

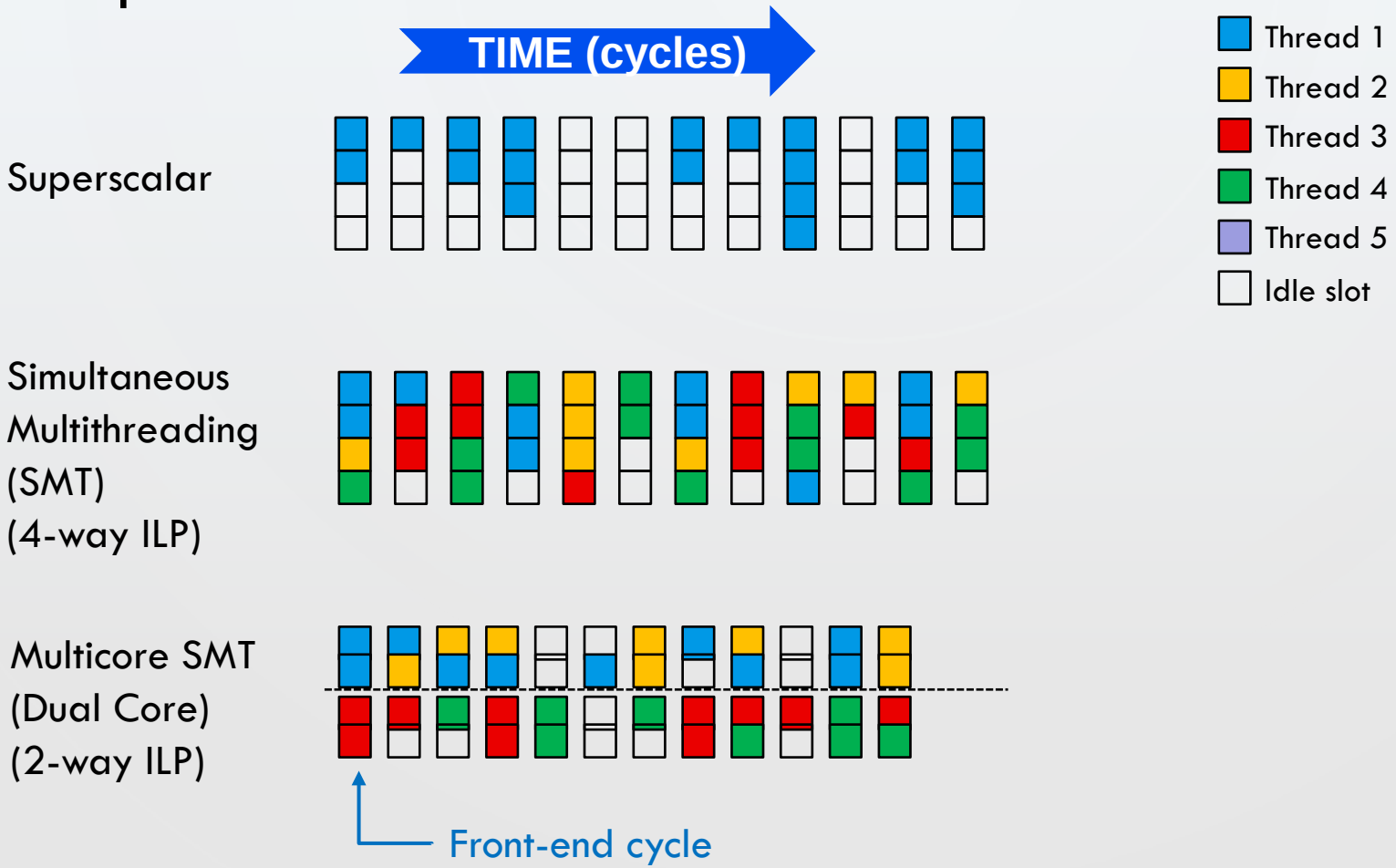
# MULTICORE CPU MODEL

- Replacing the big complex processor by several simpler ones



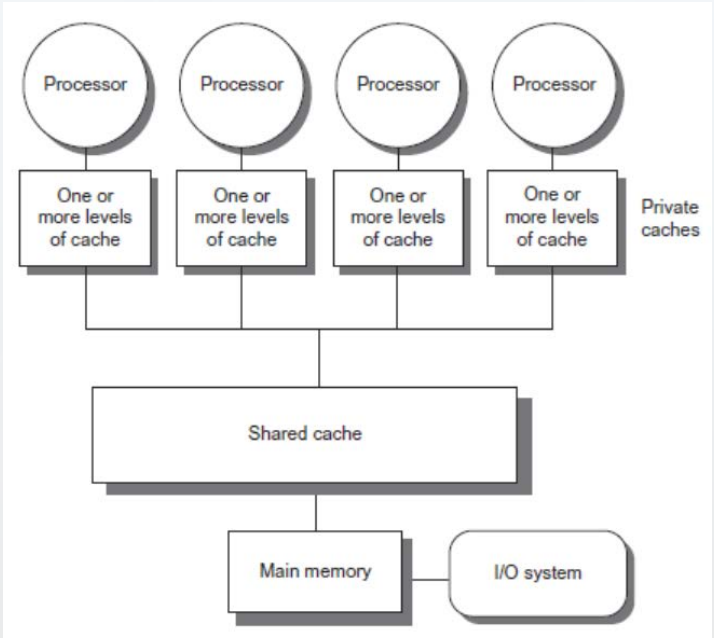
# MULTICORE PROCESSING

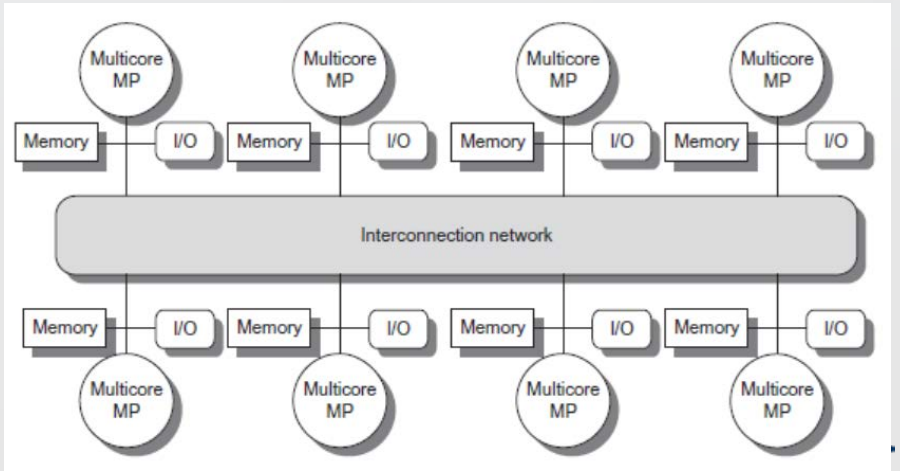
- Comparison



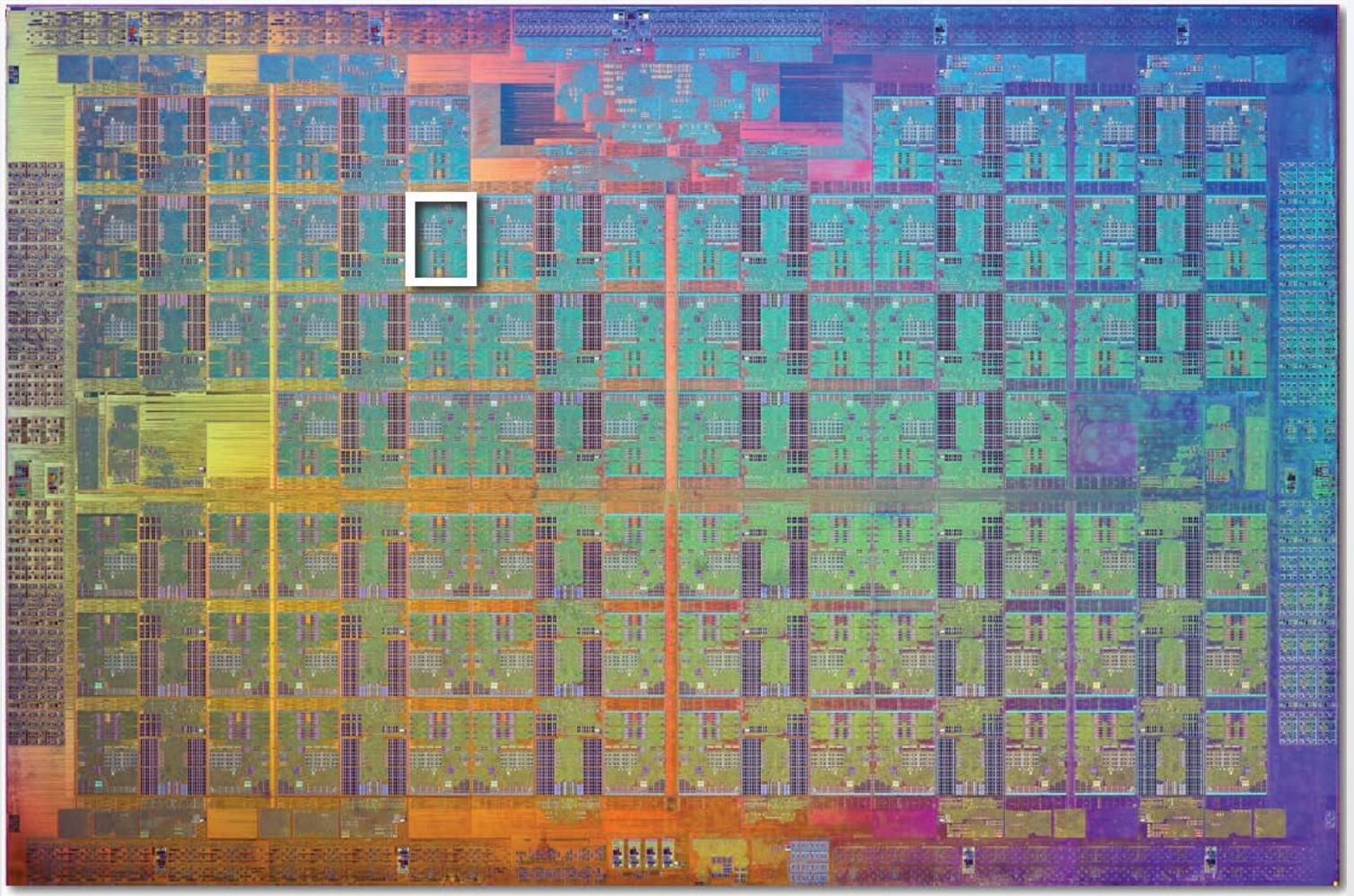
# TYPES OF MULTICORE ARCHITECTURES

- Symmetric multiprocessors (SMP)
  - Small number of cores
  - Share single memory with uniform memory latency
- Distributed shared memory (DSM)
  - Memory distributed among processors
  - Non-uniform memory access/latency (NUMA)
  - Processors connected via direct (switched) and non-direct (multi-hop) interconnection networks



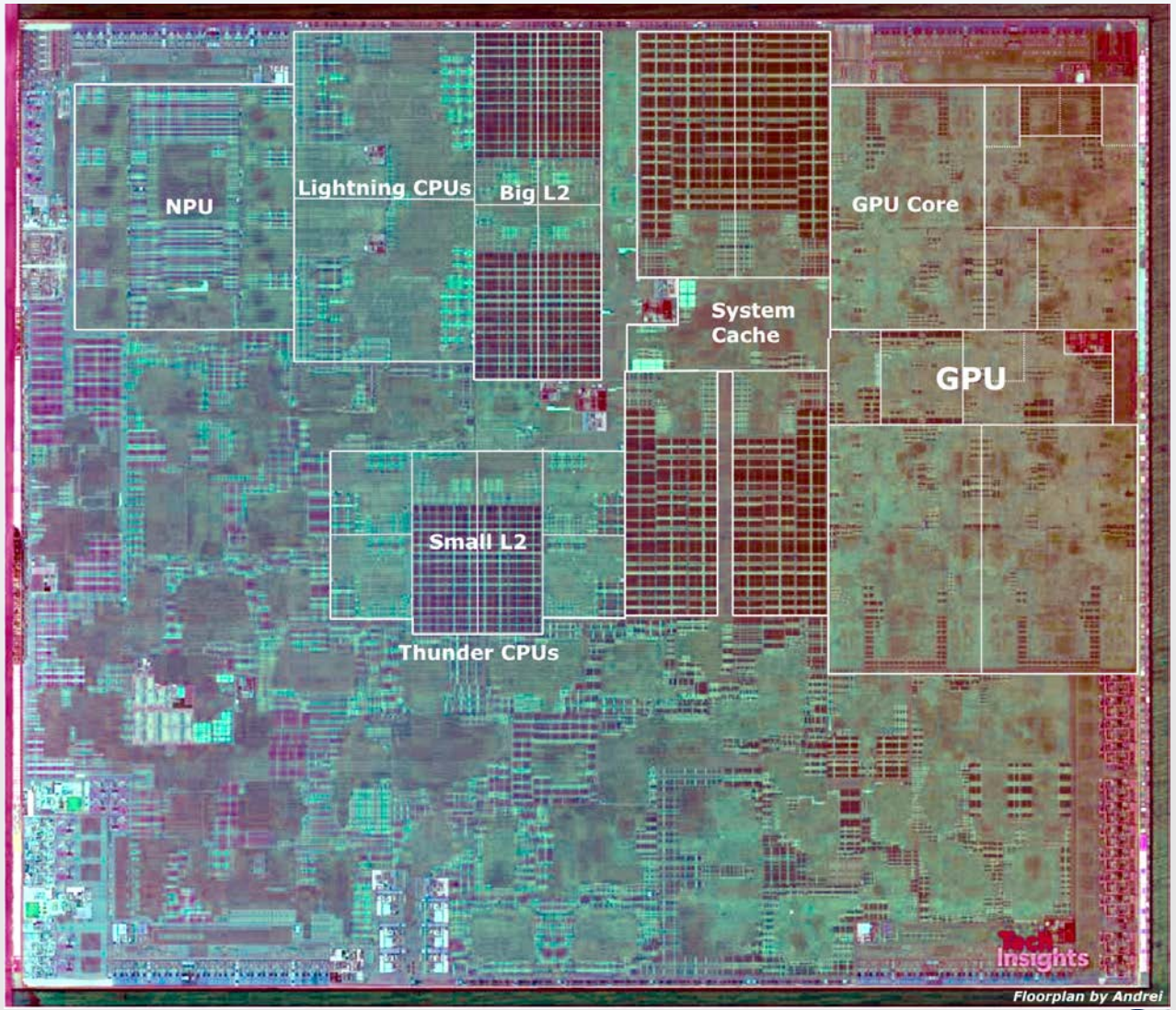
# MULTICORE EXAMPLES



Intel Xeon Phi "Knights Landing" 76-core CPU
(2015)

# MULTICORE EXAMPLES

**Apple A13 SoC**
**2 high-performance cores**
**4 energy-efficient cores**
**4-core GPU**
**8-core NPU**
**(2019)**



# PROGRAMACIÓN PARALELA

## PARALLEL PROGRAMING

TEMA 5: EXPLOTACIÓN DE PARALELISMO

# PROGRAM EXPRESSES NO PARALLELISM

- This program, after compiled, will run as one thread on one of the processor cores
  - If each of the simpler cores is 25% slower than the original single complicated one, the program now runs 25% slower

```
void sinx(int N, int terms, float* x, float* res)
{
  for (int i=0; i<N; i++)
  {
    float value = x[i];
    float numer = x[i]*x[i]*x[i];
    int denom = 6;  // 3!
    int sign = -1;

    for (int j=1; j<=terms; j++)
    {
      value += sign*numer/denom;
      numer *= x[i]*x[i];
      denom *= (2*j+2)*(2*j+3);
      sing *= -1;
    }

    res[i] = value;
  }
}
```

In contrast to ILP, **NO** hardware control to obtain several threads from a single instruction stream is available

# WE NEED TO EXPRESS PARALLELISM

- Automatic parallelization
  - Parallelism is extracted by a compiler (with the help of a library and runtime system)
  - Limitations:
    - Compiler does not know the problem, only the program
    - Compiler techniques only successful for regular programs (data dependences are easy to compute)
- Manual (programmer) parallelization
  - Programmer can **annotate** the code with compiler directives
  - Programmer can completely **rewrite** the code in parallel
    - Using a parallel language, with parallel constructs
    - Using a separated library to express parallelism

# PARALLEL PROGRAM USING DIRECTIVES

- **OpenMP**
  - Directives to express parallelism

```
void sinx(int N, int terms, float* x, float* res)
{
#pragma omp parallel for
  for (int i=0; i<N; i++)
  {
    float value = x[i];
    float numer = x[i]*x[i]*x[i];
    int denom = 6;  // 3!
    int sign = -1;

    for (int j=1; j<=terms; j++)
    {
      value += sign*numer/denom;
      numer *= x[i]*x[i];
      denom *= (2*j+2)*(2*j+3);
      sing *= -1;
    }

    res[i] = value;
  }
}
```

# PARALLEL PROGRAM USING A LIBRARY

- **Pthreads**
  - Library for thread management

```
typedef struct {
  int N;
  int terms;
  float *x;
  float *res;
} my_args;

void parallel_sinx(int N, int terms, float* x, float*
res)
{
  pthread_t thread_id;
  my_args args;

  args.N = N/2;
  args.terms = terms;
  args.x = x;
  args.res = res;

  pthread_create(&thread_id, NULL, my_thread_start,
&args);
  sinx(N-args.N, terms, x+args.N, res+args.N);
  pthread_join(thread_id, NULL);
}

void my_thread_start(void* thread_arg)
{
  my_args th_args = (my_args)thread_arg;
  sinx(th_args.N, th_args.terms, th_args.x,
th_args.res);
}
```

```
void sinx(int N, int terms, float*
x,
float* res)
{
  for (int i=0; i<N; i++)
  {
    float value = x[i];
    float numer = x[i]*x[i]*x[i];
    int denom = 6;  // 3!
    int sign = -1;

    for (int j=1; j<=terms; j++)
    {
      value += sign*numer/denom;
      numer *= x[i]*x[i];
      denom *= (2*j+2)*(2*j+3);
      sing *= -1;
    }

    res[i] = value;
  }
}
```

# SUMMARY: PARALLEL EXECUTION

- Several forms of parallel execution in modern processors:
- **Superscalar**: Exploit ILP, processing different instructions from the same instruction stream in parallel (within a core)
  - Parallelism automatically and dynamically discovered by the hardware during execution (not programmer visible)
- **SIMD**: Use multiple ALUs controlled by same instruction stream (within a core)
  - Efficient: control amortized over many ALUs; useful for data-parallel workloads
  - Vectorization done by compiler (explicit SIMD) or at runtime by hardware
  - Dependencies are known prior to execution
- **Multicore**: Use of multiple processing cores
  - Provides **thread-level parallelism (TLP)**: simultaneously execute completely different instruction streams on each core
  - Provides (modest) **simultaneous multithreading (SMT)** in each core
  - **Software** decides when to create threads (e.g., via Pthreads API)